Modern C: A Concise Introduction to C23

Independent technical-review draft

October 2026

Contents

Preface **9**
- Audience . . . 10
- What modern means . . . 10
- How to use the examples . . . 10
- Conventions . . . 10
- An author's checklist for each chapter . . . 11
- Reading path . . . 11

1. A First Look at C **11**
- 1.1 A complete program . . . 11
- 1.2 Variables and arithmetic . . . 11
- 1.3 Characters and the input stream . . . 12
- 1.4 An array and a simple histogram . . . 12
- 1.5 Functions as named operations . . . 13
- 1.6 Counting words: states rather than lines . . . 13
- 1.7 When the simplest program is not enough . . . 14
- 1.8 Reading a program from the outside inward . . . 14
- 1.9 Trace the loop, not the syntax alone . . . 14
- 1.10 Programs as small transformations . . . 15
- 1.11 The execution path from source file to program . . . 15
- 1.12 A state machine for byte-oriented input . . . 15
- 1.13 Three kinds of correctness . . . 16
- 1.14 Choosing types from contracts . . . 16

2. Types, Expressions, and Control **16**
- 2.1 Integer types and ranges . . . 16
- 2.2 Conversion is an operation . . . 17
- 2.3 Conditions and loops . . . 17
- 2.4 Short circuit and sequencing . . . 17
- 2.5 Floating point . . . 17
- 2.6 C23 spelling and initialization . . . 17
- 2.7 Integer conversion in practice . . . 18
- 2.8 Bitwise operations . . . 18
- 2.9 Evaluation order is independent of precedence . . . 18
- 2.10 A checked average . . . 19
- 2.11 Representable values before casts . . . 19
- 2.12 Division, remainder, and a dangerous minimum . . . 19
- 2.13 Floating-point accuracy is a contract . . . 20
- 2.14 A precise conversion exercise . . . 20
- 2.15 Promotions are part of the computation . . . 20

2.16 Comparisons between signed and unsigned values . . . 21
2.17 Division edge cases . . . 21
2.18 Floating-point comparisons require a scale . . . 21
2.19 Derive overflow checks rather than guessing . . . 22

3. Functions and Program Structure 22
3.1 Definitions, declarations, and headers . . . 22
3.2 Scope and lifetime . . . 23
3.3 Recursion . . . 23
3.4 Design by contract . . . 23
3.5 Out-parameters and explicit failure . . . 23
3.6 Header contracts and linkage . . . 24
3.7 Recursion versus iteration . . . 24
3.8 Exercises . . . 24
3.9 Choose the shape of an interface . . . 25
3.10 Function declarations are contracts across files . . . 25
3.11 A fully checked factorial . . . 25
3.12 Functions and test seams . . . 26
3.13 Separate syntax checking from semantic validation . . . 26
3.14 Return values and out-parameters . . . 26
3.15 File scope, linkage, and modules . . . 27
3.16 Passing arrays does not pass their sizes . . . 27
3.17 Recursive functions need a resource argument . . . 27

4. Pointers, Arrays, and Strings 28
4.1 Address and indirection . . . 28
4.2 Array arguments carry no length . . . 28
4.3 Strings are terminated byte sequences . . . 28
4.4 Copying bounded text . . . 29
4.5 Constness and pointer arithmetic . . . 29
4.6 A pointer is not an array descriptor . . . 29
4.7 Two-dimensional arrays . . . 30
4.8 Lifetime is as important as address . . . 30
4.9 Restrict and aliasing . . . 30
4.10 Exercises . . . 30
4.11 Three meanings of an array name . . . 31
4.12 One past the end is not an element . . . 31
4.13 Copy, move, and overlap . . . 31
4.14 String ownership and views . . . 31
4.15 Draw the objects before drawing pointers . . . 32
4.16 A counted slice . . . 32
4.17 Derive a subslice without overflow . . . 33
4.18 Const qualifies a path of access . . . 33
4.19 Overlap and copying . . . 33
4.20 Pointer lifetime can end without the pointer value changing . . . 33

5. Structures and Data Representation 34
5.1 A value type . . . 34
5.2 Tagged unions . . . 34
5.3 Linked records . . . 34
5.4 Bit fields and explicit masks . . . 35
5.5 Field order, padding, and ABI . . . 35
5.6 A bounded ring buffer . . . 35
5.7 Opaque structures as a design choice . . . 36
5.8 Exercises . . . 36
5.9 Structure layout as a local implementation detail . . . 36

5.10 Flexible array members . . . 37
5.11 Layout-independent serialization . . . 37
5.12 Data-structure choice follows requirements . . . 37
5.13 A structure is a value with invariants . . . 38
5.14 Padding and equality . . . 38
5.15 Tagged unions and exhaustiveness . . . 38
5.16 Intrusive versus owning lists . . . 39
5.17 Ring buffer wraparound . . . 39

6. Input, Output, and Files **39**
6.1 Reading lines safely . . . 39
6.2 Formatted I/O . . . 40
6.3 Binary I/O . . . 40
6.4 Files and cleanup . . . 40
6.5 The difference between text and bytes . . . 40
6.6 A bounded line counter . . . 41
6.7 Reading numeric text . . . 41
6.8 File cleanup paths . . . 41
6.9 Exercises . . . 42
6.10 A byte-stream copier . . . 42
6.11 Why parsing after input is often easier . . . 42
6.12 Error reporting preserves the cause . . . 42
6.13 Buffering and interleaving . . . 42
6.14 Designing a testable input function . . . 43
6.15 Streams, records, and short transfers . . . 43
6.16 Write operations have partial-success semantics . . . 43
6.17 Use a parser independent of the transport . . . 43
6.18 Standard output and error output . . . 44
6.19 A defensive byte-count example . . . 44

7. Dynamic Memory and Ownership **44**
7.1 Allocation is a request . . . 44
7.2 One owner, one release . . . 45
7.3 Resize without losing the original . . . 45
7.4 Fixed-capacity alternatives . . . 45
7.5 A resource table . . . 45
7.6 A checked growable vector . . . 46
7.7 An arena with an explicit alignment policy . . . 46
7.8 Failure-safe initialization . . . 46
7.9 Exercises . . . 47
7.10 Distinguish capacity from initialized length . . . 47
7.11 Zero-size requests in C23 . . . 47
7.12 Checked growth without losing storage . . . 47
7.13 Alignment and arenas . . . 47
7.14 Failure injection as a learning tool . . . 48
7.15 Ownership as an interface contract . . . 48
7.16 Strong and basic failure guarantees . . . 48
7.17 A checked vector implementation . . . 49
7.18 How to use an arena safely . . . 49
7.19 Resource cleanup as a state machine . . . 49
7.20 Testing the allocator's failure paths . . . 49

8. The Preprocessor and Separate Compilation **50**
8.1 Headers describe interfaces . . . 50
8.2 Prefer typed operations . . . 50
8.3 Conditional compilation . . . 50

8.4 Compile and link . . . 51
8.5 What belongs in an interface? . . . 51
8.6 Macros and evaluation hazards . . . 51
8.7 Build modes and reproducibility . . . 51
8.8 Defensive configuration . . . 52
8.9 Exercises . . . 52
8.10 A source file is not an isolated program . . . 52
8.11 Build a tiny library deliberately . . . 52
8.12 Host and target are different builds . . . 52
8.13 Preprocessor conditions versus feature probes . . . 52
8.14 Keep generated output reproducible . . . 53
8.15 A real three-file C module . . . 53
8.16 Rebuild dependencies . . . 53
8.17 Header hygiene and include order . . . 54
8.18 Libraries are composed, not merely named . . . 54

9. The C23 Language **54**
9.1 Booleans and null pointers . . . 54
9.2 Type inference with auto and typeof . . . 54
9.3 Constant expressions . . . 55
9.4 Width-aware integers . . . 55
9.5 Binary literals and digit separators . . . 55
9.6 Attributes . . . 55
9.7 New preprocessor capabilities . . . 55
9.8 Checked arithmetic and bit utilities . . . 55
9.9 A migration policy . . . 55
9.10 Generic selection . . . 56
9.11 Safer use of `typeof` . . . 56
9.12 Checked arithmetic in C23 . . . 56
9.13 Selected portability cautions . . . 56
9.14 Exercises . . . 57
9.15 What actually changes when moving from C17 . . . 57
9.16 The boundary between auto and typeof . . . 57
9.17 Modernized enumerations and attributes . . . 57
9.18 Checked arithmetic belongs at boundaries . . . 57
9.19 C23 migration example . . . 58
9.20 A testable feature inventory . . . 58
9.21 A concrete subset of C23 . . . 58
9.22 Why not use every C23 feature in the first example? . . . 59
9.23 Initialization is not byte filling . . . 59
9.24 A language revision is a collection of independent features . . . 59
9.25 C23 integer literals and readable masks . . . 60
9.26 nullptr is a value, not an owner . . . 60
9.27 Type inference should not hide the contract . . . 60
9.28 Checked integer arithmetic in a C23 library . . . 60
9.29 Choosing what to modernize . . . 61

10. Reliable Systems Programming **61**
10.1 Undefined, unspecified, implementation-defined . . . 61
10.2 Check sizes before multiplying . . . 61
10.3 The limits of volatile . . . 62
10.4 Host tests and device tests . . . 62
10.5 Protocol parsing . . . 62
10.6 Review checklist . . . 62
10.7 Establish invariants before optimizing . . . 62
10.8 Bound both time and storage . . . 62

10.9 Concurrency and atomics . . . 62
10.10 Errors are part of the API . . . 63
10.11 Exercises . . . 63
10.12 A bounded packet decoder . . . 63
10.13 Preconditions, invariants, and postconditions . . . 63
10.14 A two-stage parser . . . 64
10.15 Safety is not security . . . 64
10.16 Bounded execution requires a system model . . . 64
10.17 Reviews use independent evidence . . . 64
10.18 A fault-injection exercise . . . 65
10.19 Defensive decoding: from bytes to a validated object . . . 65
10.20 Define three failure classes . . . 65
10.21 Error recovery is a protocol decision . . . 65
10.22 Worst-case resource accounting . . . 66

11. An Extended Example: A Word Index **66**
11.1 Requirements . . . 66
11.2 Implementation . . . 66
11.3 Complexity and extensions . . . 68
11.4 From linear search to a sorted index . . . 68
11.5 A table with explicit invariants . . . 68
11.6 Complexity, measurement, and design . . . 68
11.7 Additional exercises . . . 69
11.8 Designing the input grammar . . . 69
11.9 Proving the sorted insertion . . . 69
11.10 Complexity with meaningful constants . . . 69
11.11 Test cases that reveal design errors . . . 70
11.12 Extending the implementation responsibly . . . 70
11.14 A search algorithm with a proof obligation . . . 70
11.15 The cost of sorted insertion . . . 70
11.16 Error handling belongs inside the index . . . 71
11.17 Independently test the expected output . . . 71

12. Exercises and Further Reading **71**
12.1 A progression of projects . . . 71
12.2 How to study source code . . . 72
12.3 References . . . 72
12.4 A note on editions . . . 72

12.1 Exercises as specifications **72**
12.5 Testing boundaries . . . 72
12.6 Integration project: a streaming index . . . 72
12.7 Integration project: a bounded vector . . . 73
12.8 Reading C with a reviewer . . . 73
12.9 Challenge: a portable protocol reader . . . 73
12.10 Challenge: concurrency . . . 73
12.11 Further reading in practice . . . 73
12.12 How to write convincing solutions . . . 73
12.13 Progressive memory exercises . . . 74
12.14 Progressive text exercises . . . 74
12.15 A bounded data pipeline . . . 74
12.16 A portability investigation . . . 74
12.17 A review rubric . . . 74

13. Testing, Diagnostics, and Portability **75**
13.1 Start with the contract . . . 75

13.2 Organize tests by dimensions . . . 75
13.3 Assertions for tests, not input validation . . . 75
13.4 A test harness without external libraries . . . 76
13.5 Warnings and language modes . . . 76
13.6 Sanitizers . . . 76
13.7 Unit tests and property tests . . . 77
13.8 Fuzzing a parser . . . 77
13.9 Compare optimization modes . . . 77
13.10 Run the actual target . . . 78
13.11 Document verified claims . . . 78
13.12 Exercises . . . 78
13.13 Make tests deterministic . . . 78
13.14 Test failures, not just successes . . . 78
13.15 Static analysis and sanitizers complement each other . . . 79
13.16 Differential testing . . . 79
13.17 CI as reproducible evidence . . . 79
13.18 Choose a release criterion . . . 79
13.19 A test is a contract experiment . . . 79
13.20 Use fault injection to reach unlikely branches . . . 80
13.21 Test the smallest complete interface . . . 80
13.22 Measure coverage without worshipping a number . . . 80
13.23 Reproducible CI . . . 80

14. Bits, Bytes, and Portable Data **81**
14.1 Storage units . . . 81
14.2 Masks, shifts, and flags . . . 81
14.3 Endianness is part of the format . . . 82
14.4 Representation and alignment . . . 82
14.5 The meaning of volatile . . . 82
14.6 A complete byte codec . . . 82
14.7 Exercises . . . 83
14.8 Fixed-width data is not fixed layout . . . 83
14.9 Shifts need a domain proof . . . 83
14.10 Check the complete frame size . . . 83
14.11 MMIO is not just a pointer cast . . . 84
14.12 Field tests beat raw dumps . . . 84
14.13 A wire protocol is a specification, not a structure . . . 84
14.14 Encode without partial publication . . . 84
14.15 Decode in a safe order . . . 85
14.16 Endianness and alignment are independent . . . 85
14.17 Testing message boundaries . . . 85

15. Expressions, Objects, and the Abstract Machine **85**
15.1 Values and objects . . . 86
15.2 Storage duration and scope answer different questions . . . 86
15.3 Sequenced and unsequenced evaluations . . . 86
15.4 Integer overflow and conversions . . . 87
15.5 Effective type and aliasing . . . 87
15.6 Order, lifetime, and optimizers . . . 87
15.7 A small checked interval . . . 88
15.8 Exercises . . . 88
15.9 The optimizer reasons from the language rules . . . 88
15.10 Object representation versus value . . . 88
15.11 Sequence and indirection . . . 89
15.12 Bounds and pointer provenance . . . 89
15.13 A reasoning template for code reviews . . . 89

15.14 Draw the memory model before optimizing . . . 89
15.15 Distinguish value equality from byte equality . . . 90
15.16 Expression transformations need preconditions . . . 90
15.17 Observable behavior and the optimizer . . . 90

16. Atomics, Threads, and Communication 90
16.1 A race on a counter . . . 90
16.2 Publication with release and acquire . . . 91
16.3 A bounded single-producer/single-consumer queue . . . 91
16.4 Why index wraparound is not numeric overflow . . . 92
16.5 Lifetime and shutdown . . . 92
16.6 Mutexes remain valuable . . . 92
16.7 C23 and implementation support . . . 92
16.8 Exercises . . . 92
16.9 Identify the exact shared objects . . . 93
16.10 Understanding the release/acquire pair . . . 93
16.11 Sequential tests do not demonstrate concurrency correctness . . . 93
16.12 Interrupts are outside a generic hosted-thread proof . . . 93
16.13 Lifetime and cancellation . . . 94
16.14 Memory ordering is a protocol, not a performance switch . . . 94
16.15 Read the single-producer guarantee literally . . . 94
16.16 Progress guarantees are different . . . 94
16.17 Shutdown and ownership . . . 95

17. Function Pointers, Callbacks, and Interfaces 95
17.1 A function pointer . . . 95
17.2 A callback with context . . . 95
17.3 A stateful visitor . . . 96
17.4 Sorting and the comparison contract . . . 96
17.5 Function pointers are not object pointers . . . 96
17.6 Interfaces as boundaries . . . 97
17.7 Costs and limits . . . 97
17.8 Exercises . . . 97
17.9 Make callback contracts explicit . . . 97
17.10 Context lifetime and asynchronous operations . . . 98
17.11 Design a complete-write adapter . . . 98
17.12 qsort and ordering . . . 98
17.13 Interfaces and performance . . . 98
17.14 A practical byte-sink interface . . . 98
17.15 Handling partial writes . . . 99
17.16 Test using a fake device . . . 99
17.17 Lifetime and reentrancy . . . 99

18. Text, Encodings, and Robust Parsers 100
18.1 The byte is not the character . . . 100
18.2 Parse lengths explicitly . . . 100
18.3 Decimal parsing as a checked transition . . . 100
18.4 A bounded UTF-8 decoder . . . 100
18.5 Failure and incomplete input are distinct . . . 101
18.6 The language's execution character set . . . 101
18.7 A secure parsing checklist . . . 101
18.8 Exercises . . . 101
18.9 UTF-8 validity is more than continuation bytes . . . 102
18.10 Character count versus display width . . . 102
18.11 Incremental decoding as a state machine . . . 102
18.12 Parsing machine-oriented text . . . 102

18.13 Validate before normalizing . . . 102
18.14 Separate three kinds of text boundaries . . . 103
18.15 Build the UTF-8 validity proof . . . 103
18.16 Avoid accidental locale dependencies . . . 103
18.17 Bounded streaming text . . . 103

19. Compile-Time Programming and the Preprocessor 104
19.1 Include files and translation units . . . 104
19.2 Object-like macros . . . 104
19.3 Function-like macros and double evaluation . . . 104
19.4 Stringizing and token pasting . . . 104
19.5 Conditional compilation . . . 105
19.6 C23 embedding and diagnostics . . . 105
19.7 Static assertions . . . 105
19.8 Generic selection . . . 105
19.9 An example of compile-time checks . . . 106
19.10 Exercises . . . 106
19.11 Preprocessing is not type checking . . . 106
19.12 Expand macros deliberately . . . 106
19.13 Conditional compilation and dead branches . . . 106
19.14 Using embed responsibly . . . 107
19.15 Static assertions expose assumptions early . . . 107
19.16 Build diagnostics as a teaching aid . . . 107
19.17 A preprocessor macro is not a generic function . . . 107
19.18 Use static assertions for real assumptions . . . 107
19.19 Configuration is part of the reproducible build . . . 108
19.20 Keep the generated book reproducible . . . 108

Appendix A. A Working Reference to C23 108
A.1 Translation model . . . 108
A.2 Fundamental types . . . 108
A.3 Expressions at a glance . . . 109
A.4 Declarations . . . 109
A.5 Storage, linkage, and qualifiers . . . 109
A.6 Control statements . . . 110
A.7 Behavior categories . . . 110
A.8 Selected C23 additions . . . 110
A.9 A disciplined compatibility check . . . 110
A.10 A compact declaration guide . . . 110
A.11 Conversions checklist . . . 110
A.12 Alignment and lifetime at a glance . . . 111
A.13 C23 compatibility notes . . . 111
A.14 Selected undefined operations . . . 111
A.15 References for the careful reader . . . 111

Appendix B. The Standard Library by Task 112
B.1 Length is not capacity . . . 112
B.2 Integer conversion . . . 112
B.3 Byte copy versus overlapping copy . . . 112
B.4 Character conversion . . . 112
B.5 Optionality and freestanding environments . . . 113
B.6 A checked numeric conversion . . . 113
B.7 Formatted-output contracts . . . 113
B.8 Searching and sorting . . . 113
B.9 Allocation and zeroed storage . . . 114
B.10 C23 bit utilities, precisely . . . 114

B.11 Platform limitations . . . 114
B.12 Error policy examples . . . 114

Appendix C. Selected Worked Exercises 114
C.1 Absolute difference without unsigned underflow . . . 114
C.2 Checked parsing of an unsigned decimal . . . 115
C.3 Iterative factorial with an overflow check . . . 115
C.4 Little-endian byte decoding . . . 116
C.5 Cleanup on every path . . . 116
C.6 A safe interval check . . . 116
C.7 Checked allocation dimensions . . . 116
C.8 A consistent qsort comparator . . . 117
C.9 A non-modifying failure test . . . 117
C.10 A boundary test plan for a ring buffer . . . 117
C.11 How to check an exercise solution . . . 118
C.12 A counted slice and its ownership contract . . . 118
C.13 Failure-injected vector growth . . . 118
C.14 Explain a failing test . . . 119
C.15 A bounded frame encoder and decoder . . . 119
C.16 A partial-write callback . . . 119
C.17 Separate compilation as a testable contract . . . 120
C.18 What an example demonstrates—and what it does not . . . 120
C.19 Review checklist for the three complete examples . . . 120

Appendix D. Glossary of Modern C 121

Appendix E. Conformance and Migration Checklist 122
E.1 Identify the build . . . 123
E.2 Core-language probes . . . 123
E.3 Standard-library probes . . . 123
E.4 Representation assumptions . . . 123
E.5 A practical migration order . . . 123
E.6 Deployment acceptance checklist . . . 124
E.7 Authoritative resources . . . 124

Preface

C is a compact language, but mastering it is not the same as memorizing a compact set of keywords. The interesting questions begin where a program meets actual storage, input, timing, or another component: Who owns this buffer? How large is it? What happens when conversion overflows? Which operations are portable? What can another thread observe? A useful modern introduction should make those questions part of ordinary programming practice.

This book presents C23 through small programs and progressively more demanding examples. The chapters begin with stream processing and basic expressions, proceed to functions, arrays, structures, memory allocation and compilation, and then develop the correctness concerns that matter in systems software. Later chapters explore C23 additions, binary protocols, concurrency, callbacks, text encodings, and testing.

It is intentionally an **independent original textbook**. Its concise presentation and example-first progression acknowledge the value of classic C pedagogy, but this is not a third edition of *The C Programming Language* and is not affiliated with its authors or publisher. The examples, explanations, and exercises here are newly written.

Audience

The intended reader has some programming experience but need not have used C professionally. A reader comfortable with another language can move quickly through the opening material. A reader new to pointers and manual storage management should slow down, compile every complete example, and work through the boundary-case exercises.

The later chapters are especially relevant to systems and embedded engineers. They discuss host-versus-target portability, fixed-capacity data structures, interrupt-related limitations, and memory-ordering contracts. They do not assume that every MCU provides the full hosted C library, filesystem, heap, or threads.

What modern means

C23 is the 2024 publication of ISO/IEC 9899:2024, often described as the 2023 revision. Modern C encompasses both the updated language and a more explicit engineering discipline: prototypes rather than implicit declarations, documented ownership, checked sizes, controlled resource lifetimes, clean module interfaces, and tests that challenge failure paths.

Some C23 features have uneven implementation support. A compiler can recognize `-std=c23` without supplying every related library header or freestanding runtime facility. This manuscript therefore distinguishes language rules, implementation-defined properties, and deployment-specific services. It uses modern constructs where they clarify intent, but does not pretend one compiler flag can make every old target conform completely.

How to use the examples

Listings fall into two categories. A **complete program** has its own translation unit and entry point and may be compiled and run. An **illustrative fragment** shows a specific language rule or API contract and may omit enclosing declarations, headers, and supporting functions. The standalone programs are kept in the repository's `examples/` directory; the Makefile and test scripts document how they are built.

A useful study cycle is to predict the program's result, compile it with warnings enabled, run it on normal inputs, and then design adversarial boundary cases. For pointer-heavy code, write down the pointed-to object's owner and lifetime. For arithmetic, identify intermediate types and range limits before evaluation. For input processing, state whether a malformed record is rejected, partially accepted, or reported as incomplete.

Tests are evidence rather than proofs of universal correctness. A green hosted CI run is valuable, but cannot replace device-level validation of registers, DMA, interrupts, scheduling, and target libraries. Where a complete listing is not independently compile-tested, the manuscript should not claim otherwise.

Conventions

Code uses C23 spelling such as `bool`, `nullptr`, and empty initialization where appropriate. System-specific code is labeled as such. A pointer and length together describe a counted region only when the caller actually provides storage of that length; no signature can verify an arbitrary dangling pointer. An API may require a writable out-parameter and promise to leave it unchanged on failure. Such contracts are explained when introduced and reused consistently.

Array lengths and capacities are measured in elements unless explicitly described as bytes. Byte-oriented protocols specify octet widths when needed. Integer-overflow checks occur before potentially overflowing arithmetic. Sample code favors clarity and verifiable bounds over clever expression tricks.

An author's checklist for each chapter

A complete treatment should answer four questions. First, what problem does the feature solve? Second, what does the C language actually guarantee? Third, what assumptions does the example make about its caller or platform? Fourth, how would an engineer test the normal path and its boundaries? This revision was expanded with those questions in mind.

The manuscript remains a draft while technical review, compiler verification, layout checking, and indexing continue. The intended final length is roughly 200–230 typeset pages, but page count alone is not a substitute for coherent explanation.

Reading path

Chapters 1–8 establish the everyday language and program structure. Chapters 9–12 connect the modern standard to reliability and a larger example. Chapters 13–19 deepen testing, representation, semantics, concurrency, callbacks, text, and the preprocessor. The appendices are designed to be consulted while writing programs rather than read in one uninterrupted pass.

If a reader remembers one principle, it should be this: in C, **the contract is part of the program**. A short function is not complete until its types, bounds, ownership, and failure behavior are understood.

1. A First Look at C

A C program describes operations on values and storage. It can run with very little runtime support, but it gives the programmer correspondingly little protection from mistakes. We begin with programs small enough to understand without an elaborate vocabulary.

1.1 A complete program

```c
#include <stdio.h>

int main(void)
{
    puts("hello, world");
    return 0;
}
```

The `#include` directive makes declarations from a header available. `main` is the function from which a hosted program begins. The braces contain a block. `puts` writes a string followed by a newline. A return value of zero reports success to the execution environment.

Compile with `cc -std=c23 -Wall -Wextra -Wpedantic hello.c -o hello` and run `./hello`. The `cc` command may select a compiler too old for the C23 flag; a modern GCC or Clang may be named explicitly.

The program has a declaration, an expression statement, and a return statement. There is no implicit declaration of `puts`; its prototype comes from `stdio.h`. Unlike historical C, modern C requires function declarations that accurately describe the parameters.

1.2 Variables and arithmetic

```c
#include <stdio.h>

int main(void)
{
    for (int celsius = 0; celsius <= 100; celsius += 20) {
```

```
        double fahrenheit = 9.0 * celsius / 5.0 + 32.0;
        printf("%3d %6.1f\n", celsius, fahrenheit);
    }
}
```

The loop creates, tests, and increments `celsius`. In each iteration, `fahrenheit` exists within the loop body. A value like `9.0` is floating point; integer division such as `9 / 5` would discard the fractional part. The `printf` format is an agreement between the programmer and the arguments: `%d` expects an `int`; `%f` expects a floating-point argument (a `double` after promotion).

Types are not interchangeable merely because two values can both be printed as numbers. The compile-time type controls arithmetic, conversions, available operations, and storage requirements.

1.3 Characters and the input stream

```
#include <stdio.h>

int main(void)
{
    int ch;
    unsigned long lines = 0;

    while ((ch = getchar()) != EOF) {
        if (ch == '\n')
            ++lines;
    }
    printf("%lu\n", lines);
}
```

`getchar` returns an `int`, not a `char`: it must represent every `unsigned char` value *and* the negative sentinel `EOF`. The parentheses in the loop make the assignment happen before the inequality test. The input is read until end-of-file or an input error; a production program checks `ferror(stdin)` to distinguish the two.

1.4 An array and a simple histogram

Suppose a program counts decimal digits in its input:

```
#include <stdio.h>

int main(void)
{
    unsigned long count[10] = {0};
    int ch;
    while ((ch = getchar()) != EOF) {
        if (ch >= '0' && ch <= '9')
            ++count[ch - '0'];
    }
    for (int i = 0; i < 10; ++i)
        printf("%d: %lu\n", i, count[i]);
    return ferror(stdin) ? 1 : 0;
}
```

A C array is a contiguous sequence of elements. Indexes begin with zero. The array above has exactly ten positions; a value of ten is already out of bounds. Unlike many modern languages, C does not generally check array bounds at runtime.

1.5 Functions as named operations

```
static int maximum(int a, int b)
{
    return a > b ? a : b;
}
```

A function packages an operation behind a name and an interface. `static` at file scope gives this function internal linkage: other source files cannot refer to it by this name. The conditional operator selects one of two expressions.

Exercises. 1.1 Change the temperature program to print from 100 down to 0. 1.2 Count tabs and spaces separately. 1.3 Count lines longer than a fixed limit without storing whole lines. 1.4 Explain what happens if the histogram test accidentally admits the character `':'`.

1.6 Counting words: states rather than lines

Counting lines required remembering only a number. Counting words requires remembering whether the previous character belonged to a word. A *state* is just the minimum information needed to interpret the next input.

```
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    unsigned long words = 0;
    bool inside = false;
    int ch;

    while ((ch = getchar()) != EOF) {
        bool separator = ch == ' ' || ch == '\n' || ch == '\t';
        if (separator) {
            inside = false;
        } else if (!inside) {
            inside = true;
            ++words;
        }
    }
    if (ferror(stdin)) {
        fputs("input error\n", stderr);
        return 1;
    }
    printf("%lu\n", words);
}
```

The loop recognizes a transition from *outside* to *inside* a word. Notice that the definition of a word depends on the definition of separator. Here only three ASCII whitespace characters count. Changing to `isspace` requires `<ctype.h>` and care to pass only an `unsigned char` value or `EOF`.

A state-machine description is valuable beyond text processing. An embedded serial receiver similarly recognizes transitions among waiting, payload, and checksum states, and can process one byte at a time without allocating a complete input stream.

Trace a program

For input `"  one   two\n"`, write down `inside` and `words` after each character. Two adjacent separators must not count an empty word. End-of-file also must not count an extra word.

1.7 When the simplest program is not enough

The programs in this chapter fit in small files, but each has an implicit contract. A histogram counter might overflow on a sufficiently large input. An I/O operation may fail before reaching EOF. An unexpected input byte may not be meaningful under an intended encoding. A reliable C program deliberately records which of those conditions it will handle.

Three habits will recur throughout the book:

1. Carry buffer sizes explicitly.
2. Treat resource and I/O failures as ordinary control paths.
3. Use the type system and compiler diagnostics to expose assumptions rather than conceal them.

Further exercises

1.5 Modify the word counter to report separately how many words began with an ASCII digit. **1.6** Implement the line counter using a function `count_lines(FILE *stream)` that reports failure separately from a legitimate zero count. **1.7** Determine when `unsigned long` is wide enough for your intended input volume; document a policy for counter overflow. **1.8** Write a program that collapses runs of spaces to one space while preserving newlines.

1.8 Reading a program from the outside inward

A complete program has two interfaces. The first is its *source interface*: its declarations, definitions, and required headers. The second is its *execution interface*: how input arrives, what output means, and which exit status reports success. Keeping both visible makes the examples easier to reason about.

In a hosted C environment, execution starts at `main`. Either `int main(void)` or the conventional argument-bearing form `int main(int argc, char *argv[])` may be used. Parameters named `argc` and `argv` hold the command-line argument count and vector. The argument strings are accessible through pointers; they are not fixed-sized arrays embedded in the function. A freestanding embedded implementation may use a different entry point and need not provide all hosted I/O facilities.

An output statement has a return value for a reason. `puts` returns `EOF` on output error. In a quick hello-world demonstration we ignore that possibility; in a program that promises reliable output, we check it:

```
#include <stdio.h>

int main(void)
{
    if (puts("hello, world") == EOF)
        return 1;
    return fflush(stdout) == EOF ? 1 : 0;
}
```

This detects errors reported by these stream operations. It does not guarantee that bytes have reached nonvolatile storage.

1.9 Trace the loop, not the syntax alone

When studying an unfamiliar loop, create a three-column trace: the input just consumed, the current state, and the counters. For the word counter earlier in this chapter, consider the bytes `a`, space, space, `b`, newline. The transition on the first `a` increases the count; the two adjacent spaces do not; the `b` increases it once more. Writing this trace explains the algorithm better than memorizing the loop.

Now change the input to the empty stream. The loop body never executes, and the count remains zero. Change it to a stream containing only separators: the state never enters a word. Change it to a final word with no newline: the last word is still counted, because the increment happens at word entry rather than word exit. Such tests expose the program's true specification.

A byte read from `getchar` is always represented by an `int` return type. This permits every possible `unsigned char` value as well as `EOF`. If the value is assigned first to a plain `char`, some valid byte values may be confused with the sentinel on implementations with signed `char`.

1.10 Programs as small transformations

A useful first decomposition separates three questions: What is the input domain? What state must be retained? What result must be produced? The digit histogram's input domain is a byte stream; its state is ten counters; its output is a display of those counters. Its logic makes no assumptions about keeping the whole input in memory.

The same shape reappears in firmware: read a sample or message, update bounded state, and report or act on the result. The difference is not primarily a new language construct but the stronger need to bound time, memory, and error handling. Keeping simple algorithms streaming from the outset makes the later examples easier to adapt.

Further problems: Describe precisely what the word counter considers a separator. Add overflow checks to the histogram counters using `ULONG_MAX`. Change the line counter to distinguish a newline count from a count of human-readable lines that includes a final unterminated line.

1.11 The execution path from source file to program

The small examples hide several steps that are worth separating. A preprocessor handles directives such as `#include`; a compiler translates a translation unit into object code; a linker combines objects and libraries into an executable. On a hosted implementation, the runtime arranges for `main` to execute. None of those stages guarantees that an input file exists or that a terminal will accept output.

Consider a program split between `count.c` and `main.c`. The shared function declaration belongs in a header. If `main.c` is compiled without seeing the declaration, current C does not permit inventing an implicit function declaration. That old convenience was dangerous: the compiler could assume an incompatible calling convention or return type. A correct C program states its interfaces.

A useful first experiment is to compile with `-c` and inspect the resulting object files without linking them. Then link those objects into an executable. Deliberately omit the object containing one function definition: the unresolved external symbol illustrates the boundary between compilation and linkage.

1.12 A state machine for byte-oriented input

The line and word counters suggest a general pattern. Each input byte causes a state transition, and the program emits results based on the resulting state. The same pattern recognizes a serial protocol whose messages begin with a marker, contain a length, and end with a checksum.

For a toy ASCII protocol, suppose lines are terminated by `'\n'` and may contain at most 16 ordinary bytes. A parser could maintain a buffer, a length, and one status flag. Every ordinary byte is accepted only if capacity remains. A newline terminates a record, and an overlong line is rejected rather than truncated into something that looks valid.

The key engineering step is to write the invariants before the loop: `0 <= length <= capacity`; no bytes are written beyond `buffer[length-1]`; failure has a documented outcome; and after rejecting a long line the parser must either drain to the next newline or stop processing. That final choice is a *resynchronization policy*, not a syntax issue.

Why not read the entire input file first? Streaming input requires bounded storage regardless of total input size, and it can process an indefinitely long stream. Its tradeoff is that the algorithm must remember enough state to handle records split across reads.

1.13 Three kinds of correctness

A counting program can fail in at least three distinct ways.

First, the algorithm can be logically wrong: perhaps two consecutive spaces are counted as a word. Second, it can invoke an operation the language does not define, such as accessing an array at index ten when its valid indexes are zero through nine. Third, it can encounter an ordinary external error: the input stream may fail, or the output may not be writable.

A test for one class does not necessarily reveal the others. Trace the state machine to reason about the first. Make bounds explicit and use compiler diagnostics and sanitizers to help investigate the second. Check I/O results to address the third.

1.14 Choosing types from contracts

For `getchar`, `int` is the correct return type because the function must represent `EOF` separately from ordinary character values. For an array length, `size_t` is the natural type because the standard library uses it to represent object sizes. For a fixed-width protocol field, use a suitable type such as `uint16_t` if the target provides it.

The preferred type depends on meaning rather than an aesthetic preference for unsigned or wide integers. A counter of unbounded input events can still overflow; using `unsigned long` does not make the data unlimited. Choose a maximum supported volume and decide what to do if it is exceeded.

Guided exercises

1.9 Write a table of the input states for the word counter: separator, word byte, and EOF. For each, state which variables change. **1.10** Propose a record parser with a 16-byte buffer and show the sequence of state changes for a 17-byte record. **1.11** Modify the histogram to report an error before incrementing a counter already at `ULONG_MAX`. **1.12** Build a two-translation-unit program and explain exactly which step fails when a required object file is omitted.

2. Types, Expressions, and Control

The most important questions about an expression are not only what value it computes but which type it uses and whether every intermediate operation is defined.

2.1 Integer types and ranges

C provides `char`, `short`, `int`, `long`, and `long long`, each with signed and unsigned variants where applicable. The exact widths are implementation-dependent within standard constraints. Use `<stdint.h>` when a protocol requires an exact width such as `uint32_t`; these exact-width typedefs exist only when the implementation provides suitable types. `size_t` from `<stddef.h>` describes object sizes and array indexes.

```
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>
int main(void)
{
    uint32_t mask = UINT32_C(1) << 7;
```

```
    printf("%08" PRIx32 "\n", mask);
}
```

The formatting macros such as `PRIx32` are declared in `<inttypes.h>`, which is included above. Prefer unsigned integers for bit masks and modular arithmetic, not as a generic cure for negative values. Unsigned overflow wraps modulo a power of two; signed overflow has undefined behavior.

2.2 Conversion is an operation

```
int a = 7;
int b = 2;
double x = a / b;         /* 3.0, division occurred as int */
double y = (double)a / b; /* 3.5 */
```

Conversions occur at assignments, calls, returns, and within arithmetic. Integer promotions may change a narrow type to `int`. Mixed signed and unsigned arithmetic deserves particular suspicion: compare values only after choosing an appropriate common domain.

2.3 Conditions and loops

The forms `if`, `else`, `switch`, `while`, `do`, and `for` govern control flow. A controlling scalar expression is false if it compares equal to zero, true otherwise. Use braces even for short branches in code that will evolve.

```
unsigned gcd(unsigned a, unsigned b)
{
    while (b != 0) {
        unsigned remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

Each iteration reduces the second operand unless it is already zero. The termination argument is as useful as the program itself: a loop should have a quantity that progresses toward a bound.

2.4 Short circuit and sequencing

`&&` and `||` evaluate left to right and skip the right operand when its value cannot affect the result. Thus `p != nullptr && *p == 5` can test a pointer before dereferencing it, though `nullptr` requires C23 support. Avoid compact expressions that modify the same scalar multiple times without proper sequencing: `i = i++ + 1` is not a clever increment; it has undefined behavior.

2.5 Floating point

Floating-point values are approximations. A decimal fraction like 0.1 often cannot be represented exactly in binary. Equality comparisons can be appropriate for exact assignments and sentinels; they are often unsuitable as tolerance tests for numerical computations. Relative and absolute tolerances should be chosen from the problem's scale, not copied as universal constants.

2.6 C23 spelling and initialization

C23 recognizes `bool`, `true`, and `false` as keywords, and permits empty initialization:

```
struct point { int x, y; };
struct point origin = {};
bool ready = false;
```

An empty initializer gives the object an empty-initialization value (for this structure, both fields become zero). Check compiler support on older toolchains.

Exercises. 2.1 Implement absolute difference for unsigned integers without underflow. 2.2 Write `gcd` tests including zeros. 2.3 Explain why `int n = sizeof array;` can narrow an object size. 2.4 Determine which operations in a signed integer sum can overflow.

2.7 Integer conversion in practice

Consider `size_t`, an unsigned type used for object sizes. It is tempting to subtract two lengths and then test whether the result is negative:

```
size_t remaining = capacity - used;
```

If `used > capacity`, this subtraction does not produce a negative number; unsigned arithmetic wraps. Instead, establish the precondition before computing the difference:

```
if (used > capacity)
    return false;
size_t remaining = capacity - used;
```

A second trap is evaluating mixed integer types. A comparison involving an unsigned operand may convert a signed value to an unsigned type. It is better to check a negative value before a conversion than to hope the usual arithmetic conversions preserve its mathematical meaning.

2.8 Bitwise operations

Bit masks encode independent boolean properties in one integer. Shift counts must be valid for the promoted left operand's width. For portable protocol data, perform shifts in an explicitly selected unsigned type.

```
#include <stdint.h>

enum {
    FRAME_READY = 1u << 0,
    FRAME_ERROR = 1u << 1,
    FRAME_RETRY = 1u << 2
};

unsigned flags = FRAME_READY | FRAME_RETRY;
bool retry = (flags & FRAME_RETRY) != 0;
flags &= ~FRAME_RETRY;
```

The `|` operator combines selected bits, `&` tests or retains bits, and `~` complements all bits in the promoted type. Using a mask is not the same as testing a boolean expression with `||` or `&&`; those logical operators produce zero or one and may short-circuit.

An important embedded distinction: masking an integer held in RAM is straightforward; read-modify-write of a device register may trigger hardware-specific behavior. Consult the peripheral's reference manual before applying an apparently innocent `register |= bit`.

2.9 Evaluation order is independent of precedence

Multiplication binds more tightly than addition, but precedence tells the compiler how expressions are grouped, not the order of their operands' evaluation. For example, `f() + g()` does not impose an ordering between calls. The programmer can create a sequence explicitly:

```c
int left = f();
int right = g();
int result = left + right;
```

Now the calls happen in the order of the statements, though addition still requires a representable result. Avoid constructions that both modify and read a scalar without the necessary sequencing. Compact code is not an excuse for surprising evaluation.

2.10 A checked average

For integers in a constrained environment, accumulating into a wider type can help, but a wider type is not a proof of no overflow. Even a 64-bit sum can overflow if the input is long enough. A fully safe routine must either check each addition, constrain the input count and element ranges, or use a documented alternative algorithm.

A good interface for an average also distinguishes the empty-input case from a real average of zero. A return value alone does not always provide enough information.

Further exercises

2.5 Write a function that sets and clears one bit of a `uint32_t` given a bit position. Reject positions outside 0–31. **2.6** Show an example in which adding two valid `int` values overflows, and implement a guard that avoids evaluating the addition. **2.7** Explain the difference between `a & b == 0` and `(a & b) == 0`. **2.8** Design a loop over `unsigned` values that handles `UINT_MAX` without wrapping forever.

2.11 Representable values before casts

A cast requests a conversion; it does not assert that the mathematical value fits. If an external protocol provides a `uint32_t` length and a function expects an `int` length, test the range *before* conversion. A cast performed first may already have discarded information.

```c
#include <stdint.h>
#include <limits.h>
#include <stdbool.h>

bool length_to_int(uint32_t length, int *out)
{
    if (out == nullptr || (uintmax_t)length > (uintmax_t)INT_MAX)
        return false;
    *out = (int)length;
    return true;
}
```

Here `uintmax_t` gives a common unsigned domain capable of representing both operands; the positive limit `INT_MAX` is safe to convert. The routine assigns its output only on success. This pattern works for sizes, array indexes, and serialized integer fields.

C23 requires two's-complement representation for signed integers, but that does **not** make signed overflow defined. `INT_MAX + 1` is not a portable wraparound operation. The representation model and the rules for evaluating expressions are separate questions.

2.12 Division, remainder, and a dangerous minimum

For integer division, the fractional part is discarded toward zero. The remainder satisfies `(a / b) * b + a % b == a` when the quotient is representable and `b` is nonzero. The expression `INT_MIN / -1` can overflow the signed result on a typical two's-complement `int` and has undefined behavior. Division by zero is undefined for integer arithmetic.

An absolute-value function with signed output cannot represent the absolute value of `INT_MIN` on the usual two's-complement implementation. Instead of writing `abs(INT_MIN)` and assuming success, choose a wider type where available or provide a checked interface.

2.13 Floating-point accuracy is a contract

Floating-point arithmetic has finite precision and a finite exponent range. A result can round differently from the exact real-number result; adding a very small number to a very large one may change nothing at all. Reassociating sums can alter their results:

```
double a = 1.0e20;
double b = -1.0e20;
double c = 3.0;
/* (a+b)+c and a+(b+c) need not be equal. */
```

A numerical algorithm should state its permitted error, expected input scale, handling of infinities and NaNs, and acceptable loss of significance. The rules can also depend on whether the target implements IEC 60559 floating-point facilities. In embedded computation, fixed-point is sometimes chosen not because floating point is categorically wrong, but because reproducibility, speed, or resource bounds favor an integer representation with an explicit scale.

2.14 A precise conversion exercise

Write a checked function accepting a signed `long` and returning a `uint16_t` if and only if the input is between zero and `UINT16_MAX`. The correct order is: reject negative input; compare the nonnegative input in an appropriate common type; then perform the cast. Test `-1`, zero, one, the maximum, and one beyond the maximum wherever that value is representable.

Be careful with `sizeof`: its result is `size_t`, not `int`. Printing it with `%d` is an argument-type mismatch. Use `%zu` for `size_t`. Comparing object sizes and subtracting indexes also requires careful thought: an unsigned subtraction cannot express a negative distance unless the design explicitly uses a signed difference type such as `ptrdiff_t` and checks its range.

Further problems: Derive a safe check for `a * b` with two positive `size_t` values. Show the difference between `(unsigned)-1` and a signed negative one. Explain why shifting a negative signed value is a poor choice for portable bit-manipulation code.

2.15 Promotions are part of the computation

Small integer types are often promoted before arithmetic. Suppose `uint8_t a = 200, b = 100;`. On a common target where `int` represents every value of `uint8_t`, the expression `a + b` is evaluated in `int`, producing 300. Assigning that result back to `uint8_t` then performs a conversion that produces a value modulo 256. These are two different stages.

Do not infer the expression's type solely from the declared type of its operands. When shifts or bitwise complement are involved, promotions can be particularly surprising. A `~` applied to a narrow unsigned value can first promote it to `int`, yielding a value with more significant bits than the programmer expected.

If a protocol needs an eight-bit mask, perform the operation in a known unsigned domain and intentionally narrow the result only after masking:

```
#include <stdint.h>

uint8_t low_byte(uint32_t value)
{
    return (uint8_t)(value & UINT32_C(0xff));
}
```

The mask states exactly which bits belong in the result.

2.16 Comparisons between signed and unsigned values

Consider `int delta = -1;` and `size_t count = 4;`. The expression `delta < count` can convert the negative `int` to an unsigned type, depending on the usual arithmetic conversions. On a typical 64-bit platform, it evaluates in a way a casual mathematical reading does not predict.

A reliable design checks negativity first:

```
bool valid_index(int index, size_t count)
{
    return index >= 0 && (size_t)index < count;
}
```

The && operation evaluates the cast and comparison only after establishing that the signed input is nonnegative. The cast is then value-preserving whenever `size_t` has sufficient range for the nonnegative `int`; implementations with unusual widths deserve a more general conversion policy.

Prefer to keep array positions as `size_t` from the beginning when negative indexes are not meaningful. But do not convert a potentially negative external value to `size_t` before validating it.

2.17 Division edge cases

Integer division by zero is undefined. The signed quotient may also be unrepresentable: on ordinary two's-complement systems, `INT_MIN / -1` is the familiar example. A library routine parsing a user-supplied divisor should test both conditions before evaluating the operation.

```
#include <stdbool.h>
#include <limits.h>

bool divide_int(int a, int b, int *out)
{
    if (out == nullptr || b == 0 ||
        (a == INT_MIN && b == -1))
        return false;
    *out = a / b;
    return true;
}
```

The output is unchanged on failure. If the function returns a quotient and remainder, compute them only after the same validation. C integer division truncates toward zero; the remainder has the sign rules implied by that division, not the always-nonnegative modulo convention used in some mathematical texts.

2.18 Floating-point comparisons require a scale

A universal test such as `fabs(a - b) < 1e-6` is usually not a universal numeric contract. It may reject two large quantities whose relative difference is tiny, and it may accept two tiny values whose relative difference is enormous.

A numerical application should choose absolute tolerance, relative tolerance, or a combination according to the units and problem:

```
#include <math.h>
#include <stdbool.h>

bool approximately_equal(double a, double b,
                         double absolute, double relative)
```

```
{
    if (!isfinite(a) || !isfinite(b) ||
        absolute < 0.0 || relative < 0.0)
        return false;
    double difference = fabs(a - b);
    double scale = fmax(fabs(a), fabs(b));
    return difference <= absolute || difference <= relative * scale;
}
```

This is a teaching fragment with an explicit finite-input policy; even here, intermediate overflow can make `a - b` or `relative * scale` nonfinite for extreme values. That may be acceptable for the application, or it may require a more careful algorithm. A numerical routine is correct relative to a chosen error model, not simply because it uses `double`.

2.19 Derive overflow checks rather than guessing

For two nonnegative `size_t` operands, `a * b` fits when either operand is zero or `a <= SIZE_MAX / b`. Notice why the division is performed only when `b != 0`. The test avoids executing multiplication until representability has been established.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

bool multiply_size(size_t a, size_t b, size_t *out)
{
    if (out == nullptr || (b != 0 && a > SIZE_MAX / b))
        return false;
    *out = a * b;
    return true;
}
```

When zero-sized allocations are undesirable, a caller can separately reject zero operands. Arithmetic validity and resource-policy validity are different matters.

Worked questions

What is the type of `sizeof buffer`? Why can a byte count greater than `INT_MAX` not safely be passed as an `int` length without a range check? Why can `INT_MIN` not always be negated? What does `a & b == 0` mean without additional parentheses? For each, answer by applying the language rules rather than by running one compiler and memorizing its output.

3. Functions and Program Structure

Good C programs put complexity into small, explicit interfaces. Functions make this possible, but they do not hide ownership, sizes, or error conditions unless the programmer makes those matters part of the contract.

3.1 Definitions, declarations, and headers

```
/* statistics.h */
#ifndef STATISTICS_H
#define STATISTICS_H
#include <stddef.h>
double average(const double values[static 1], size_t count);
#endif
```

The `static 1` in an array parameter is a promise that an argument refers to at least one element. It is not a runtime bounds check. For an interface that accepts zero elements, prefer `const double *values` and state whether a null pointer is acceptable when `count == 0`.

```
#include "statistics.h"

double average(const double *values, size_t count)
{
    if (count == 0) return 0.0; /* choose and document policy */
    double total = 0.0;
    for (size_t i = 0; i < count; ++i) total += values[i];
    return total / (double)count;
}
```

Zero is not always a good error signal. Later we will return status codes and write results through out-parameters.

3.2 Scope and lifetime

A variable defined in a block has block scope. An ordinary local variable generally has automatic storage duration: its lifetime ends when the block invocation ends. A file-scope object has static storage duration. `static` within a function preserves an object's value across calls, but introduces shared state that complicates testing and concurrent execution.

3.3 Recursion

```
unsigned long factorial(unsigned n)
{
    return n <= 1 ? 1UL : (unsigned long)n * factorial(n - 1);
}
```

This mathematical definition is compact, but unsigned overflow makes its result unsuitable for large inputs, and deep recursion consumes stack. A useful recursive function requires both a terminating case and a bounded resource argument. For most production factorial calculations, an iterative implementation with an overflow check is easier to defend.

3.4 Design by contract

A function that converts a decimal string should say whether leading whitespace is accepted, which characters terminate conversion, how overflow is reported, and whether partial parses are accepted. A good interface often has the form

```
bool parse_unsigned(const char *text, unsigned *result);
```

A successful call writes the result. A failed call leaves the output unchanged by convention. We can implement this without guessing whether an error sentinel might also be a valid result.

Exercises. 3.1 Implement `parse_unsigned` with overflow detection using `UINT_MAX`. 3.2 Replace a recursive factorial with an iterative, checked version. 3.3 Create a header/source pair and link it with a separate test file.

3.5 Out-parameters and explicit failure

A function that returns an integer can use every integer value as an ordinary result. That leaves no spare sentinel for failure. An interface can instead return success and write the result through a pointer:

```c
#include <stdbool.h>
#include <stddef.h>
#include <limits.h>

bool sum_nonnegative(const unsigned *items, size_t n,
                     unsigned *result)
{
    if (result == nullptr || (n != 0 && items == nullptr))
        return false;

    unsigned total = 0;
    for (size_t i = 0; i < n; ++i) {
        if (items[i] > UINT_MAX - total)
            return false;
        total += items[i];
    }
    *result = total;
    return true;
}
```

This function makes two promises: failure leaves `*result` unchanged, and success provides the exact sum. The first promise is maintained by assigning to the output only after all checks succeed. Its caller still has an obligation: the pointer must refer to a writable `unsigned` object. A null check cannot detect a dangling pointer.

The result type `bool` expresses a simple yes-or-no outcome. More elaborate interfaces can return an enumeration of errors: invalid argument, overflow, and unsupported operation are often more helpful than one generic false value.

3.6 Header contracts and linkage

Suppose a program has three translation units, `main.c`, `statistics.c`, and `statistics.h`. The header should state the function's public declaration and any types required to call it. The source file defining the function should include the same header; this makes disagreements visible to the compiler.

An identifier with external linkage may be referred to across translation units. A `static` function at file scope instead has internal linkage. Prefer internal linkage for helpers that are not part of the module's interface. It keeps the global symbol namespace small.

Do not put ordinary external object definitions in headers. If each including source file defines the same external object, linking can fail with multiple definitions. Place a declaration such as `extern unsigned counter;` in the header and exactly one definition `unsigned counter;` in a source file.

3.7 Recursion versus iteration

The mathematical definition of a recursive function does not establish that the implementation fits within the machine's resources. Each recursive invocation may need stack space. A function that explores a linked data structure may have recursion depth proportional to its length.

Use recursion when it reflects a bounded structure clearly. Prefer iteration when stack bounds are critical or when the recursive call can be replaced with an ordinary loop. For embedded programs, provide a maximum-depth argument before treating recursive code as suitable for production.

3.8 Exercises

3.4 Design a `parse_temperature` interface distinguishing invalid text from out-of-range input. **3.5** Implement a checked sum for signed `int` elements. **3.6** Split `sum_nonnegative` into a public header,

implementation, and executable tests. **3.7** Show how to preserve the output argument's value after each error. **3.8** Explain the difference between linkage and storage duration.

3.9 Choose the shape of an interface

Consider three interfaces for parsing a decimal value:

```
unsigned parse(const char *text);
bool parse(const char *text, unsigned *out);
enum parse_status parse(const char *text, unsigned *out);
```

The first has no unambiguous error channel if every `unsigned` value is valid. The second distinguishes success from failure, and the third can explain why parsing failed. Their usefulness depends on the caller's needs. A library for configuration files might distinguish empty input, invalid character, and range error; a tiny internal helper might only require a boolean result.

An out-parameter contract should explicitly say what happens on failure. A practical convention is *strong failure safety*: validate into local state and assign to `*out` only after success. That avoids forcing callers to guess whether they can reuse a previous result following an error. It does not protect against invalid pointers supplied by callers.

3.10 Function declarations are contracts across files

Imagine `parse.c` defines a function but `main.c` has copied its declaration incorrectly. Separate compilation may not catch a mismatch without a shared header. Include the public header in the implementation file as well as at call sites.

```
/* parse.h */
#ifndef PARSE_H
#define PARSE_H
#include <stdbool.h>
bool parse_unsigned(const char *text, unsigned *out);
#endif
```

The header describes the interface; it does not allocate storage or call the function. A linker can find the definition later when both translation units are linked. Be cautious about external variables in headers: `extern int errors;` is a declaration; `int errors;` can be a tentative definition and should not be duplicated indiscriminately across source files.

`static` at file scope gives helper functions internal linkage. Its purpose is not to make a function execute faster. `inline` also does not compel a compiler to substitute the body at call sites; it is part of the language's definition and linkage rules, not a measured performance guarantee.

3.11 A fully checked factorial

The earlier recursive factorial demonstrates recurrence, not a production overflow policy. A robust iterative version has to guard the multiplication and consider the loop-counter's boundary:

```
#include <limits.h>
#include <stdbool.h>

bool factorial_checked(unsigned n, unsigned long *out)
{
    if (out == nullptr)
        return false;
    unsigned long value = 1;
    for (unsigned i = 2; i <= n; ) {
        if (value > ULONG_MAX / i)
            return false;
```

```
        value *= i;
        if (i == n)
            break;
        ++i;
    }
    *out = value;
    return true;
}
```

The explicit final-iteration break prevents an unsigned loop counter from wrapping when `n == UINT_MAX`. In practice the multiplication check will reject very large values long before that, but the loop's correctness should not depend on an unrelated range fact when a simple structure makes termination obvious.

3.12 Functions and test seams

A routine that mixes input reading, parsing, allocation, and printing is harder to test than four smaller operations with explicit contracts. The goal is not an arbitrary number of functions. It is to isolate decisions: one function transforms input bytes; one validates values; one stores them; a boundary function handles the operating environment.

For host-based firmware tests, a function that accepts a byte buffer and a length can be tested without a UART peripheral. A thin UART adapter may read bytes on the device and forward them to the parser. This separation is useful because transport errors and parsing errors have different causes and remedies.

Further problems: Write a header and implementation for a checked decimal parser. Define a status enumeration for invalid syntax versus overflow. Test that failure preserves the output. Explain why a static local variable may complicate recursion and concurrent calls.

3.13 Separate syntax checking from semantic validation

A parser does two jobs: it recognizes a sequence of characters and it interprets the recognized sequence as a value. Combining both stages into an unchecked conversion makes it easy to accidentally accept trailing garbage or overflow.

Consider a decimal parser with this contract: the input must be a nonempty sequence of ASCII digits, its value must fit in `unsigned`, and on failure the output must remain unchanged. A clear algorithm is:

1. Reject a null input or null output pointer.
2. Reject empty text.
3. Start an accumulator at zero.
4. For each byte, reject anything outside `'0'` through `'9'`.
5. Before multiplying by ten, check that the new value is representable.
6. Assign the final accumulator to the output once, after the whole string succeeds.

`examples/parse_unsigned.c` implements this design. It deliberately differs from `strtoul` by rejecting signs and spaces. Neither behavior is inherently better; the application's grammar decides.

For a parser that receives a counted span, reject both a null pointer with nonzero length and any claimed length that is not actually backed by valid storage. The latter is a caller precondition C cannot test generically.

3.14 Return values and out-parameters

Why return a status and write a value through an out-parameter rather than return a structure containing both? Either design can work. In C, a small result structure is often natural:

```c
struct parse_result {
    bool ok;
    unsigned value;
};
```

With this interface, the caller does not manage a separate output pointer. The function can return an all-zero result on failure, then fill both members on success. It may be clearer for small values; an out-parameter may be preferable when writing a large object or retaining a long-established ABI.

Whichever design you choose, state the failure contract. A boolean-returning function that partly overwrites the output before reporting failure is much harder for callers to use correctly than one that commits a completed result once.

3.15 File scope, linkage, and modules

C has no built-in class module system. The conventional unit of modularity is a header plus one or more source files. The header exposes declarations; the source file defines those operations and keeps helpers private with `static`.

A source file should include its own public header before defining the corresponding functions. That forces the compiler to compare the declared and defined types. If callers include the same header, interfaces cannot silently diverge without diagnostics.

A declaration is not always a definition. For a shared file-scope object, the header can use `extern unsigned event_count;`, while precisely one source file supplies `unsigned event_count = 0;`. Repeating a non-`extern` external object definition across headers or translation units is not a sound substitute for a module.

3.16 Passing arrays does not pass their sizes

Suppose a function accepts `const int values[10]`. That parameter declaration is adjusted to a pointer type; the compiler does not automatically supply a count of ten and ordinarily cannot check what the caller passed. A more honest general interface accepts `const int *values, size_t count`.

The type `const int *` says the function may not modify the elements *through that pointer*. It does not guarantee ownership, lifetime, or thread safety, and it does not make the underlying object immutable if other permitted aliases exist.

To express a fixed minimum bound, C permits array parameter syntax with `static` in the brackets. Such a declaration is a caller contract, not dynamic bounds enforcement. It should be used only when every valid call really supplies enough elements.

3.17 Recursive functions need a resource argument

Recursion can closely express a tree traversal or mathematical recurrence, but every call consumes some execution resources. A desktop process may have ample stack; a microcontroller task may have only a small fixed stack.

When evaluating recursive code, ask whether the maximum depth is bounded by the input contract. Can a malicious or malformed input create arbitrarily deep nesting? What happens when the recursion reaches its base case? A recursive algorithm with logarithmic depth may be reasonable; a recursive traversal of an unconstrained linked list often deserves an iterative form or a documented maximum depth.

Guided exercises

3.9 Implement the structure-returning parser result and compare its call sites with the out-parameter version. **3.10** Write a self-contained public header and compile it as the first include in a separate test file. **3.11** Replace a recursively implemented byte count with a loop and identify the changed resource bounds. **3.12** Design an API to read a sensor value that can report 'not ready', 'hardware error', and 'valid sample' distinctly.

4. Pointers, Arrays, and Strings

Arrays own contiguous elements; pointers refer to objects or functions. The relationship is close, but the two are not the same thing.

4.1 Address and indirection

```
int value = 12;
int *p = &value;
*p = 13;          /* modifies value */
```

A pointer's validity depends on the referenced object's lifetime, alignment, and access permissions. Dereferencing a null, dangling, or otherwise invalid pointer has undefined behavior. C23 adds the keyword `nullptr` for a null pointer constant; legacy code commonly uses `NULL`.

4.2 Array arguments carry no length

```
#include <stddef.h>

int sum(const int *data, size_t count, long long *out)
{
    if (out == nullptr || (count != 0 && data == nullptr))
        return -1;
    long long total = 0;
    for (size_t i = 0; i < count; ++i)
        total += data[i]; /* caller must ensure representable result */
    *out = total;
    return 0;
}
```

The signature carries a count because a pointer does not record the bounds of the original array. This implementation still assumes the sum fits in `long long`; a fully checked version needs an overflow test before each addition.

4.3 Strings are terminated byte sequences

A C string is an array of `char` with a terminating zero byte. `strlen` measures the number of bytes before that terminator, not necessarily the number of Unicode characters. `sizeof` measures a type or an object, not the current length of a string.

```
char name[] = "C23";
size_t capacity = sizeof name;  /* 4 including '\0' */
```

UTF-8 represents some characters with multiple bytes. A procedure that uppercases arbitrary UTF-8 by manipulating each `char` is not a correct Unicode case converter.

4.4 Copying bounded text

```c
#include <stddef.h>

bool copy_text(char *dst, size_t capacity, const char *src)
{
    if (dst == nullptr || src == nullptr || capacity == 0)
        return false;
    size_t i = 0;
    while (i + 1 < capacity && src[i] != '\0') {
        dst[i] = src[i];
        ++i;
    }
    dst[i] = '\0';
    return src[i] == '\0';
}
```

This function assumes valid source storage and that source and destination do not overlap; it signals truncation rather than silently claiming success. It is not a replacement for a general `memmove`.

4.5 Constness and pointer arithmetic

`const int *p` prohibits modifying an `int` through `p`. `int *const p` prohibits assigning another address to `p`. Pointer arithmetic is defined only within an array object (including the permitted one-past position), subject to the language's constraints. A one-past pointer may be compared or subtracted appropriately but not dereferenced.

Exercises. 4.1 Implement `strlen` using a pointer and compare with an index version. 4.2 Explain the difference between `char *p = "abc";` and `char p[] = "abc";`. 4.3 Implement `reverse` for a mutable byte array of known length. 4.4 Add overlap detection or explicitly reject overlap in a copy API.

4.6 A pointer is not an array descriptor

The expression `sizeof array` is useful inside the scope where `array` is an actual array object. Once an array expression is converted to a pointer for passing to a function, its bound is ordinarily lost.

```c
#include <stddef.h>

static size_t count_values(const int *values, size_t count)
{
    size_t found = 0;
    for (size_t i = 0; i < count; ++i)
        if (values[i] != 0) ++found;
    return found;
}

int main(void)
{
    int values[] = {3, 0, 8, 0, 1};
    size_t count = sizeof values / sizeof values[0];
    return count_values(values, count) == 3 ? 0 : 1;
}
```

The caller supplies the element count. The callee accepts that count as a contract; C cannot generally establish that a non-null pointer references as many elements as claimed.

4.7 Two-dimensional arrays

A real two-dimensional array occupies contiguous storage in row-major order:

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

The expression `matrix` in most contexts converts to a pointer to an array of three `int`, not an `int **`. This distinction is essential: a pointer-to-pointer usually points to another pointer object, whereas a 2-D array's rows are laid out directly next to each other.

A function that requires a known column count can state it:

```
int matrix_total(size_t rows, const int values[rows][3])
{
    int result = 0; /* contract: sum must fit in int */
    for (size_t r = 0; r < rows; ++r)
        for (size_t c = 0; c < 3; ++c)
            result += values[r][c];
    return result;
}
```

C23 retains variable length array support as specified by the standard, with implementation differences relevant to storage-duration choices. Where fixed dimensions suffice, fixed arrays make capacity easier to reason about.

4.8 Lifetime is as important as address

Never return the address of an automatic local object:

```
int *bad_pointer(void)
{
    int local = 7;
    return &local; /* invalid after this invocation ends */
}
```

The problem is not that the stack address must immediately change; it is that the object's lifetime has ended. The behavior of later dereferencing that pointer is not defined. Returning a value, writing through a caller-supplied pointer, or returning an allocated object with a documented owner are legitimate alternatives.

4.9 Restrict and aliasing

The qualifier `restrict` describes an access pattern that permits optimizations when the programmer satisfies its association requirements. It is a promise, not a runtime test. Do not add it to a public API merely because it sounds faster. If two pointer arguments can legally refer to overlapping storage, the interface must not assert a restriction that the caller violates.

4.10 Exercises

4.5 Implement an in-place reversal of a `char` array given its exact length. **4.6** Explain why `int **p = matrix;` is incompatible with a two-dimensional `int matrix[2][3]`. **4.7** Write a function to find a value in a slice represented by pointer and length. **4.8** Return both minimum and maximum through checked out-parameters. **4.9** Explain what happens to pointers into a block after successful `realloc`.

4.11 Three meanings of an array name

In many expressions an array expression converts to a pointer to its first element. Important exceptions include operands of `sizeof`, unary `&`, and other contexts with specified non-decay behavior. For a declared array:

```
int items[5] = {};
size_t bytes = sizeof items;
int *element = items;
int (*whole)[5] = &items;
```

`element` points to an `int`; `whole` points to the entire five-element array. Incrementing the former advances by one `int`; incrementing the latter would advance by one whole array. The types convey this difference and permit the compiler to diagnose incompatible pointer uses.

When an array is passed as a function argument, a declaration such as `void process(int values[10]);` still adjusts to a pointer parameter. The `10` in that form does not create a checked ten-element slice. Carry a count unless the function's contract fixes the dimension and callers are required to honor it.

4.12 One past the end is not an element

Pointer arithmetic can produce the one-past position of an array. It is useful as an end marker, but it cannot be dereferenced. For a nonempty array, `begin = &items[0]` and `end = begin + count` can represent a half-open range `[begin, end)`.

Comparing or subtracting arbitrary unrelated object pointers is not the same as comparing array positions. Pointer subtraction is defined for pointers into the same array (including one past), subject to representability in `ptrdiff_t`. When `count` may be enormous, indexes of type `size_t` can be more straightforward than pointer differences.

A null pointer is not an empty array that can be incremented freely. A zero-length buffer interface should explicitly describe whether it accepts `nullptr`, and code must avoid forming `nullptr + 0` merely to make an end pointer.

4.13 Copy, move, and overlap

`memcpy` copies object-representation bytes from one valid region to another but requires nonoverlapping regions. `memmove` supports overlap. Both require sufficiently large regions. Neither can prove that a claimed size is correct.

A safe shift within an array might use:

```
#include <string.h>
/* Preconditions: capacity >= count + 1 and position <= count. */
memmove(&items[position + 1], &items[position],
        (count - position) * sizeof items[0]);
```

Those preconditions matter. Check capacity and `position` before computing an address or multiplication. Also ensure the item type permits moving its object representation in the intended way; plain C value records generally do, whereas pointers inside records may carry ownership semantics not preserved by shallow copying.

4.14 String ownership and views

A string literal has static storage duration; modifying it through a pointer produces undefined behavior. By contrast, `char text[] = "abc";` defines a mutable array whose elements can be changed. The type `const char *` commonly signals that a function borrows text without modifying it, but `const` alone does not guarantee the memory remains alive.

A counted string view, `struct text_view { const char *data; size_t size; };`, can represent slices that are not null-terminated. Such views are convenient for parsers, but passing their pointers to `strlen` or `printf("%s")` is unsafe unless termination is separately guaranteed. A view usually borrows storage and should not outlive its owner.

Further problems: Implement a bounded substring search that never relies on a terminator. Explain the difference between `int (*)[4]` and `int **`. Write tests for a zero-length slice and for the final legal element index.

4.15 Draw the objects before drawing pointers

For a declaration such as

```
int values[4] = {10, 20, 30, 40};
int *p = &values[1];
```

draw four adjacent `int` objects and one separate pointer object. The pointer is not a second element of the array; it stores a value designating the second element. `p + 1` points at the third element. `*(p + 1)` reads that element. `p[1]` is defined in terms of the same indexing relationship.

Now draw `&values`. Its type is pointer to an array of four `int`, whereas `values` ordinarily converts to pointer to its first `int`. These pointers may designate the same starting address in memory, yet their types control pointer arithmetic: incrementing a pointer-to-array advances by the whole array size.

This is why declarations matter even when two pointer values print alike.

4.16 A counted slice

A plain pointer to the first element does not contain a length. A small value type can carry the two pieces together:

```
#include <stddef.h>

struct int_slice {
    const int *data;
    size_t length;
};
```

The slice usually *borrows* storage. It does not allocate anything, and copying the structure copies the pointer and length rather than copying the elements. The owner must keep the array alive for the duration of every use of the slice.

A safe access function may return a boolean result and use an output pointer:

```
#include <stdbool.h>

bool slice_get(struct int_slice s, size_t index, int *out)
{
    if (out == nullptr || index >= s.length ||
        s.data == nullptr)
        return false;
    *out = s.data[index];
    return true;
}
```

This checks the index against the *declared* slice length. Its precondition still requires `s.data` to point to at least `s.length` readable integers when `s.length` is nonzero. A dangling pointer or dishonest length cannot be repaired by the index check.

The standalone example `examples/slices.c` demonstrates both direct access and the creation of a bounded subslice.

4.17 Derive a subslice without overflow

A subslice starts at `offset` and contains `count` elements. It is tempting to check `offset + count <= length`, but unsigned addition may overflow. Instead, require `offset <= length` and `count <= length - offset`. That subtraction is evaluated only after the first condition holds.

When `count == 0`, you may represent the new slice using a pointer to an existing valid position, including the one-past position of a nonempty array. If the original slice is empty and its data pointer is null, do not form `data + 0`: pointer arithmetic on a null pointer is not a portable way to construct a valid empty slice. An implementation can return a null data pointer for a zero-length result.

4.18 Const qualifies a path of access

Compare these declarations:

```
const int *read_only_elements;  /* elements not modifiable through pointer */
int value = 0;
int *const fixed_address = &value;
```

The first pointer variable may be reassigned, but it cannot be used to assign to the designated `int`. The second pointer variable itself cannot be reassigned, but it can modify the designated `int` if that object is modifiable and the pointer is valid.

A `const`-qualified pointer is not an ownership system. A borrowed `const char *` can become dangling when another part of the program releases its storage. Conversely, a constant object with static storage duration may remain alive for the whole program. Lifetime and permission to modify are separate properties.

4.19 Overlap and copying

Suppose `char letters[] = "abcdef";`. Moving the region `letters[0..3]` one position to the right creates overlapping source and destination regions. `memcpy` is not the right operation; `memmove` is specified to handle such overlap.

Before calling either library function, establish that both source and destination contain the requested number of valid bytes. The function names do not infer buffer capacity. For a type with pointer members, byte-wise copying the enclosing structure still performs a shallow copy; it does not duplicate heap storage referenced by members.

4.20 Pointer lifetime can end without the pointer value changing

A local variable's lifetime ends when the relevant block invocation ends. A heap allocation's lifetime ends when it is freed. Successful `realloc` ends the old allocation's lifetime and may return a different address. In each case an old pointer can retain an address-like bit pattern while becoming unusable for dereferencing the former object.

A good habit is to avoid returning borrowed pointers to transient storage. Instead return a value, let the caller supply output storage, or explicitly transfer ownership of allocated storage under an API contract.

Guided exercises

4.10 Draw a four-element array, a pointer to element two, and a pointer to the whole array; determine the element reached by each valid pointer increment. **4.11** Implement `slice_find`

returning an index through an output pointer, distinguishing 'not found' from an invalid input. **4.12** Implement a reverse operation on a mutable slice and test empty and single-element slices. **4.13** Explain why copying a slice structure does not copy its elements. **4.14** Write a subslice function whose arithmetic remains defined at `SIZE_MAX`.

5. Structures and Data Representation

Structures associate fields under one type. Enumerations name integral alternatives. Unions allow members to share storage, but they require disciplined tracking of which interpretation is meaningful.

5.1 A value type

```
struct sample {
    unsigned channel;
    double measurement;
    bool valid;
};

struct sample reading = {
    .channel = 2,
    .measurement = 3.75,
    .valid = true
};
```

Designated initializers name members instead of depending on their order. Structure assignment copies member values. Padding bytes may exist between fields; they are not stable serialized data. Do not write a structure directly to a wire protocol or disk format and assume portability across targets.

5.2 Tagged unions

```
enum value_kind { VALUE_INTEGER, VALUE_REAL };

struct value {
    enum value_kind kind;
    union {
        long integer;
        double real;
    } data;
};
```

The tag establishes which member the surrounding program treats as active. An API should expose constructors and accessors rather than relying on callers to maintain an undocumented convention.

5.3 Linked records

```
struct node {
    int key;
    struct node *next;
};
```

A linked list illustrates the difference between a logical collection and its physical representation: nodes may be discontiguous, each allocation may fail, and traversal follows explicit links. In

embedded systems, a bounded array can often be simpler and more predictable than a heap-backed list.

5.4 Bit fields and explicit masks

Bit fields can make local state concise, but details of layout and allocation are implementation-defined. For device registers and portable serialized formats, fixed-width unsigned integers with shifts and masks often make data representation clearer. Hardware access adds concerns about `volatile`, synchronization, barriers, and the device's register semantics; `volatile` alone does not make an operation atomic.

Exercises. 5.1 Represent a timestamp with a structure and validate its fields. 5.2 Implement a fixed-capacity pool of list nodes without `malloc`. 5.3 Encode two unsigned 4-bit fields into a byte and decode them.

5.5 Field order, padding, and ABI

A structure establishes a logical association among members, but not a portable wire representation. A compiler may insert padding to meet alignment requirements. The order of named non-bit-field members is preserved in address order, yet `sizeof(struct record)` may exceed the sum of member sizes.

```
struct record {
    unsigned char kind;
    unsigned count;
};
```

There may be padding between `kind` and `count`, or after the final member. Different target architectures or compiler options may create different representations. Avoid `fwrite(&record, sizeof record, 1, stream)` as a long-term file format unless the representation is intentionally fixed for that exact environment.

To serialize an unsigned multi-byte value, select an exact format and explicitly encode bytes. That code is easier to test and more portable than depending on host endianness or struct layout.

5.6 A bounded ring buffer

A ring buffer holds a fixed number of elements and wraps its index back to the beginning. We use a separate count to distinguish an empty buffer from a full one.

```
#include <stdbool.h>
#include <stddef.h>

enum { QUEUE_CAPACITY = 8 };

struct queue {
    unsigned data[QUEUE_CAPACITY];
    size_t head;
    size_t count;
};

bool queue_push(struct queue *q, unsigned value)
{
    if (q == nullptr || q->count == QUEUE_CAPACITY)
        return false;
    size_t tail = (q->head + q->count) % QUEUE_CAPACITY;
    q->data[tail] = value;
    ++q->count;
```

```
    return true;
}

bool queue_pop(struct queue *q, unsigned *out)
{
    if (q == nullptr || out == nullptr || q->count == 0)
        return false;
    *out = q->data[q->head];
    q->head = (q->head + 1) % QUEUE_CAPACITY;
    --q->count;
    return true;
}
```

Initialize the entire queue with `struct queue q = {};`. This implementation is appropriate for sequential calls; it is **not** a concurrent lock-free queue. Sharing it between an interrupt and a thread introduces separate ordering, atomicity, and scheduling requirements.

One invariant states that `count` stays between zero and capacity. Another states that `head` is a valid index. Both remain true after successful and failed operations, assuming a properly initialized queue.

5.7 Opaque structures as a design choice

Sometimes callers need a type's behavior, not its layout. A header may declare `struct handle;` without defining its members. A source file privately defines the structure and exports functions taking pointers to it. This allows implementations to change without recompiling all callers, though allocation and ownership must be carefully arranged.

Opaque types are especially useful for devices, parsers, and protocol sessions. A public, by-value structure is often preferable for tiny value types that need no hidden invariants.

5.8 Exercises

5.4 Trace the queue above across eight pushes, one failed push, three pops, and three more pushes. **5.5** Add a `queue_peek` operation that does not remove an item. **5.6** Test that a failed pop leaves the output unchanged. **5.7** Encode `kind` and `count` in a documented byte order without copying the structure representation. **5.8** Compare the memory requirements of this queue and a linked queue for the same maximum capacity.

5.9 Structure layout as a local implementation detail

A structure value is a collection of logically related members, not a universally portable byte record. A compiler may insert padding before, between, and after members so that each can be accessed at an appropriate alignment. The expression `sizeof(struct sample)` therefore answers a question about this implementation's object layout; it does not define a network packet size.

Use `offsetof` from `<stddef.h>` when you genuinely need a member's offset within a structure. Avoid guessing offsets from the declaration. Embedded systems sometimes depend on a vendor-defined register structure layout, but that dependency should be explicit, reviewed against the compiler ABI, and separated from protocol serialization.

A structure assignment copies the member values, including pointer *values*, not heap allocations reachable through those pointers. If a record contains `char *name`, copying the record makes two references to the same storage. It does not perform a deep copy, extend the pointed-to object's lifetime, or establish a second independent owner.

5.10 Flexible array members

A final structure member can be a flexible array. It has no fixed element count in the structure's definition:

```
#include <stddef.h>
#include <stdlib.h>
#include <stdint.h>

struct message {
    size_t length;
    unsigned char bytes[];
};
```

A caller may allocate extra storage sufficient for the header and a requested number of byte elements. The size computation must be checked against `SIZE_MAX` before the call to `malloc`, and the allocated object must remain alive during access. This representation is useful for variable-length messages in one allocation, but it is inappropriate when a strictly bounded static buffer is preferable.

Flexible array members have restrictions: they belong at the end of an eligible structure, and the containing type cannot be used in every context where an ordinary fixed-size structure can. They also do not turn the member into a bounds-checked array; the actual allocation size still matters.

5.11 Layout-independent serialization

A portable encoder establishes each output byte deliberately. Consider a record with an 8-bit kind and 16-bit little-endian length. Its wire representation can use exactly three octets regardless of padding in the in-memory structure:

```
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

bool encode_header(unsigned char *dst, size_t size,
                   uint8_t kind, uint16_t length)
{
    if (dst == nullptr || size < 3) return false;
    dst[0] = kind;
    dst[1] = (unsigned char)length;
    dst[2] = (unsigned char)(length >> 8);
    return true;
}
```

This example assumes an octet-oriented destination and should be used only where the target's byte representation meets that contract. The field width and endianness are design decisions, independent of the host machine.

5.12 Data-structure choice follows requirements

A linked list offers insertion without moving contiguous records, but each node needs storage and a link, and traversal is not random access. A fixed array has locality and predictable overall capacity but may need to shift elements during insertion. A ring buffer is excellent for FIFO traffic but does not support arbitrary deletion efficiently.

For an MCU message queue with a known maximum occupancy, the fixed array or ring buffer may be easier to justify than a heap-backed list. If the queue is shared with an interrupt, however, a nonconcurrent ring implementation remains unsafe without a synchronization design. Capacity, ownership, timing, and concurrency are separate requirements.

Further problems: Implement a deep-copy operation for a record owning a dynamically allocated string. Show how to reject a flexible-array allocation whose size calculation overflows. Write an encoder and decoder for the three-octet header, testing both directions and malformed lengths.

5.13 A structure is a value with invariants

A structure gives related fields one type, but the programmer decides which combinations of fields are meaningful. Imagine a temperature reading containing a numeric value, a unit identifier, and a validity flag. A declaration permits all combinations; the interface should say which represent actual measurements.

One design creates values through a constructor-like function that validates the inputs. An alternative stores an explicit status enumeration instead of a boolean flag so it can represent sensor-not-ready, disconnected, and measured states separately. The point is not to make every structure opaque; it is to choose a representation whose invariants are easy to state and preserve.

Designated initialization helps when a structure evolves because it names the intended members. A missing member is initialized according to the appropriate language rules, which may or may not express a valid application value. A status enumeration whose zero value has no defined meaning can produce a logically invalid initialized object despite being a perfectly valid C object.

5.14 Padding and equality

Two structures can have equal corresponding members while their padding bytes differ. Using `memcmp(&a, &b, sizeof a)` to test semantic equality is therefore not a general solution. Compare the meaningful fields instead:

```
struct measurement { unsigned id; double value; };

bool same_measurement(struct measurement a, struct measurement b)
{
    return a.id == b.id && a.value == b.value;
}
```

The floating comparison here deliberately treats NaNs as unequal; whether this is appropriate depends on the application's data model. If bitwise reproducibility matters, define representation-specific behavior rather than assuming the members' == operators implement it.

For serialization, encode only documented fields in a documented order. Writing `sizeof(struct measurement)` bytes to a file can unintentionally expose padding bytes and platform-specific floating representations.

5.15 Tagged unions and exhaustiveness

A union can store one of several representations in shared storage. It does not automatically record which interpretation is active. A common approach pairs the union with a tag:

```
enum value_kind { VALUE_INT, VALUE_FLOAT };
struct value {
    enum value_kind kind;
    union { int integer; double real; } data;
};
```

A function consuming `struct value` should examine the tag before accessing the corresponding member. C23 does not provide algebraic data types with exhaustive pattern matching; a `switch` over an enum can still have invalid values due to corrupted input or casts. An explicit default error path is useful at trust boundaries.

A tagged union is particularly effective for a parser returning one of several record kinds. It makes illegal reinterpretations easier to detect in code review and separates the logical type tag from the memory representation of the payload.

5.16 Intrusive versus owning lists

A linked-list node can own its payload, or the payload can contain the linkage needed to participate in the list. An intrusive list avoids a separate node allocation but ties the item's lifetime to list membership. An owning list may allocate a node and payload together or separately.

Before writing the first insert function, choose whether the list owns items. Can a caller free an item while it is linked? Can an item belong to two lists? What does removing an element do to its lifetime? These questions determine correctness more than the syntax of `next`.

On microcontrollers, a fixed pool of nodes and explicit occupancy flags may be preferable to unbounded heap allocation. It bounds memory, can eliminate allocation fragmentation, and makes exhaustion an ordinary status that tests can exercise.

5.17 Ring buffer wraparound

The earlier queue has capacity eight, with independent `head` and `count` fields. To find the tail, it computes `(head + count) % capacity`. This works because the queue invariant constrains `head` and `count`, and their sum is small enough to avoid overflowing `size_t`.

Trace a full cycle. Push values 0 through 7; the queue is full. A ninth push fails without modifying state. Pop 0, 1, and 2; now head is 3 and count is 5. Push 8, 9, and 10; the physical array slots reused are 0, 1, and 2. Subsequent pops return 3 through 10 in order.

The same logical sequence can be represented differently in physical storage. Clients should observe only the queue's documented behavior, not assume elements are contiguous in chronological order inside its array.

Exercises

5.9 Write an equality function for a structure containing an integer, a floating value, and an optional identifier; state the NaN policy. **5.10** Add a status tag to a variant record and reject an unsupported tag on decoding. **5.11** Design a fixed-capacity node pool for a list and describe release behavior. **5.12** Trace ring-buffer state after each push and pop, identifying array index and logical position.

6. Input, Output, and Files

C's hosted I/O library is built around streams. A stream may refer to a terminal, ordinary file, or implementation-specific source. Errors should be treated as part of the computation.

6.1 Reading lines safely

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char line[256];
    while (fgets(line, sizeof line, stdin) != nullptr) {
        size_t n = strlen(line);
        bool complete = n > 0 && line[n - 1] == '\n';
```

```
        printf("bytes=%zu complete=%s\n", n, complete ? "yes" : "no");
    }
    if (ferror(stdin)) {
        fputs("input error\n", stderr);
        return 1;
    }
}
```

A full buffer does not imply a complete line. The next `fgets` may read the rest. Embedded applications frequently need a bounded protocol parser rather than a line-oriented parser.

6.2 Formatted I/O

`printf` uses a format string to interpret subsequent arguments. A mismatch between conversion specifications and argument types can yield undefined behavior. Never use untrusted input as the format string:

```
printf("%s", user_text); /* fixed format */
```

For integers of exact width, use `<inttypes.h>` formatting macros. For `size_t`, use `%zu`. Avoid relying on the fact that two types happen to share a representation on one platform.

6.3 Binary I/O

```
unsigned char buffer[1024];
size_t got = fread(buffer, 1, sizeof buffer, stdin);
if (got < sizeof buffer && ferror(stdin)) {
    fputs("read failure\n", stderr);
}
```

A binary protocol needs a documented byte order, field widths, bounds, and error policy. Reading bytes into a native structure is not sufficient. The portable approach decodes fields explicitly.

6.4 Files and cleanup

```
FILE *fp = fopen("output.txt", "w");
if (fp == nullptr) {
    perror("output.txt");
    return 1;
}
if (fputs("record\n", fp) == EOF || fclose(fp) == EOF)
    return 1;
```

The short example needs care: if `fputs` fails, short-circuit evaluation skips `fclose`, leaking the open stream. Real cleanup logic must close the file on every path, then report either the write error or the close error. A successful write call does not guarantee that data is durably stored.

Exercises. 6.1 Write a line counter that identifies I/O failure. 6.2 Read a little-endian 32-bit unsigned integer from four bytes. 6.3 Rewrite the file example so it always closes its stream.

6.5 The difference between text and bytes

A text stream may transform representations, notably line endings on some systems. Binary I/O is suited to exact bytes. If a file is a protocol packet or serialized image, open it with `"rb"` or `"wb"` in hosted implementations and define the byte format explicitly.

A read of `n` bytes is not guaranteed to deliver `n` bytes in one call. A short `fread` result must be interpreted using `feof` and `ferror`, and the caller must decide whether a short record is valid or malformed.

6.6 A bounded line counter

Some tasks need only one pass over input and almost no memory:

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

bool count_newlines(FILE *stream, unsigned long *result)
{
    if (stream == nullptr || result == nullptr)
        return false;

    unsigned long count = 0;
    int ch;
    while ((ch = fgetc(stream)) != EOF) {
        if (ch == '\n') {
            if (count == ULONG_MAX)
                return false;
            ++count;
        }
    }
    if (ferror(stream))
        return false;
    *result = count;
    return true;
}
```

The function counts newline characters, which may differ from a human notion of lines if the final input has no trailing newline. Documenting that distinction matters more than choosing one rule universally. The caller owns the stream, so this function neither opens nor closes it.

6.7 Reading numeric text

Using `scanf` indiscriminately can make validation difficult: conversion success, partial input, range errors, and leftover text all matter. For untrusted input, reading a bounded line and then using `strtol` or another checked parser often exposes the contract more clearly.

A parser should specify whether whitespace is accepted, whether leading `+` or `-` is legal, whether trailing characters are rejected, and the range of accepted values. A caller should not infer all of those policies from the function name alone.

6.8 File cleanup paths

A file-writing function commonly needs two independent success checks: whether the write succeeded, and whether closing the stream succeeded. Avoid short-circuiting away the close operation:

```
bool ok = fputs("record\n", fp) != EOF;
if (fclose(fp) == EOF)
    ok = false;
```

This makes the cleanup action unconditional once the stream has been opened successfully. It does not guarantee that written data is durable after a power loss, since operating systems and storage devices may buffer writes.

6.9 Exercises

6.4 Define whether a final non-newline-terminated sequence counts as a line, then implement your policy. **6.5** Decode a four-byte length prefix and reject a payload that would exceed a caller-specified capacity. **6.6** Compare a binary stream with a text stream on a platform that transforms line endings. **6.7** Design error reporting that preserves both a write failure and a close failure. **6.8** Explain why a successful `fwrite` is not a promise of power-failure durability.

6.10 A byte-stream copier

Reading and writing streams is not a single operation when short transfers are possible. A robust file copier reads a bounded block, then writes every byte obtained or reports an error. The `fwrite` interface returns the number of complete elements written; when the element size is one byte, that count can be compared directly with the requested byte count.

The repository's `examples/file_copy.c` illustrates this pattern for standard input and output. It checks `ferror(stdin)` when a short read occurs and `fflush(stdout)` before returning success. It still does not guarantee durability against an operating-system crash or power loss. That stronger guarantee is outside the portable C stream contract and may require platform-specific synchronization.

A binary copier preserves byte values; a text-processing program may interpret newline conventions, encodings, and locale. The distinction should be part of the program's specification.

6.11 Why parsing after input is often easier

An unchecked numeric scan mixes syntax recognition with conversion. It can be difficult to determine whether a trailing character remains or a number was only partially consumed. Separating operations makes behavior explicit: read a bounded line; determine whether the entire line matches the required grammar; then perform checked numeric conversion.

For example, a temperature configuration file might permit optional leading whitespace but forbid any trailing nonspace characters. That is a different grammar from a telemetry message consisting of exactly four decimal digits. Neither grammar is universally correct. A well-designed parser names and tests the distinction.

Use `strtol` or `strtoul` when their grammar matches the intended input. Set `errno` to zero before the call, use the end pointer to detect whether any digits were converted and whether unconsumed input remains, and inspect `ERANGE`. Then check that the result fits the application type. A successful library conversion is not proof that all application constraints were met.

6.12 Error reporting preserves the cause

A function returning only zero or one may be sufficient for a toy program. In a larger system, distinguish an open failure, a read error, malformed content, a short write, and a close failure. These conditions have different remedies and should be logged or propagated without losing their meaning.

If a write fails and closing the file also fails, what should the caller see? One design returns the primary failure and logs cleanup failures separately. Another uses a structured result containing both statuses. The choice should be deliberate. Simply overwriting the first error with the last operation's result can hide the original problem.

6.13 Buffering and interleaving

Stream output may be buffered. `fflush` requests transmission of the C library's buffered output to the host environment, not permanent storage. Mixing multiple threads' writes or mixing `stdio` with lower-level operating-system file descriptors can introduce ordering and buffering issues.

Portable code should synchronize concurrent stream operations appropriately and should not assume that a line-oriented message is always an indivisible transaction.

6.14 Designing a testable input function

Instead of making a parser call `getchar` itself, define a function accepting `const unsigned char *input` and a `size_t` length. Then a small adapter reads from a stream and passes counted buffers to the parser. Host tests can exercise empty input, malformed records, and boundaries without inventing a terminal or fake file.

Further problems: Specify how a line reader handles a trailing newline, embedded NUL, and a buffer too small for an entire line. Implement a status type distinguishing EOF from error. Explain why `ferror` must be consulted after an I/O operation fails rather than treating every `EOF` result as a clean end of data.

6.15 Streams, records, and short transfers

The C stream library exposes a sequence of bytes or characters; it does not magically preserve an application's record boundaries. Reading 100 bytes from a binary file says nothing about where one protocol record ends. If the format has a length field, the parser must validate that field and obtain exactly the corresponding number of bytes or explicitly report incomplete input.

A complete fixed-size record reader may loop until it has accumulated the required count, encounters an error, or reaches clean EOF before the record is complete. These outcomes are different. In particular, clean EOF between records is normal, whereas EOF in the middle of a required record often indicates truncation.

For an interactive terminal, line processing introduces other questions. What counts as end-of-line on this host? Is the input a text stream or a binary stream? How should excessively long lines be rejected or drained? Do not use `fgets` as if it guaranteed one whole logical line when its buffer is finite.

6.16 Write operations have partial-success semantics

A `fwrite` call returns the number of complete elements written, and a short result signals failure under its contract. For arbitrary byte counts, choose an element size of one and compare the returned count with the requested bytes. In a broader abstraction over operating-system calls, writes may also return a positive short count and require a loop to finish the request. Do not conflate the contracts of different APIs.

A robust writer distinguishes the amount of data accepted, the error status, and whether retry is meaningful. On nonblocking devices, a temporary inability to write should not necessarily be treated as fatal. Conversely, looping forever on a zero-byte result would violate bounded-progress expectations.

When writing a file, checking `fclose` remains important because some errors can surface after prior buffered operations appeared successful. Even a successful close does not universally mean data is physically durable after sudden power loss.

6.17 Use a parser independent of the transport

A reusable packet decoder can accept a pointer, length, and output destination:

```
bool decode_packet(const unsigned char *bytes,
                   size_t length,
                   struct packet *out);
```

The parser does not call `getchar`, open a file, or read a UART register. Instead, adapters gather bytes from those sources and call the same decoder. A desktop test can supply a deliberately truncated packet, an invalid length, or a checksum mismatch without requiring hardware. Device-level tests can then focus on acquisition and timing.

There is a cost: the adapter must buffer enough bytes for a record, or the parser must be designed as a streaming state machine. The right choice depends on maximum frame size and available RAM.

6.18 Standard output and error output

`stdout` and `stderr` serve different roles. Machine-readable output should be written to `stdout`, while explanatory error messages usually belong on `stderr`. A program that produces a table can then be piped into another tool without contaminating its data with diagnostics.

For command-line tools, establish an exit-status convention: zero for success and nonzero for failure. If there are many error categories, the exact exit values may be less informative than a precise error message, but the contract should still be predictable.

An embedded firmware component may not have any of these streams. It may instead return a structured status to its caller or emit a diagnostic over a separately configured telemetry channel. The error model survives even when the transport differs.

6.19 A defensive byte-count example

Suppose a header says the payload is `length` bytes, and the caller has a `capacity`-byte buffer. Reject an oversized payload before copying. If the total frame needs five extra octets, check `capacity >= 5` and then `length <= capacity - 5`. A direct test of `length + 5 <= capacity` can overflow unsigned arithmetic before the comparison.

This tiny transformation is a recurring pattern in systems programming: convert a potentially overflowing sum into a subtraction after establishing its precondition.

Exercises

6.9 Design a status enumeration for clean EOF, incomplete record, I/O error, and valid record. **6.10** Implement a line reader that rejects overlong input but drains until newline so it can resynchronize. **6.11** Write a host test for a packet decoder that uses no filesystem calls. **6.12** Explain when checking `fflush` and `fclose` matters and what neither operation guarantees about power-loss durability.

7. Dynamic Memory and Ownership

Every object occupies storage for a particular lifetime. Automatic objects usually live for one block invocation. Static objects live for the execution of the program. Dynamic objects, obtained through allocation functions, persist until explicitly freed.

7.1 Allocation is a request

```
#include <stdlib.h>
#include <stdint.h>
#include <stddef.h>

int *make_array(size_t count)
{
    if (count == 0 || count > SIZE_MAX / sizeof(int))
```

```
        return nullptr;
    return malloc(count * sizeof(int));
}
```

The multiplication is checked before it can overflow. This interface rejects zero-sized requests. Other interfaces may choose different zero-size semantics, but they should document them. New storage from `malloc` has indeterminate contents. Use `calloc` for zero-initialized storage, while still checking the requested dimensions.

7.2 One owner, one release

```
int *samples = make_array(100);
if (samples == nullptr)
    return 1;
/* use samples[0] through samples[99] */
free(samples);
samples = nullptr;
```

The variable `samples` is not the allocated object; it holds an address. After `free`, all aliases that would access the old object are invalid. Resetting one variable does not repair copies of the address.

A useful API states: who allocates, who frees, whether buffers may be shared, which functions can retain pointers, and which operations invalidate earlier pointers. These are conventions implemented by programmers, not runtime guarantees supplied by the language.

7.3 Resize without losing the original

```
void *tmp = realloc(buffer, new_size);
if (tmp == nullptr) {
    /* buffer remains allocated when new_size > 0 */
    return false;
}
buffer = tmp;
```

The example assumes `new_size > 0` and that `buffer` is a valid reallocatable allocation. Assigning `realloc` directly to the only saved pointer risks a leak on failure. Successful reallocation may move storage, invalidating interior pointers.

7.4 Fixed-capacity alternatives

Embedded systems often trade flexible allocation for predictable resource bounds. A fixed-size arena can allocate successive objects from a caller-provided array. It must honor alignment and report exhaustion; an arena reset invalidates all objects in it. Pools that recycle uniform slots provide a different tradeoff. Neither allocator model is automatically thread-safe.

7.5 A resource table

Resource	Acquisition	Release
Heap block	`malloc`, `calloc`	`free`
File stream	`fopen`	`fclose`
Automatic array	enter block	leave block
Static object	program initialization	program termination

Exercises. 7.1 Implement checked dynamic-array growth. 7.2 Write a fixed-capacity arena with explicit alignment policy. 7.3 Identify double-free, leak, and use-after-free paths in a linked-list teardown.

7.6 A checked growable vector

A dynamic array needs three measurements: the pointer to the allocation, the number of initialized elements, and its capacity. The invariant is `length <= capacity`. Growing is exceptional, not required on every insertion.

```
struct vector {
    int *data;
    size_t length;
    size_t capacity;
};
```

A growth policy might double capacity, but the multiplication needs an overflow check. When capacity cannot double, the allocator must either use a smaller sufficient capacity or report failure. The success of `realloc` invalidates pointers into the old allocation even if the returned address happens to equal the old address; callers should obtain fresh pointers from the vector after a change.

The vector's *ownership* can be stated simply: it owns `data`, its caller owns the vector structure, and exactly one destroy operation releases the allocation. Copying the structure by assignment copies its pointer, not its ownership. Two such copies must not both be destroyed.

7.7 An arena with an explicit alignment policy

An arena hands out storage from a fixed block. Its main benefits are bounded overall consumption and cheap bulk reset. Correct alignment must be established for each allocation. A simple arena supporting only objects with alignment no stricter than `max_align_t` can use storage suitably aligned at its base, round offsets upward without overflow, and reject requests that do not fit.

Alignment is part of the contract, not a detail to ignore because a particular CPU tolerates misaligned loads. DMA buffers and cache maintenance impose further target-specific constraints that a generic C arena does not automatically meet.

7.8 Failure-safe initialization

Consider creating an object that owns a buffer and opens a file. If the second step fails, the first resource must be released. Design initialization so that either every required resource is acquired, or every partial acquisition is undone. A cleanup label can be a sensible C technique when a function has many resource-acquisition steps.

```
int status = 1;
void *buffer = malloc(1024);
if (buffer == nullptr) goto done;
FILE *file = fopen("data.bin", "rb");
if (file == nullptr) goto done;
/* use resources */
if (fclose(file) != 0) goto done;
status = 0;
done:
free(buffer);
return status;
```

This sketch requires `<stdio.h>` and `<stdlib.h>`. It closes the file on the successful path, but it is not a general model for cleanup after all possible failures within the use block. For several resources, initialize handles to known values and centralize cleanup that checks each handle.

7.9 Exercises

7.4 Implement a vector with `init`, `append`, and `destroy`. **7.5** Demonstrate how copying a vector structure creates two pointer aliases. **7.6** Design a resettable arena and describe which pointers become invalid after reset. **7.7** Write a function that acquires two resources and releases both on every failure path.

7.10 Distinguish capacity from initialized length

An allocation of space for ten integers does not automatically initialize ten integer values. With `malloc`, reading elements before they are assigned is invalid. With `calloc`, the allocated bytes are zero-initialized; this yields the all-bits-zero representation of every byte, which for integer objects gives the value zero. For pointers and some other types, all-bits-zero bytes are not universally a promise of the semantic null or zero value on every conceivable C implementation.

A vector records both allocated *capacity* and initialized *length*. An append operation writes an element into unused capacity before increasing the length. A destroy operation releases the allocation once and invalidates all borrowed element pointers.

When designing a C API, use distinct names for these concepts. Confusing `length` and `capacity` is a recurring source of out-of-bounds reads and writes.

7.11 Zero-size requests in C23

A historical compatibility trap deserves explicit attention. In C23, calling `realloc(ptr, 0)` has undefined behavior. Avoid using it as a shorthand for freeing a block. Instead, use `free(ptr)` when the desired operation is release, and define an explicit empty-state policy in your container API.

The behavior of `malloc(0)` is not a portable way to obtain a usable object. An API can reject zero-sized element arrays, represent them with a null pointer and zero capacity, or use another documented convention. The key is consistency at every call site.

7.12 Checked growth without losing storage

To grow an array, first choose a new capacity greater than or equal to the required length; then check whether `capacity * sizeof element` fits in `size_t`; then call `realloc` using a temporary pointer. Only on success do you update the stored pointer and capacity.

The example vector in `examples/vector.c` follows this approach for simple `int` elements. Because it uses a doubling policy, a very large capacity may be rejected rather than trying every possible smaller capacity. That is an acceptable documented policy for a teaching example; it is not the only valid design.

No pointer into an allocation may be assumed valid after a successful `realloc`. Recompute interior pointers from the returned base. A failed nonzero-size `realloc` leaves the old allocation intact, which is why assigning the result directly to the only owner is dangerous.

7.13 Alignment and arenas

C object types have alignment requirements. A byte arena can hand out storage for types only when it aligns the returned address appropriately and has enough remaining bytes. Rounding an

offset upward can overflow; check the arithmetic. A generic aligned allocator must consider the actual address of the backing allocation, not just an offset from an arbitrarily aligned base.

If an arena is reset, every allocation it had issued becomes invalid for application purposes, even though the bytes physically remain. The arena is attractive when a whole group of temporary objects has the same lifetime. A free-list pool is better when individual fixed-size objects must be reclaimed independently.

7.14 Failure injection as a learning tool

Ordinary runs rarely exercise allocation failures. For a unit test, wrap `malloc` and `realloc` behind function pointers or an allocator object, then provide a test implementation that fails on a specified request. Confirm that failure neither leaks the old allocation nor changes the vector's logical length or existing element values.

This is a useful general lesson: error paths deserve executable tests, not just reassuring comments.

Further problems: Modify the vector to distinguish allocation failure from arithmetic overflow. Implement `vector_reserve` that leaves all state untouched on failure. State the ownership implications of `struct vector copy = original;`. Write an arena allocation test covering zero bytes, alignment and exhaustion.

7.15 Ownership as an interface contract

C allocation functions do not attach a runtime owner to each block. Ownership is a design convention: one component is responsible for eventually releasing a resource, while other components may temporarily borrow access to it.

A vector may own its element allocation. A slice into that vector borrows the memory. When the vector grows with `realloc`, every previously borrowed element pointer must be considered invalid, even if the new allocation happens to return the same numerical address.

A useful public header should state whether a function borrows its input, retains it after returning, or transfers ownership. For example, `vector_append` can copy an `int` value and retain no caller pointer. An asynchronous send operation that merely borrows a buffer may require the caller to keep that buffer alive until a completion callback. An API that copies the payload on submission has a different resource and timing contract.

These distinctions do not require a new language feature. They require precise documentation and tests.

7.16 Strong and basic failure guarantees

An append operation can promise that a failed allocation leaves the vector exactly as it was: same pointer, length, capacity, and existing elements. That is a strong failure guarantee. A more complex operation might promise only that the structure remains valid and usable, even though its contents have changed; that is a weaker but sometimes acceptable guarantee.

To achieve strong failure behavior, compute requirements locally, allocate into a temporary pointer, and commit state only after success. With `realloc`, a failed nonzero-size request leaves the old object allocated, so the implementation must not overwrite its only saved pointer with a null result.

The choice of a growth factor also affects behavior. Doubling gives amortized efficient append in the ordinary case, but may request more memory than immediately needed and can reject near-maximum capacities. A constrained embedded application may prefer fixed capacity or an arena with known bounds.

7.17 A checked vector implementation

The companion `examples/vector_checked.c` implements a small vector of `int`. It has initialization through an empty aggregate, append with checked capacity arithmetic, retrieval through an out-parameter, and destruction.

Its invariants are:

- `length <= capacity`;
- `data == nullptr` when capacity is zero;
- for positive capacity, `data` is an owned allocation of at least `capacity * sizeof(int)` bytes;
- elements in the half-open range `[0, length)` have been assigned meaningful values.

The append path computes a new capacity, checks both doubling and allocation-byte multiplication, and performs reallocation through a temporary pointer. If allocation fails, its earlier state remains intact. The test checks many appends, retrieval at boundaries, and destruction.

A complete library would also consider whether copying the `struct vector` itself is permitted. A shallow copy creates two apparent owners of the same allocation. For this teaching interface, copying an owning vector by assignment is prohibited by the ownership contract. C cannot enforce that restriction in the type system.

7.18 How to use an arena safely

An arena can provide a bounded amount of dynamic-like storage without making a separate operating-system allocation for each item. But an arena still needs an alignment policy. A request for a `double` object must be aligned suitably for `double`; a byte pointer incremented without regard to alignment is not sufficient.

When resetting the arena, think of every borrowed pointer into it as invalidated by the allocator's application-level lifetime rules. Even if old bytes remain readable on the hardware, the program must not depend on objects whose storage has been repurposed.

A simple arena is not automatically thread-safe. The same considerations apply to a pool of fixed-size objects. Concurrency, memory ordering, and synchronization remain system-level responsibilities.

7.19 Resource cleanup as a state machine

A function acquiring two independent resources can be modeled with states: neither acquired, buffer acquired, buffer and file acquired, and both released. Every failure transition must eventually reach the released state.

One C technique uses a single cleanup block with handles initialized to known safe values. Another uses small helper functions that release partial objects. Neither technique is inherently better in all contexts. The key is that cleanup must run on every exit path that owns resources, including paths triggered by short reads, failures to close a stream, and errors during an intermediate transformation.

Avoid placing release operations inside a boolean expression that short-circuits when an earlier operation fails. Such expressions can unintentionally skip cleanup.

7.20 Testing the allocator's failure paths

Ordinary desktop runs may never fail a small allocation, leaving a large fraction of an owning container's control flow untested. A test allocator can reject the next request predictably. With this seam, verify that failed append does not change length, capacity, pointer, or previously stored values.

Sanitizers can discover use-after-free and certain out-of-bounds errors along executed paths, but they do not prove the ownership design is correct. A written ownership contract plus fault-injection tests is a stronger foundation.

Guided exercises

7.8 Add `reserve` to the vector while preserving the strong failure guarantee. **7.9** Implement an allocator adapter that fails after N successful requests. **7.10** Explain why a shallow copy of the vector must not be destroyed independently. **7.11** Contrast an arena reset with `free` on an individual block. **7.12** Design a fixed-memory message pool for 32 records with allocation, release, and double-release detection.

8. The Preprocessor and Separate Compilation

The C preprocessor transforms translation units before compilation. It helps share declarations and configure programs, but its token-level substitutions must not be mistaken for typed functions.

8.1 Headers describe interfaces

```
/* counter.h */
#ifndef COUNTER_H
#define COUNTER_H

#include <stddef.h>
struct counter { size_t value; };
void counter_increment(struct counter *c);

#endif
```

```
/* counter.c */
#include "counter.h"
void counter_increment(struct counter *c)
{
    if (c != nullptr) ++c->value;
}
```

The header is included by both the definition and callers, allowing the compiler to check their consistency. The include guard prevents repeated definitions within a translation unit.

8.2 Prefer typed operations

```
#define SQUARE(x) ((x) * (x))
```

This macro evaluates its argument twice; `SQUARE(i++)` produces undefined behavior due to unsequenced modifications. A function avoids repeated evaluation:

```
static inline int square(int x) { return x * x; }
```

The function has one parameter evaluation, though signed multiplication can still overflow. `inline` is not a promise that machine code will contain no call.

8.3 Conditional compilation

```
#if defined(DEVICE_TARGET)
    /* device-specific interface */
#else
```

```
    /* host implementation */
#endif
```

Keep conditional code small and localized. C23 adds `#elifdef` and `#elifndef` as clearer forms for common conditional cases. Feature-test macros should verify actual support rather than assume the entire standard library is implemented because a compiler accepts `-std=c23`.

8.4 Compile and link

For two files:

```
cc -std=c23 -Wall -Wextra -Wpedantic -c counter.c
cc -std=c23 -Wall -Wextra -Wpedantic -c main.c
cc counter.o main.o -o program
```

The compilation stage separately translates source files; the linker resolves symbols and constructs the executable. An undefined reference usually means the program declared a symbol but no linked object provided its definition.

Exercises. 8.1 Write a minimal `Makefile` for the two-source example. 8.2 Replace a parameterized macro with an inline function. 8.3 Explain the distinction among declaration, tentative definition, and external definition.

8.5 What belongs in an interface?

A header exposes names and types necessary for clients. A private source file keeps the implementation's helper functions, internal objects, and algorithms. Public headers should be self-contained: including one in an otherwise empty translation unit should not require secret prior includes.

A useful test compiles a short file containing only the header and a minimal `main`. This catches hidden dependencies that may otherwise be masked by the include order in a larger build.

8.6 Macros and evaluation hazards

Function-like macros are token substitution, not calls. This matters for evaluation count, operator binding, statement boundaries, and type checking. Compare:

```
#define MIN_BAD(a, b) ((a) < (b) ? (a) : (b))
static inline int min_int(int a, int b)
{
    return a < b ? a : b;
}
```

The macro may evaluate one argument twice, while the function evaluates each call argument once. Even the function does not specify which argument expression is evaluated first; do not write code that depends on call-argument evaluation order.

8.7 Build modes and reproducibility

A program's observed behavior depends on source code, implementation, target, optimization settings, language standard, and linked libraries. Keep warning flags in normal development builds. Use optimization during testing as well as debugging, since undefined behavior may appear differently at different optimization levels.

The C standard does not dictate a build system. A small Makefile can teach separate compilation effectively. Larger projects benefit from explicit dependency tracking, toolchain files, and reproducible CI environments.

8.8 Defensive configuration

Prefer feature tests to compiler-name checks. Detecting a compiler vendor does not prove that a header is available or that a function links with the intended C library. For a C23 library feature, compile and link a minimal program using it before enabling the code path. Document fallbacks for older supported targets.

8.9 Exercises

8.4 Write a header-only compile probe. **8.5** Show why a macro with arguments containing ++ can change program behavior. **8.6** Build a two-file project with a separate `-c` step and explain a linker error. **8.7** Define what toolchain details must be recorded to reproduce a firmware build.

8.10 A source file is not an isolated program

Suppose one source file includes a header declaring `int calculate(int);`, and another defines `double calculate(double)`. Each source can appear locally plausible, but the program violates the intended cross-file interface. Include the same header in the implementation file, build with warnings enabled, and avoid hand-copied prototypes.

At link time, the toolchain resolves external references. A linker error about an undefined reference can mean the implementation object was not linked, the identifier has internal linkage, or the symbol spelling does not match. Distinguish such linkage errors from a compiler diagnosis about a malformed declaration.

8.11 Build a tiny library deliberately

A three-file project can contain `counter.h`, `counter.c`, and `main.c`. Compile the implementation and main program into separate object files, then link them. In a Makefile, express dependencies so a changed header causes both translation units to rebuild. A project that rebuilds only the implementation after changing a public structure definition may link stale object code with an incompatible layout.

For larger projects, generated dependency files or a build system such as CMake can automate this bookkeeping, but understanding the underlying dependency graph remains valuable.

8.12 Host and target are different builds

An embedded program may compile once for a desktop host and once for an Arm microcontroller. The compiler, target ABI, linker script, startup code, libraries, and permissible memory layout can be different. A program's `sizeof(long)`, pointer representation, object alignment and supported atomics must not be inferred from the host build.

Keep portable algorithms in files that both environments can compile. Isolate peripheral drivers and target-specific register definitions behind narrow interfaces. This makes host-side unit tests useful without pretending they replace hardware validation.

8.13 Preprocessor conditions versus feature probes

A macro such as `__STDC_VERSION__` can tell you which language revision an implementation claims for a translation mode. It cannot prove that optional facilities or the desired target C library are available. To depend on `<stdckdint.h>`, compile and link a small test that actually includes the header and calls a checked arithmetic operation. To depend on `#embed`, compile an actual embedding directive using the chosen toolchain.

The distinction matters because a C front end and its runtime library are often delivered separately. Firmware toolchains may ship specialized or partial libraries.

8.14 Keep generated output reproducible

The manuscript's PDF is generated from Markdown. A reliable release records the source commit, document toolchain versions, fonts, and build command. The page count and layout can change when these vary. Committing a PDF without a verified successful build may give readers a stale document that does not match the latest chapters.

Use a CI workflow to build, inspect, and publish the PDF after source changes, with an explicit failure whenever required tools or referenced files are missing. An automation configuration in a repository is a plan for a build, not evidence that the build actually passed.

Further problems: Write Makefile rules for two object files sharing a header. Explain the meaning of `-c`. Design a two-compiler CI matrix and identify how to distinguish compiler-support failure from a programming error.

8.15 A real three-file C module

A header becomes useful when more than one translation unit agrees to an interface. The companion example under `examples/modules/` consists of a header, an implementation, and a separate client program. It computes a checksum over a bounded byte span. The calculation is intentionally elementary; the lesson is how declarations, definitions, and build dependencies cooperate.

The header contains the declaration:

```
#ifndef BOOK_CHECKSUM_H
#define BOOK_CHECKSUM_H

#include <stddef.h>

unsigned book_checksum(const unsigned char *bytes, size_t count);

#endif
```

The implementation includes that same header, rather than independently restating the function's type. It also places any helper functions under internal linkage with `static`. The client includes the header and calls the function. The build compiles both source files and links their object code.

Why insist that the implementation include its own header? Suppose a developer changes the parameter from `size_t` to `unsigned` in only the implementation. Without the header, different translation units could compile against incompatible assumptions. With a common header, the compiler can diagnose disagreement where the function is defined.

8.16 Rebuild dependencies

A Makefile must know more than the final executable name. If `checksum.h` changes, both the implementation that defines the operation and the client that calls it may need recompilation. For a small demonstration, an explicit recipe that depends on all three files is sufficient:

```
modules_demo: examples/modules/main.c examples/modules/checksum.c examples/modules/checksum.h
    $(CC) $(CFLAGS) examples/modules/main.c examples/modules/checksum.c -o $@
```

For larger projects, compile to separate object files and generate header dependencies automatically. Otherwise a stale object file may be linked against an updated structure layout or changed interface.

This particular checksum is not a cryptographic digest. It is a teaching example for module boundaries and input-length contracts. Using it for message authentication would be a security design error.

8.17 Header hygiene and include order

A well-designed public header should compile when included first in an otherwise minimal translation unit. It should include the standard headers needed by its own declarations instead of relying on the client to include them beforehand.

Include guards prevent repeated textual inclusion within one translation unit, not multiple external definitions across separate source files. Define ordinary global storage in one implementation file; put only an `extern` declaration in the header if clients truly need to refer to it.

An API can often avoid exporting global state at all. Pass a context structure or opaque handle through function arguments so tests can create independent instances without accidental interference.

8.18 Libraries are composed, not merely named

A static library usually packages compiled object files into an archive. A shared library adds platform-specific dynamic-linking rules and lifetime constraints. The C language itself specifies translation and linkage concepts but does not require a particular operating-system library format.

An embedded firmware image may link application code, an RTOS, a runtime library, startup code and a linker script in one final step. The C code can be portable while its final link process is highly target-specific.

Exercises

8.8 Compile the module example with separate `-c` steps. **8.9** Remove the implementation file from the link command and explain the resulting diagnostic. **8.10** Delete `<stddef.h>` from the header and determine why standalone inclusion now fails. **8.11** Add a unit-test client using the same header but a different `main`.

9. The C23 Language

C23 is the 2024 revision of the ISO C standard, commonly called C23. It modernizes familiar C without turning it into C++. Its features are best adopted where they improve clarity or correctness; portable projects must still verify compiler and standard-library implementation support.

9.1 Booleans and null pointers

```c
bool valid = true;
int *ptr = nullptr;
if (!valid || ptr == nullptr) {
    /* handle absence */
}
```

`bool`, `true`, and `false` are language keywords in C23. `nullptr` has type `nullptr_t` and denotes a null pointer constant without relying on a macro such as `NULL`.

9.2 Type inference with auto and typeof

```c
int count = 3;
auto copy = count;            /* copy has type int */
typeof(count) another = 4;   /* another has type int */
```

The C23 `auto` form is a type inference declaration, not the C++ rules in their entirety. `typeof` provides a type based on another expression or type name. C23 also supplies `typeof_unqual`,

discarding qualifiers and atomicity from the resulting type in its specified way. These mechanisms can reduce duplicated types but should not obscure API contracts.

9.3 Constant expressions

```
constexpr int channels = 4;
int measurements[channels];
```

C23 introduces `constexpr` for certain objects initialized by constant expressions. This is not C++ compile-time function evaluation. Keep the distinction between an expression computable at translation time and a runtime `const` object.

9.4 Width-aware integers

Where supported, `_BitInt(N)` and `unsigned _BitInt(N)` provide bit-precise integer types. Their availability and limits depend on implementation support. They can be valuable for numerical kernels and hardware-oriented code, but ordinary public interfaces may prefer portable fixed-width types when widths are already covered by `<stdint.h>`.

9.5 Binary literals and digit separators

```
unsigned mask = 0b1010'0101;
```

Binary literals communicate bit patterns; the single-quote separators make long constants easier to read. A leading `0b` is now standardized rather than a vendor extension.

9.6 Attributes

```
[[nodiscard]] int save_record(void);
[[maybe_unused]] static int debug_setting = 0;
```

Standard attribute syntax provides optional diagnostics and intent. `[[nodiscard]]` encourages checking a result; the exact diagnostic is implementation-dependent. Attributes do not magically enforce a full error-handling policy. C23 also specifies `[[deprecated]]`, `[[fallthrough]]`, and `[[noreturn]]` as well as forms of `[[unsequenced]]` and `[[reproducible]]`.

9.7 New preprocessor capabilities

C23 adds `#embed` for incorporating binary data at translation time and `#warning` for diagnostics. `__VA_OPT__` permits more reliable conditional comma handling in variadic macros. Implementation support for `#embed`, in particular, requires explicit checking.

9.8 Checked arithmetic and bit utilities

C23 specifies `<stdckdint.h>` with `ckd_add`, `ckd_sub`, and `ckd_mul` for checked integer arithmetic, and `<stdbit.h>` for unsigned integer bit operations. These facilities may be missing in otherwise C23-capable runtime libraries. A project that depends on them should run configure-time feature tests. Do not assume a working `-std=c23` flag implies complete header support.

9.9 A migration policy

First build with maximum practical warnings. Remove implicit declarations and invalid conversions; make allocation-size checks explicit; audit dangerous signed arithmetic; identify any reliance on extensions. Introduce `nullptr`, boolean keywords, binary literals, designated/empty initializers, and attributes only when supported by the actual deployment toolchains. A robust migration preserves behavior before improving style.

Exercises. 9.1 Replace a platform-specific `__typeof__` declaration with standardized spelling where available. 9.2 Use `[[nodiscard]]` on a status-returning function and observe diagnostics. 9.3 Write a feature-test program for `<stdckdint.h>`. 9.4 Explain why `constexpr` is not simply another spelling for `const`.

9.10 Generic selection

C's `_Generic` chooses an expression based on a controlling expression's type. It supports type-directed interfaces while remaining a compile-time selection mechanism. Generic selection is not dynamic dispatch.

```
#define magnitude(x) _Generic((x), \
    float: fabsf, \
    double: fabs, \
    long double: fabsl \
)(x)
```

This example requires `<math.h>`. It illustrates a common pattern: select an appropriately typed function, then call it. The supported type list is intentionally explicit; passing an unsupported type is a constraint violation unless a `default` association is provided. `_Generic` predates C23, but remains important when designing a modern C interface.

9.11 Safer use of `typeof`

When an expression's type is cumbersome, `typeof` can keep an interface aligned with the expression itself. It does **not** turn C into a language with automatic lifetime management. A pointer inferred from another pointer has the same potential lifetime hazards.

Use inferred types when duplication creates maintenance risk, but write explicit types when they communicate important width, ownership, or protocol constraints.

9.12 Checked arithmetic in C23

When available, `<stdckdint.h>` supports checked integer calculations that report overflow through a boolean result rather than leaving the programmer to reconstruct each type's boundary conditions.

```
#include <stdckdint.h>
int result;
if (ckd_add(&result, a, b)) {
    /* the mathematical result is not representable in result's type */
} else {
    /* result contains the sum */
}
```

The above is a C23-library-dependent fragment, not a guarantee that every compiler installing a C23 front end also ships this header. For targets without it, a tested manual check or compiler-specific checked-arithmetic primitive may be appropriate.

9.13 Selected portability cautions

Not all useful C23 additions are equivalent from a portability standpoint. Binary literals and attributes may be available before other features. `#embed` support can vary independently from the core language. Bit-precise integers may have target-specific maximum widths. Headers such as `<stdbit.h>` may be absent from older C libraries.

Keep a portability table stating which implementation combinations were actually tested. The standard defines what a conforming implementation must do; deployment engineering determines which implementation is in hand.

9.14 Exercises

9.5 Build a `_Generic` wrapper that chooses the correct absolute-value function for three floating types. **9.6** Write a small compile probe for `nullptr`, `typeof`, and `constexpr`. **9.7** Compare manual overflow checks with `ckd_add` when available. **9.8** Explain why a compiler's version string alone cannot establish support for `<stdbit.h>`.

9.15 What actually changes when moving from C17

C23 is not merely a new spelling of old constructs. It removes traditional, unprototyped function declarations and old-style function definitions. In C23, `void work()` declares a function taking no arguments; older C dialects treated an empty parameter list differently outside a function definition. This is especially important in legacy headers.

C23 also fixes signed integer representation to two's complement. It does **not** define signed overflow. The two statements are compatible: an implementation has a required representation, while the result of evaluating an overflowing signed arithmetic expression remains undefined.

The macro `__STDC_VERSION__` is defined as `202311L` in C23 mode for conforming implementations. Checking this can select a language-level path, but it cannot certify that a particular runtime library or optional capability is present.

9.16 The boundary between auto and typeof

C23 introduces type inference for declarations with `auto` and standardizes `typeof` and `typeof_unqual`. These facilities eliminate some redundant type spellings. They do not provide templates, classes, overload resolution, or C++-style deduction rules.

A library's public signature should normally state the types important to its binary contract: `uint32_t`, `size_t`, or a named structure. Locally, inferred types can be useful where the type is unambiguous from the initializer and repeating it would be error-prone.

C23 `constexpr` applies to certain constant objects under specific restrictions; it is not general C++ constant evaluation. For example, C23 does not make an arbitrary user-written function eligible for execution during translation merely because it is called from a constant initializer. A `constexpr` object should not be used to promise behavior that the actual compiler has not implemented.

9.17 Modernized enumerations and attributes

C23 adds facilities for enumerations with explicitly selected underlying integer types. They can make intent clear when enum values appear in compact data structures or interfaces. Still, an enum object in a wire format must be serialized as an explicitly chosen integer representation; the in-memory layout should not be transmitted wholesale.

Attributes such as `[[nodiscard]]` help advertise contracts. They improve diagnostics but are not enforcement mechanisms: a caller may still discard a return value and a conforming implementation's diagnostic behavior depends on the attribute rules. Code review and tests remain necessary.

9.18 Checked arithmetic belongs at boundaries

When the implementation supplies `<stdckdint.h>`, `ckd_add`, `ckd_sub`, and `ckd_mul` can help test whether the mathematical result is representable in the result type. Use them when dealing with

byte counts, sizes and input-derived values. Do not confuse this check with validating the semantic limits of a protocol. A packet length may fit in a `size_t` while still exceeding the application's maximum frame length.

Some deployments provide a C23 compiler without the corresponding C23 library headers. The right adaptation is a feature check at configuration time, or a small, tested fallback module with exactly the same contract. A bare compiler-version comparison is less reliable than verifying the required operation.

9.19 C23 migration example

Suppose a legacy API relied on `malloc(count * width)`. Migrating to C23 is a chance to fix the arithmetic contract before introducing new syntax:

```
#include <stdint.h>
#include <stddef.h>
#include <stdlib.h>
#include <stdbool.h>

bool allocate_bytes(size_t count, size_t width, void **out)
{
    if (out == nullptr || count == 0 || width == 0 ||
        count > SIZE_MAX / width)
        return false;
    void *memory = malloc(count * width);
    if (memory == nullptr)
        return false;
    *out = memory;
    return true;
}
```

The routine refuses zero-sized requests and leaves the output unchanged on failure. It does not assert that all targets offer a usable heap. In freestanding firmware, a caller-provided arena may be preferable.

9.20 A testable feature inventory

Record support separately for language syntax, library headers, implementation-defined properties, and target environment. The first category includes `nullptr`, `typeof`, and `constexpr`. The second includes `<stdckdint.h>` and `<stdbit.h>`. The third includes integer widths and alignment. The fourth includes available heap, streams, threads, and atomics.

Further problems: Port an old K&R-style declaration to a C23 prototype. Test whether a compiler supports `#embed` independently from `-std=c23`. Show that a C23 `constexpr` object is not a user-defined compile-time function. Design a feature probe for a C23 standard-library header.

9.21 A concrete subset of C23

The following complete program deliberately selects language features that are supported by the book's intended GCC 14 and Clang 18 test matrix. It does not exercise the newer library headers or `#embed`, which must be checked independently.

```
#include <stdio.h>

struct point { int x, y; };

int main(void)
{
```

```
    auto flags = 0b1010'0101u;
    typeof(flags) another = 0b0000'0011u;
    struct point origin = {};
    int *missing = nullptr;
    bool good = (flags & another) == 1u;

    static_assert(sizeof(flags) == sizeof(unsigned));
    if (!good || origin.x != 0 || origin.y != 0 ||
        missing != nullptr)
        return 1;
    puts("C23 essentials: OK");
}
```

The integer literal is explicitly unsigned. The digit separator is a single quote between digits. The `auto` declaration infers the type from its initializer, and `typeof(flags)` names that type again. Empty initialization initializes both structure members to zero. `nullptr` provides a null pointer constant. The `static_assert` condition is evaluated at translation time.

The repository keeps a corresponding standalone program in `examples/c23_basics.c`. When teaching this listing, compare what the compiler knows at translation time with what the program checks at runtime.

9.22 Why not use every C23 feature in the first example?

A language revision consists of many individually specified changes. A compiler may support the new keywords but lack the new embedding directive. Its system C library may lack `<stdckdint.h>` or `<stdbit.h>`. A microcontroller toolchain may implement only a subset of hosted interfaces.

Choosing a well-supported subset for the introductory example makes the core lesson reproducible. Dedicated feature probes are the right place to demonstrate `constexpr`, bit-precise integers, the newer enumeration facilities, and the new library interfaces across toolchains.

9.23 Initialization is not byte filling

`struct point origin = {};` uses language-level initialization, not a raw `memset`. C's initialization rules set members to their appropriate empty-initialization values. This distinction matters for structures containing pointers or floating members, because an all-bits-zero representation is not universally promised to be the semantic null-pointer or floating zero representation.

For an ordinary aggregate, correct language initialization is usually both clearer and more portable than relying on representation bytes.

Further problems: Rewrite the example without `auto` and `typeof`. Identify which expressions are evaluated at translation time and which during execution. Build the same file under a C17 dialect and record the diagnostics instead of assuming all failures have the same cause.

9.24 A language revision is a collection of independent features

Using `-std=c23` selects a language mode, but the compiler and standard library are not always delivered or updated together. A cross-compiler may recognize `nullptr` and `typeof` while its libc lacks `<stdckdint.h>`. The correct deployment procedure tests the exact construct or library interface that the project uses.

Classify features before adopting them. Some are purely syntactic improvements, such as binary literals and more convenient forms of initialization. Some change type expressiveness, such as bit-precise integers. Others require new translation support, such as `#embed`. Still others require implementation of additional library headers.

That taxonomy prevents a migration effort from becoming a blind replacement of every old idiom with a new-looking construct.

9.25 C23 integer literals and readable masks

Binary literals represent bit patterns compactly:

```
unsigned control = 0b1010'0001u;
```

The single quotes separate digit groups without affecting value. This is useful when a device control register is documented by bit positions. It does not solve the hardware semantics: whether a register is read-only, write-one-to-clear, or requires reserved bits depends on the device manual.

Clang 18 in some configurations diagnoses the standardized C23 binary-literal syntax with a GNU-extension warning. Our CI retains `-Werror` but suppresses that one diagnostic for Clang 18. A warning emitted by a compiler version is not necessarily evidence that the source violates the language standard.

9.26 nullptr is a value, not an owner

C23's `nullptr` is a null pointer constant with type `nullptr_t`. It makes pointer intent clearer than writing integer zero in a pointer context. But replacing `NULL` with `nullptr` does nothing to prevent a non-null dangling pointer, a wrongly sized array, or a double free.

The following check rejects absence, not invalid lifetime:

```
if (buffer == nullptr)
    return false;
```

A buffer that was freed but whose old pointer has not been reset may still compare non-null. The API must prevent access after the owner's release, not hope that an address test will discover it.

9.27 Type inference should not hide the contract

C23's `auto` can infer a local type from its initializer. This can avoid spelling a complex expression type twice, and `typeof` can express a related declaration. However, the public interface of a protocol decoder should not hide that its input is `const unsigned char *` with a `size_t` length or that a field is a fixed-width integer.

At public boundaries, explicit types communicate ABI and range requirements. Inside a function, type inference is helpful when the initializer makes the meaning obvious. Use it as a maintenance aid, not a substitute for reviewing conversions.

9.28 Checked integer arithmetic in a C23 library

The `ckd_add`, `ckd_sub`, and `ckd_mul` interfaces from `<stdckdint.h>` express checked operations with a result pointer. They are especially useful for sizes and accumulated counts.

A feature probe should include the header, call a chosen macro with values that fit and values that overflow, and link the resulting program using the actual target C library. If the header is missing, a separately tested implementation of equivalent arithmetic checks may be reasonable. A compiler version alone is insufficient evidence.

The application may need a limit stricter than the type's representable range. A frame size of 65,000 bytes can be representable in `size_t` yet impossible to buffer in a 2 KB MCU arena. Checked arithmetic and application resource checks solve different problems.

9.29 Choosing what to modernize

A conservative migration can proceed in three phases. First, make the old program correct under its current contracts: fix implicit assumptions, mismatched prototypes, unchecked sizes and invalid conversions. Next, select the C23 language mode and confirm equivalent behavior under tests. Finally, adopt newer features where they improve readability or reliability.

Do not rewrite a working fixed-size state machine solely to exhibit `_BitInt` or `typeof`. A textbook should show what the feature contributes and what its portability cost is. For firmware, support across required cross-toolchains is often more important than showcasing every new keyword.

Guided exercises

9.9 Create a compilation probe for `constexpr` and compare its syntax with `const`. **9.10** Implement a feature check for `<stdckdint.h>` without assuming its existence. **9.11** Explain what `nullptr` does not protect against. **9.12** Convert a simple C17 program to C23 mode while preserving the behavior of every input test. **9.13** Document the actual warning produced by two compilers for a binary literal and distinguish a diagnostic from a normative rule.

10. Reliable Systems Programming

The language gives precise rules for certain operations and deliberately leaves other matters to implementations or gives no meaning to invalid programs. Reliability begins with knowing the boundary.

10.1 Undefined, unspecified, implementation-defined

An **undefined** operation has no required behavior: indexing past an array or signed overflow are examples. **Implementation-defined** behavior obliges an implementation to document a choice, such as properties of `char`. **Unspecified** behavior allows a choice among permitted outcomes without requiring that choice to be documented.

Never make the program's correctness depend on one observed result of undefined behavior, even if a particular build seems stable.

10.2 Check sizes before multiplying

```
#include <stdint.h>
#include <stdlib.h>
#include <stddef.h>

bool allocate_samples(size_t count, int **out)
{
    if (out == nullptr) return false;
    if (count == 0 || count > SIZE_MAX / sizeof(int)) return false;
    int *p = malloc(count * sizeof *p);
    if (p == nullptr) return false;
    *out = p;
    return true;
}
```

The caller must release `*out` after success. The function leaves `*out` unchanged on failure. For C23, `bool` is a keyword. For maximum compatibility with older dialects, include `<stdbool.h>`.

10.3 The limits of volatile

`volatile` is useful for accesses with externally observable behavior, including certain memory-mapped I/O, when used according to implementation and hardware rules. It is not a synchronization primitive and does not prevent data races. Atomics from `<stdatomic.h>` provide operations with explicit memory-order semantics, when supported by the target's implementation.

10.4 Host tests and device tests

Test numerical functions on a host where instrumentation is available. Enable sanitizers such as AddressSanitizer and UndefinedBehaviorSanitizer where toolchains support them. Unit tests do not replace target validation for memory maps, interrupts, scheduling latency, peripheral behavior, or floating-point differences.

10.5 Protocol parsing

A parser accepting a length and a byte buffer is easier to audit than one that searches indefinitely for a terminator. Distinguish malformed input, insufficient data, overflow, and unsupported versions. Parse into temporary state and publish a complete result only after validation.

10.6 Review checklist

Ask of each interface: Are sizes explicit? Are output objects initialized on success only? Is ownership documented? Are integer conversions checked? Are assumptions about endianness and alignment written down? Can cleanup happen on every error path? Is concurrent access synchronized?

Exercises. 10.1 Enumerate five undefined-behavior risks in a string decoder. 10.2 Design a bounded ring buffer and define its full and empty states. 10.3 Explain why `volatile int flag` is not generally a safe cross-thread signal.

10.7 Establish invariants before optimizing

An invariant is a property required to remain true across operations. A ring buffer may maintain `count <= capacity`; a vector may maintain `length <= capacity`; a parser may maintain `position <= input_length`. The easiest time to discover a broken invariant is at the operation that first violates it.

Checks should precede the risky operation. Checking a pointer after dereferencing it, or checking a sum after signed overflow, is too late. An optimizer may assume undefined behavior never occurs and transform subsequent code accordingly.

10.8 Bound both time and storage

An embedded function should be evaluated not only for logical correctness but also for its worst-case time and memory needs. A loop over a fixed-size buffer has an explicit bound. A dynamically grown table or a scan to an unbounded terminator may not.

Real-time behavior adds scheduling, interrupt latency, contention, and priority inversion. None can be solved merely by choosing C23 syntax. The language supports expressing the operations; the system architecture determines when they can run.

10.9 Concurrency and atomics

Sharing a non-atomic object between threads without appropriate synchronization can create a data race, resulting in undefined behavior. `volatile` does not repair this. Atomics in

<stdatomic.h> provide carefully specified atomic operations; mutexes and other primitives may be preferable where a critical section covers several related objects.

Memory-order choices require a proof about which values must be visible to which threads. Start with sequentially consistent operations unless the design has a demonstrated reason to use weaker ordering. Interrupt-to-thread communication also depends on the implementation's interrupt model and target hardware.

10.10 Errors are part of the API

A function's result contract should specify what happens when it receives a null pointer, an invalid length, an unrecognized packet, insufficient output storage, or a resource failure. The failure behavior may need to preserve caller state. Document that guarantee and test it.

The absence of a crash does not prove memory safety. A parser can mis-handle a length while still writing inside allocated storage, and a numerical routine can return a plausible but incorrect value after an invalid conversion.

10.11 Exercises

10.4 Write invariants for a fixed-capacity queue and construct tests preserving them. **10.5** Make a parser reject overflow before computing `offset + length`. **10.6** Describe what synchronization an interrupt-driven producer and thread consumer would require. **10.7** Identify whether each of four sample failures is undefined, unspecified, or implementation-defined behavior.

10.12 A bounded packet decoder

A practical parser should never trust an unvalidated input length. The companion program `examples/packet_decode.c` reads a miniature binary packet with magic byte `0xA5`, version byte `1`, a two-byte little-endian payload length, at most 64 payload bytes, and a one-byte XOR checksum. This deliberately simple checksum demonstrates framing but does **not** provide cryptographic integrity or meaningful protection against intentional tampering.

The parser first checks the minimum header and checksum size. Next it validates the magic and version. It constructs the declared payload length from bytes, without casting the input buffer to a structure. Before accessing the payload, it verifies the declared length against both the fixed maximum and the available input bytes. Only after validating the complete packet and checksum does it write the decoded result into the caller's object.

One subtle design choice is to build a local candidate object and assign it to the caller only on success. This gives failure an important contract: the output remains unchanged. The implementation uses the subtraction `size - HEADER_SIZE - 1` only after proving the total size is at least `HEADER_SIZE + 1`, so the subtraction cannot underflow.

The program also exercises an invalid checksum and truncated input. It was compiled and executed locally using GCC 14 in C2x dialect, with `-Wall -Wextra -Wpedantic -Werror` and AddressSanitizer and UndefinedBehaviorSanitizer enabled. This verifies that executable under the tested configuration, not every possible input or toolchain.

Exercises. 10.8 Add a failure reason enum without changing the output-on-failure guarantee. 10.9 Generate exhaustive byte-sequence inputs for packets of length zero through four. 10.10 Replace the XOR checksum with a documented CRC and explain why even a CRC is not a cryptographic authentication code.

10.13 Preconditions, invariants, and postconditions

A robust C function can be described by three kinds of statements. Preconditions specify what the caller must supply; invariants describe properties that remain true during execution; postcon-

ditions describe what success and failure mean.

For a byte decoder, a precondition may be that any non-null buffer pointer refers to at least the supplied length of readable storage. An invariant may be that the parser index never exceeds the input length. The successful postcondition is a fully decoded object; the failure postcondition is that the output object remains unchanged.

These distinctions matter because not every invalid pointer can be detected with a null check. A caller that lies about an object's bounds violates the contract. The function can still protect itself against internally computed out-of-range indexes by validating lengths before access.

10.14 A two-stage parser

An effective pattern is *validate, then commit*. Use temporary state to decode a candidate; inspect the entire record; only then write the result.

```
struct header candidate = {};
/* Parse candidate from a counted byte buffer. */
if (!valid_magic || !valid_length || !valid_checksum)
    return false;
*out = candidate;
return true;
```

This is a fragment, not a standalone example. The important point is that errors do not leave a partially updated public result. Chapter 10's packet example uses a bounded payload and this publication policy.

10.15 Safety is not security

Memory safety protects against invalid storage access. A parser can be memory-safe yet accept maliciously large, inconsistent, or semantically forbidden requests. A one-byte XOR checksum may detect some accidental errors but is not authentication; an attacker can usually alter data and restore a valid XOR.

For an external protocol, consider resource exhaustion, replay, malformed framing, version downgrades, and authorization separately from buffer bounds. The task determines whether cryptographic integrity, rate limits, and stronger protocol validation are required.

10.16 Bounded execution requires a system model

Worst-case running time is more than an algorithmic big-O expression. In a real-time task, include interrupt preemption, cache behavior, memory contention, blocking calls, and other scheduling delays. A loop of exactly 128 iterations is bounded in iteration count; it is not automatically guaranteed to meet a microsecond deadline under every scheduler and cache state.

Avoid heap allocation and unbounded blocking on a critical path unless their timing has been proven acceptable. For many embedded applications, fixed-size data structures, preallocation, and explicit timeout behavior simplify the argument.

10.17 Reviews use independent evidence

Different validation methods find different errors. Compiler warnings detect suspicious constructs. Static analysis explores possible paths. Sanitizers catch classes of runtime faults when the relevant paths execute. Property tests explore behavioral invariants. Hardware-in-the-loop tests reveal device-specific assumptions.

No single passing test suite proves correctness, particularly for concurrent programs. For a critical algorithm, pair executable tests with a short written proof sketch: define ranges, explain why every memory access is within them, and explain why every loop terminates.

10.18 A fault-injection exercise

Build a streaming decoder with an injected byte source that can return a short read, an I/O error, or clean EOF. Give each outcome a distinct status and assert that the caller never sees half-decoded output. Repeat with a deliberately full output buffer and with a record whose claimed payload length is larger than available storage.

Further problems: Construct a threat model for the toy packet protocol. Explain how a full buffer and an overflowed integer differ. Write a reviewer checklist for atomic queue shutdown and object lifetime.

10.19 Defensive decoding: from bytes to a validated object

A packet decoder is a good place to practice several C contracts at once. A caller supplies a byte buffer and its length. The decoder must establish a valid format before touching output state.

For a toy protocol, define the frame as a one-byte magic number, one-byte version, two-byte little-endian payload length, payload bytes, and a checksum octet. The decoder first confirms that at least the fixed header and checksum are present. Only then does it read the header fields. Next it verifies the payload length against both its application-defined maximum and the remaining input capacity. Finally, it computes the checksum and copies the completed candidate into the caller's output.

The order matters. If code reads `bytes[3]` before checking `length >= 4`, a later bounds check cannot undo the invalid read. If it calculates `payload_length + 5` without checking representability, a large unsigned value could wrap. If it writes the public result field-by-field and later discovers an invalid checksum, the caller sees partially updated state.

A strong decoder therefore has a simple publication point: all validation succeeds, and only then is `*out` assigned.

10.20 Define three failure classes

Not all failures have the same meaning. A useful API might distinguish malformed input, incomplete input, and exhausted output capacity. Malformed input violates the format even if more bytes were supplied. Incomplete input may be a valid prefix that needs more data. Insufficient output capacity means the input might be valid, but the caller did not provide enough destination storage.

An RTOS message receiver can react differently: wait for the rest of an incomplete frame, discard malformed input and resynchronize, or apply backpressure when output storage is full. Collapsing all three into a boolean failure can make the transport logic awkward.

The simplest teaching program can return `bool`, but a production interface should choose the status representation based on its callers.

10.21 Error recovery is a protocol decision

After rejecting a malformed frame, should the receiver scan for the next magic byte? Doing so may recover quickly, but payload bytes could themselves contain that marker. A more robust framing design might escape reserved bytes, use a fixed header with a validated length and checksum, or rely on an out-of-band boundary.

The algorithm's *resynchronization* rule belongs in the protocol specification. Otherwise two devices may interpret the same corrupt stream differently despite both having memory-safe parsers.

10.22 Worst-case resource accounting

To evaluate firmware use, compute worst-case storage and work. A parser with a maximum 64-byte payload, a four-byte header, and a checksum can bound its input record storage. The actual C output structure may be larger due to padding, so assess `sizeof` on the target when reserving a pool of output records.

The per-record checksum loop has at most a fixed number of iterations. That proves an iteration bound, not a deadline: execution time also depends on scheduling, memory placement, compiler settings, and hardware. A task that blocks on a mutex or output stream may take longer than its local loop suggests.

Exercises

10.11 Implement a status enumeration with valid, incomplete, malformed and no-capacity cases. **10.12** Design a resynchronization policy when a payload contains the magic byte. **10.13** Calculate an upper bound on parser storage independent of input stream length. **10.14** Test that all error paths preserve the output structure's meaningful members.

11. An Extended Example: A Word Index

A word counter is a small program that demonstrates streaming input, fixed-capacity state, tokenization, searching, and error reporting. We deliberately avoid dynamic allocation so the memory bound is explicit.

11.1 Requirements

Read standard input one byte at a time. Treat ASCII letters and digits as part of words, fold uppercase ASCII to lowercase, store up to 1024 distinct words of at most 31 bytes, and print each word with its occurrence count. Reject excess distinct words rather than corrupting the table.

This is an ASCII-oriented utility, not a general Unicode word segmenter. Its fixed-size table and linear search make worst-case costs visible.

11.2 Implementation

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include <stddef.h>

enum { MAX_WORDS = 1024, MAX_LEN = 31 };

struct entry {
    char word[MAX_LEN + 1];
    unsigned long count;
};

static struct entry entries[MAX_WORDS];
static size_t used;

static bool letter_or_digit(int c)
{
    return (c >= 'a' && c <= 'z') ||
           (c >= 'A' && c <= 'Z') ||
           (c >= '0' && c <= '9');
```

```
}

static char lower_ascii(int c)
{
    if (c >= 'A' && c <= 'Z') c += 'a' - 'A';
    return (char)c;
}

static bool add_word(const char *word)
{
    for (size_t i = 0; i < used; ++i) {
        if (strcmp(entries[i].word, word) == 0) {
            if (entries[i].count == ~0UL) return false;
            ++entries[i].count;
            return true;
        }
    }
    if (used == MAX_WORDS) return false;
    strcpy(entries[used].word, word); /* bounded by tokenizer */
    entries[used].count = 1;
    ++used;
    return true;
}

int main(void)
{
    char word[MAX_LEN + 1];
    size_t len = 0;
    int ch;
    bool too_long = false;
    while ((ch = getchar()) != EOF) {
        if (letter_or_digit(ch)) {
            if (len < MAX_LEN)
                word[len++] = lower_ascii(ch);
            else
                too_long = true;
        } else if (len != 0 || too_long) {
            if (too_long) {
                fputs("word exceeds limit\n", stderr);
                return 1;
            }
            word[len] = '\0';
            if (!add_word(word)) {
                fputs("word table full or counter overflow\n", stderr);
                return 1;
            }
            len = 0;
        }
    }
    if (ferror(stdin)) {
        fputs("input error\n", stderr);
        return 1;
    }
    if (too_long) {
        fputs("word exceeds limit\n", stderr);
        return 1;
```

```
    }
    if (len != 0) {
        word[len] = '\0';
        if (!add_word(word)) return 1;
    }
    for (size_t i = 0; i < used; ++i)
        printf("%-31s %lu\n", entries[i].word, entries[i].count);
    return ferror(stdout) ? 1 : 0;
}
```

The total table storage is fixed. The program checks every insertion, rejects too-long words, and refuses counter wraparound. The final output is in order of first appearance, not lexicographic order.

11.3 Complexity and extensions

Each word may be compared against up to 1024 existing entries, so the lookup time is linear in the number of distinct words. A hash table can improve average lookup, but then needs collision handling, a load-factor policy, and carefully bounded worst-case behavior. Sorting entries before printing provides alphabetical output without changing the tokenizer.

Exercises. 11.1 Sort the final table alphabetically. 11.2 Replace linear lookup with open addressing. 11.3 Count overflowed word lengths rather than aborting. 11.4 Implement Unicode-aware tokenization using a deliberately selected library, documenting its normalization rules.

11.4 From linear search to a sorted index

The first implementation searches entries in order of first appearance. If there are *u* distinct words, one lookup may require *u* comparisons. We can instead keep the table sorted and use binary search to locate an existing word or the place to insert a new one.

For a sorted array with bounds `lo` and `hi`, the active range is half-open: `[lo, hi)`. At each iteration, compute `mid = lo + (hi - lo) / 2`; this avoids the potential overflow of `(lo + hi) / 2`. Compare the word at `mid` with the query. If it sorts earlier, the search continues after `mid`; otherwise it continues at or before `mid`. The range narrows on every iteration.

When `lo == hi`, the search ends. The same position is the insertion point for a new word. Shifting entries one position to the right preserves sort order. Lookup costs logarithmic comparisons, but insertion still takes linear time in the number of entries. This is a useful reminder that improving one operation does not automatically improve the entire algorithm.

11.5 A table with explicit invariants

The sorted-array representation has three important invariants: (1) `used` never exceeds `MAX_WORDS`, (2) entries in `[0, used)` are sorted lexicographically, and (3) every stored word contains at most `MAX_LEN` ASCII bytes followed by a zero terminator. Before incrementing a count, test against `ULONG_MAX`. Before inserting, confirm unused capacity. These checks protect both memory bounds and numerical meaning.

The companion complete program, `examples/word_index.c`, demonstrates sorted indexing, ASCII normalization, fixed storage, and failure reporting. It deliberately treats words exceeding the maximum length as an error instead of silently truncating them into potentially ambiguous keys.

11.6 Complexity, measurement, and design

For a text with *n* input bytes and *u* unique words, lexical scanning is linear in *n*. Binary lookup is *O(log u)* string comparisons per completed word; moving records for insertion is *O(u)* in the

worst case. Since each string is at most 31 bytes, comparison work has a fixed bound. The static table capacity also provides a fixed memory bound.

A hash table could give expected near-constant lookup, with tradeoffs for collision policy, load factor, hashing cost, and predictability. A balanced tree could avoid moving array entries but would require more bookkeeping. Neither is categorically superior: requirements determine the data structure.

11.7 Additional exercises

11.5 Prove that the binary-search interval strictly shrinks on every iteration. **11.6** Add a configurable maximum word count, without introducing dynamic allocation. **11.7** Implement a frequency-first ordering at print time, preserving alphabetical order on ties. **11.8** Test the empty input, a final word without newline, repeated separators, a full table, and an overlong word. **11.9** Benchmark insertion workloads with vocabulary ordered forward, reverse, and randomly.

11.8 Designing the input grammar

The index accepts ASCII letters and decimal digits and treats everything else as a separator. That simple grammar has two important consequences: punctuation splits words, and any byte outside the specified ASCII set is not recognized as part of a word. A UTF-8 text file may therefore produce surprising tokens unless the program is explicitly extended to decode Unicode.

The rule for overlong words is equally important. Truncating a word after 31 bytes is not merely losing display information; two different long words can collapse into the same key. Rejecting overlong tokens prevents an incorrect count from looking authoritative.

The program also chooses to count only complete words and to accept a final word that has no trailing separator. That behavior is tested separately because streaming programs often get end-of-file boundaries wrong.

11.9 Proving the sorted insertion

The maintained invariant is that `entries[0]` through `entries[used-1]` are sorted and distinct. Binary search finds the first position whose word is not less than the input. There are two cases:

- If an entry at that position compares equal, increment its count after checking for overflow.
- Otherwise, if capacity permits, move the suffix one element right, copy the new word into the gap, set its count to one, and increment `used`.

The move proceeds from the right edge toward the insertion point. Moving in the opposite direction with ordinary assignments could overwrite entries before they were copied. The special bound `used < MAX_WORDS` ensures one free slot exists before any shift.

The search interval `[lo, hi)` shrinks because each iteration either increases `lo` beyond the midpoint or lowers `hi` to the midpoint. Since its endpoints are integers in a finite range, the loop terminates. The midpoint calculation `lo + (hi-lo)/2` avoids adding two potentially large bounds.

11.10 Complexity with meaningful constants

Let `w` be the number of word occurrences, `u` the number of distinct words, and `L` the maximum key length. Tokenization scans input bytes once. A binary search performs at most logarithmically many comparisons in `u`, and each comparison can inspect up to `L` bytes. A new entry may require copying `O(u)` records. Thus workloads dominated by repeated words may be substantially cheaper than workloads dominated by inserting new distinct words.

For a fixed table of 1024 entries and 31-byte keys, the resource ceiling is explicit. That bounded design may matter more in firmware than the asymptotic advantage of a dynamic hash table. A hash implementation is a useful improvement exercise, not a universally better choice.

11.11 Test cases that reveal design errors

Use independent expected outputs for: no input; one word; a final word without newline; repeated separators; uppercase and lowercase variants; words exactly at the maximum length; a word one byte too long; the maximum distinct-word count; and an attempted extra distinct word.

A property test can also compare the sum of all printed counts with the number of successfully recognized word occurrences. This is not a complete proof, since two errors could cancel, but it detects many accidental drops and duplicates.

11.12 Extending the implementation responsibly

An open-addressed hash table needs a defined hash function, collision resolution, load limit, and behavior when no free slot exists. Returning failure is better than silently overwriting an unrelated entry. If the application must preserve sorted output, sort the final records or maintain a separate ordering; hashing by itself does not provide alphabetical order.

A Unicode-aware version is a different product. It must define normalization and word-boundary rules, decide how case folding works, and bound its memory for multibyte text. Simply applying `tolower` to each UTF-8 byte is not enough.

Further problems: Provide a proof that the insertion loop cannot access index `MAX_WORDS`. Write a test generating 1024 distinct terms. Compare sorted-array and hash-table lookup under high repetition. Design a stable frequency ordering for ties.

11.14 A search algorithm with a proof obligation

Binary search is brief enough to fit on a few lines, but it should be understood through its invariant. At the start of each iteration, the insertion point lies in the half-open interval `[lo, hi)`. Every entry strictly before `lo` compares less than the requested word. Every entry at or after `hi` compares greater than or equal to it.

Set `mid = lo + (hi - lo) / 2`. If the middle entry compares less than the request, the insertion position must follow `mid`, so set `lo = mid + 1`. Otherwise, it is at or before `mid`, so set `hi = mid`. When `lo == hi`, exactly one insertion position remains.

The implementation can then determine whether the requested word already exists at that position. If so, it increments a counter only after verifying that increment is representable. If not, it checks table capacity before shifting existing records.

The proof is useful when changing the data structure. One off-by-one mistake can produce a program that appears correct for a small vocabulary but corrupts storage when inserting at the beginning or end.

11.15 The cost of sorted insertion

Binary search reduces the number of comparisons needed to locate a word, but maintaining a sorted array costs more when inserting new records. If there are already `u` distinct words and the new one belongs at the front, the program shifts `u` entries. The worst-case cost of inserting `u` distinct keys is therefore quadratic in `u`, even though each lookup performs only logarithmically many comparisons.

That is not automatically unacceptable. With a hard limit of 1,024 short keys, the worst-case memory usage is small and predictable. A microcontroller may value a fixed allocation and simple failure policy more than average-case insertion speed. For a large text corpus with millions of terms, a hash table or tree is likely more appropriate.

Measure the actual workload: repeated tokens benefit from quick lookup and do not shift records, while a sorted stream of unique tokens repeatedly inserts near one end. Big-O notation tells us

the growth trend, but a real resource bound must still account for maximum keys and fixed string lengths.

11.16 Error handling belongs inside the index

Rejecting a long token rather than truncating it prevents two different words from becoming the same key. Rejecting an eleventh-hundredth distinct key rather than writing past the table prevents memory corruption. Rejecting counter overflow prevents the program from silently reporting a smaller count than the truth.

After any rejection, decide whether the program should stop, skip the offending token and continue, or expose a detailed status. The teaching implementation stops at the first error. A long-running service might recover, but only with a documented strategy that avoids partially committing a word.

The internal operation `add_word` returns success or failure; a larger program could use an error enumeration to distinguish table-full from count-overflow. That would improve diagnostics without changing the sorted-array algorithm.

11.17 Independently test the expected output

A decoder and encoder written with the same wrong byte order can agree with each other. Likewise, a word index and an expected-result generator copied from the same algorithm can share the same defect. For a handful of text samples, write the expected alphabetical output manually.

Then add generated larger cases with invariants: total counts must equal accepted word occurrences, entries must be strictly sorted and unique, and no count may be zero. For a stable random test, record the seed so failures can be reproduced.

Exercises

11.10 Trace the search interval for inserting a word at the beginning, middle, and end of a five-element table. **11.11** Estimate the maximum number of shifts for inserting 1,024 distinct words in reverse order. **11.12** Design an error enumeration that distinguishes a long word, full table, input failure, and counter overflow. **11.13** Write an independent expected output for a file containing repeated words with differing capitalization.

12. Exercises and Further Reading

The best way to learn C is to predict a program's behavior before compiling it, then explain any discrepancy. Exercises should include ordinary cases, boundaries, and failure cases.

12.1 A progression of projects

Project A — Bounded line reader. Implement a function that fills a caller-supplied buffer, reports bytes stored, distinguishes newline from EOF, and signals truncation. Write tests for zero-capacity buffers, one-character buffers, long lines, embedded zero bytes, and input errors.

Project B — Integer parser. Parse unsigned decimal text into `unsigned` without `atoi`. Reject empty input and nondigits; detect overflow before multiplying by ten or adding the next digit. Show why checking after an arithmetic overflow is too late.

Project C — A fixed pool. Implement allocation and release of equal-sized records from a fixed array. Reject invalid and repeated releases. Describe whether the pool can be used concurrently, and if so how it is synchronized.

Project D — Binary message reader. Decode a header with a version byte, 16-bit little-endian payload length, and checksum. Check all bounds before decoding. Test malformed lengths and truncated buffers.

Project E — Word index. Improve Chapter 11 with a hash table, sorting, and a test driver. Benchmark it using increasing vocabulary sizes and measure memory usage.

12.2 How to study source code

Read short functions and annotate their contracts. What objects must exist at entry? What inputs are valid? Which values may overflow? What state changes after success? What state is preserved after failure? Which implementation-defined assumptions are present?

Compiler warnings are evidence to inspect, not text to suppress reflexively. Sanitizers find bugs only on executed paths; code review and carefully constructed tests remain essential.

12.3 References

- ISO/IEC 9899:2024, *Programming languages — C* (normative specification).
- cppreference, C language reference, especially its C23 features and library pages.
- GCC C standards implementation status and command-line dialect options.
- Clang C language status and diagnostics documentation.
- The ISO C Secure Coding Rules and CERT C guidance, for additional defensive-programming perspectives.

12.4 A note on editions

This is an independent original textbook. Classic C teaching remains valuable for its economy of examples, but 2026 readers also need accurate discussion of defined behavior, lifetime, ownership, toolchain variation, safer interfaces, and C23's additions. Future manuscript revisions should add tested solutions, further real-world worked examples, portable build checks, and an index before a complete book-length edition is declared.

12.1 Exercises as specifications

An exercise is complete only when it states what the program accepts, what it returns, and how failure is reported. Consider a decimal parser: what does it do with an empty string, leading spaces, a plus sign, invalid characters, and a number larger than the destination type? The interesting part of the problem is often deciding these details before coding.

12.5 Testing boundaries

For a bounded collection with capacity N, test insertion at sizes zero, one, N−1, and N, then one additional insertion that must fail without damaging existing records. For signed arithmetic, test the largest and smallest representable values, opposite signs, and zero. For a byte parser, test all shorter-than-required inputs.

12.6 Integration project: a streaming index

Read Chapter 11's sorted word index and complete the following experiments:

1. Validate a file containing zero words and a file ending with an unterminated word.
2. Check that capitalization is folded only for ASCII letters and that digits remain in the word.
3. Generate 1,024 unique words and verify the table fills without overrun.
4. Attempt one more unique word and confirm the operation fails.

5. Compare counts against an independent reference implementation for randomly generated ASCII input.
6. Measure runtime for sorted and reverse-sorted unique vocabularies to investigate insertion cost.
7. Replace the sorted array with a fixed-capacity open-addressed hash table; state the collision policy, load factor, and failure semantics.
8. Describe how a UTF-8 version would need a more sophisticated definition of words, code points, and normalization.

12.7 Integration project: a bounded vector

Start from `examples/vector.c`. Add `reserve`, `clear`, and `get` operations with contracts. Make failed growth preserve the old allocation and data. Test failed allocation by injecting an allocator that rejects selected requests. Discuss why assigning one vector object to another is not a deep copy.

12.8 Reading C with a reviewer

For every function, identify which variables represent length and capacity, which arithmetic might overflow, which objects are borrowed versus owned, whether outputs change on failure, which operations may allocate, and whether concurrent calls can safely share state. A program may be short and still require serious reasoning.

12.9 Challenge: a portable protocol reader

Design a packet decoder accepting `const unsigned char *bytes` and `size_t length`. A packet contains a magic byte, a version, a two-byte little-endian payload length, payload bytes, and an eight-bit checksum. Reject incomplete packets before reading fields; reject lengths inconsistent with the buffer; do not write the output structure until the checksum passes. Provide cases for exact minimum length, maximum supported payload, bad version, invalid checksum, and length overflow.

12.10 Challenge: concurrency

Design a producer-consumer queue for one interrupt-side producer and one thread-side consumer. State why Chapter 5's plain queue is not automatically safe under concurrent access. Choose either an RTOS synchronization primitive with appropriate interrupt semantics or a target-validated atomic design. Document memory-order assumptions and why the chosen operations are legal in both execution contexts.

12.11 Further reading in practice

After solving a problem, read the relevant normative language clause or trusted reference and compare it with the implementation's documentation. A standards-conforming program is not merely one that a compiler accepts; conformance involves the behavior required by the language and library specification.

12.12 How to write convincing solutions

A solution should include more than a working program. State assumptions, list invariants, justify any bounds arithmetic, and show representative tests. A design that accepts invalid input only because one particular compiler tolerated it is not a complete solution.

A useful submission format has five short parts: (1) contract, (2) algorithm, (3) implementation, (4) test table, and (5) limitations. In a small exercise this may fit on one page. In a hardware-related problem, also identify dependencies on the toolchain and target.

12.13 Progressive memory exercises

Start with a fixed array of integers. Add a function returning its count of nonzero elements. Next write a version accepting a pointer and a length. Then add dynamic allocation with a checked size calculation. Finally add a failure-injecting allocator and verify that unsuccessful growth preserves existing state.

These stages teach the same abstraction with increasing responsibility. They also show how a simple algorithm can remain unchanged when its storage policy changes.

12.14 Progressive text exercises

First write an ASCII decimal parser that rejects every non-digit. Next permit leading spaces while still requiring at least one digit. Then add an optional sign and specify the behavior for signed minimum and maximum values. Finally switch from a terminating string to a counted byte span.

For each stage, list the accepted grammar explicitly. Tests should include zero, negative values when allowed, empty input, trailing junk, maximum representable values, and one value beyond the chosen range.

12.15 A bounded data pipeline

Combine three earlier projects into an application: read a binary message, validate its header and checksum, then append a decoded value to a fixed-size queue. Your contract must explain what happens when the queue is full. Does the program reject the message, drop the oldest value, or report backpressure? Each is a meaningful policy choice.

The test should inject short packets, malformed lengths, checksum failures, and queue saturation, and verify that rejected packets do not alter queue contents.

12.16 A portability investigation

Compile a set of probes that report `CHAR_BIT`, `sizeof(int)`, `sizeof(void *)`, the availability of `uint32_t`, and support for selected C23 features. Do not treat the output of one workstation as universal. Repeat on a second compiler, a different architecture, or a freestanding target where possible.

Compare the results with the preconditions stated in the book's byte-oriented and dynamic-memory examples. This exercise joins language rules to engineering practice.

12.17 A review rubric

Assess work in four dimensions, not one. **Correctness**: do the algorithms meet their contracts? **Safety**: are arithmetic and storage bounds checked? **Clarity**: can another engineer understand the code and its assumptions? **Verification**: do the tests challenge boundaries rather than merely demonstrating one successful input?

A concise, correct implementation with explicit limits is preferable to a larger one that hides failures in complicated abstractions.

Further problems: Review a peer's parser without running it, then compare your findings with compiler and sanitizer output. Build a small executable that verifies a public header compiles independently. Explain which claims in a test report are observed and which remain assumptions.

13. Testing, Diagnostics, and Portability

A C program can compile successfully and still be wrong. Compiler diagnostics report only what a tool can establish from the program and its selected options. Tests sample behavior for chosen inputs. Static analysis may identify suspicious paths. Sanitizers instrument executable operations. None of these alone proves that a C program is correct, but used together they provide an effective development discipline.

This chapter develops a concrete workflow for the small programs in this book.

13.1 Start with the contract

Before building a test, state what success means. Suppose a function computes the sum of a sequence of unsigned integers. Its contract might say:

- The input is a pointer to at least `count` elements, or a null pointer if `count` is zero.
- The result pointer must address writable storage.
- On success, the output is the exact mathematical sum.
- On failure, the output is unchanged.
- Overflow is an error rather than a request for modular arithmetic.

These statements determine useful tests. They also identify cases that a test **cannot** legitimately exercise: passing a dangling pointer or claiming a buffer has more elements than it actually has violates the caller's contract. Running such a case and observing a crash does not constitute a useful portable API result.

Tests should therefore separate *valid edge cases* from intentional demonstrations of undefined behavior.

13.2 Organize tests by dimensions

For a bounded operation, look for boundaries in several dimensions:

Dimension	Representative tests
Count	zero, one, capacity minus one, capacity, capacity plus one
Value	zero, ordinary value, largest representable value
Text	empty, valid, invalid digit, too long, no trailing newline
Storage	sufficient capacity, insufficient capacity, allocation failure
State	empty collection, full collection, wraparound, repeated use
I/O	clean EOF, short record, read error, write error

A useful test does not merely show that a function returns the expected value. It checks all promised postconditions. If failure must preserve an output, initialize it to a recognizable value before invoking the function, then verify it remains unchanged.

13.3 Assertions for tests, not input validation

The `assert` macro in `<assert.h>` checks a condition when assertions are enabled. Failed assertions terminate the program with a diagnostic. Defining `NDEBUG` can disable them.

```
#include <assert.h>
#include <stddef.h>

static int first(const int *values, size_t count)
{
    assert(values != nullptr && count != 0);
```

```
    return values[0];
}
```

This is appropriate only if callers are required to satisfy the precondition and an assertion failure is an acceptable development-time response. It is not a substitute for checking user-supplied input in a production interface. A release build may omit the assertion altogether, after which a bad call still attempts the invalid access.

Never place an operation with required side effects only inside `assert`; disabling assertions would also remove that operation.

13.4 A test harness without external libraries

For a small project, a conventional `main` that returns nonzero on failure is enough:

```
#include <stdbool.h>
#include <stdio.h>

static bool is_even(unsigned n)
{
    return n % 2U == 0;
}

int main(void)
{
    if (!is_even(0) || !is_even(8) || is_even(7))
        return 1;
    puts("tests passed");
    return 0;
}
```

The build system can run the executable and use its exit status to determine success. Larger suites should report the particular case that failed, but a simple test program can be both readable and valuable.

A command such as `printf 'a b c' | ./word_count` tests a whole program through its standard input and output. Such integration tests complement direct function tests, which are better at reaching internal boundary cases.

13.5 Warnings and language modes

A practical first compiler invocation is:

```
gcc -std=c23 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Werror -O2 example.c -o example
```

The exact useful flags vary by compiler, codebase, and target. In particular, `-Wconversion` can be noisy in legitimate numeric work, and `-Werror` can cause builds to fail when the compiler introduces new diagnostics. The correct response is to review warnings and make an explicit project policy, not to silence them indiscriminately.

Compile in the actual language mode. A compiler accepting one C23 keyword does not necessarily implement every feature or library facility. When a project supports more than one compiler, build with both. Different diagnostics often reveal different classes of mistakes.

13.6 Sanitizers

On supported host targets, AddressSanitizer can detect many out-of-bounds accesses, use-after-free errors, and other memory defects. UndefinedBehaviorSanitizer can detect selected unde-

fined operations, such as certain signed overflows or invalid shifts. They do not cover every form of undefined behavior, and their effectiveness depends on executing the erroneous path.

A useful build on a supported GCC or Clang environment is:

```
cc -std=c23 -O1 -g -fno-omit-frame-pointer \
   -fsanitize=address,undefined example.c -o example_san
./example_san
```

Sanitizer-enabled executables are typically host-oriented. Their instrumentation, runtime overhead, and dependencies can make them unsuitable for microcontrollers. Test algorithms on a host, then separately verify target-specific assumptions.

Do not interpret one clean sanitizer run as proof of a fully safe program. Test case selection remains decisive.

13.7 Unit tests and property tests

A unit test checks a specific expected value. A *property test* checks a relationship that should hold for many inputs.

For example, consider an in-place reversal of an array. If the operation is correct, reversing twice should restore the original sequence. The property holds for empty arrays, one-element arrays, and arrays with duplicates. A test driver can generate random arrays and check this relationship.

Similarly, if a serializer and parser implement the same format, decoding an encoded valid value should produce an equivalent value. This does not prove either routine matches an external specification: they could share the same mistake. Include independent golden test vectors alongside round-trip tests.

13.8 Fuzzing a parser

A fuzzer supplies many automatically generated byte sequences and observes whether the program crashes, violates a memory safety check, hangs, or triggers an assertion. A parser interface like

```
bool decode_packet(const unsigned char *bytes,
                   size_t length,
                   struct packet *result);
```

is well suited to fuzz testing because its input is a bounded byte sequence. The harness should supply valid storage of the declared size, initialize output state, invoke the parser, and verify that failure preserves the output.

Fuzzing discovers bugs; it does not establish absence of bugs. Use corpus inputs for known valid packets and edge cases, and preserve any newly discovered failing inputs as deterministic regression tests.

13.9 Compare optimization modes

Undefined behavior can cause surprising differences between `-O0` and `-O2`. A compiler is permitted to optimize under the assumption that no undefined operation occurs. It may therefore eliminate checks that seem defensive if the source has already performed an invalid operation.

When debugging a difference between builds, investigate lifetime, aliasing, integer overflow, uninitialized objects, sequencing, and incorrect function declarations. Do not rely on a particular assembly listing as the definition of C semantics.

13.10 Run the actual target

A host test cannot substitute for hardware validation when code depends on interrupt timing, cache coherency, memory-mapped registers, DMA, alignment restrictions, an RTOS, or the implementation's floating-point behavior.

The most effective architecture separates portable algorithms from target-specific adapters. For a firmware project, a packet decoder can be tested exhaustively on the host while the UART driver is validated on the real board. Integration tests must then establish that the adapter passes the right byte sequences and lengths to the decoder.

13.11 Document verified claims

A verification report should record the compiler and version, C library if relevant, command line, target architecture, test suite, and result. State the difference between *committed test code* and *tests actually executed*. If a test was not run, do not write that it passed.

For this manuscript, `make check` invokes a shell-based regression suite, while a GitHub Actions workflow can provide repeatable hosted checks. Both should be run on the precise committed revision before a stable release is declared.

13.12 Exercises

13.1 Add a test for unsigned-sum overflow that verifies the output argument remains unchanged.

13.2 Write a reversibility property test for a bounded array reversal.

13.3 Create a fuzzer harness that invokes the packet decoder on random byte sequences and checks output preservation.

13.4 Compare warning diagnostics on the same example under GCC and Clang. Explain differences without suppressing them globally.

13.5 State which properties of a ring buffer can be established with single-threaded tests and which need concurrency-specific reasoning.

13.6 Design a verification matrix for a program that must run on an x86-64 development host and an ARM Cortex-M microcontroller.

13.13 Make tests deterministic

A regression test should produce the same expected result from the same inputs under documented conditions. Tests based on timing thresholds, random seeds without recording them, or unspecified evaluation order are difficult to reproduce. When randomness is useful, record the seed and provide a way to replay the failing case exactly.

For the book's word index, generate a fixed set of vocabulary and expected counts. For the bounded queue, push and pop a known sequence through wraparound. For the byte decoder, use independent golden vectors defined by the protocol rather than relying only on round trips.

13.14 Test failures, not just successes

Many defects hide on error paths. Test the allocation path that fails, the input that is one byte too short, the array that is exactly full, and the output that must not change when conversion fails.

A useful method is *fault injection*. A test allocator can fail on the third request; a fake byte source can return an I/O error after a chosen number of bytes; a fake output sink can accept only a prefix of a write. The production algorithm should treat these failures as ordinary results.

When an API promises strong failure safety, initialize the output with an unusual value and verify it remains unchanged after every rejection. Be careful not to compare structure padding bytes with `memcmp` and interpret differences as changes in member values.

13.15 Static analysis and sanitizers complement each other

A static analyzer inspects source or intermediate representation without necessarily running every path. It may detect impossible conditions, potential null dereferences, and unchecked return values. False positives exist, and the analyzer's assumptions must be understood.

AddressSanitizer can catch many invalid memory accesses when they execute. UndefinedBehaviorSanitizer can catch selected invalid shifts, arithmetic, and other errors. Neither usually establishes absence of data races; ThreadSanitizer is a different instrument and has platform-specific limitations. Sanitizers also change timing and memory layout, so embedded hardware behavior still requires separate validation.

For numerical kernels, compiler warnings and differential tests with independent reference implementations may reveal issues that memory sanitizers cannot.

13.16 Differential testing

When implementing a custom parser, compare its accepted strings and decoded values with a second trusted implementation **after matching their grammars**. If one parser allows leading whitespace and the other forbids it, disagreement may be by design rather than a bug.

For a numeric function, compare against a reference with higher precision or a well-understood library routine over representative ranges. For a protocol codec, compare byte-level outputs against test vectors written from the format specification.

A test that encodes and decodes using two functions sharing the same mistaken endianness can pass while violating the external protocol. Independent expectations are essential.

13.17 CI as reproducible evidence

A GitHub Actions job should report its actual compiler and version, the selected dialect, and the command that ran. Separate compilation, unit tests, sanitizer builds and document generation into distinguishable steps so a failed PDF toolchain is not mistaken for failing C code.

A CI configuration file is not itself proof of a passing build. Evidence requires inspecting an actual completed workflow run for the exact commit. If the repository cannot expose run status through the available integration, say that verification is pending.

13.18 Choose a release criterion

A reasonable first reading edition can require all examples to compile under two supported compilers, all deterministic tests to pass, all Markdown links to resolve, and the generated PDF to include every chapter and appendix. A publishable technical book needs more: technical review, a coherent index, verified page layout, exercise solutions, and clear statements of compiler-specific features.

Further problems: Design a differential test for a little-endian decoder using independent literal expected values. Explain why a clean sanitizer run cannot prove every array index is valid. Add a fault-injectable allocation layer to a dynamic vector and report which postconditions remain true.

13.19 A test is a contract experiment

Testing becomes more systematic when each case is derived from a documented precondition, invariant, or postcondition. For a vector, the invariants include `length <= capacity` and valid

ownership of allocated storage. For a packet parser, failure must not modify the public result. For an SPSC queue, the producer and consumer must observe values in FIFO order under the single-producer/single-consumer protocol.

A test should state what it establishes. A successful sequential queue test demonstrates ordering and wraparound for the tested operations. It does not establish freedom from data races if producer and consumer execute simultaneously. A clean AddressSanitizer run shows that the tested execution did not trigger any instrumented errors; it does not prove all paths safe.

13.20 Use fault injection to reach unlikely branches

If a container allocates only a few dozen bytes, ordinary development runs rarely experience allocation failure. A design that can accept a resizing callback allows a test to deliberately return `nullptr` and verify strong failure safety. The book's `vector_checked.c` follows this pattern.

An analogous technique works for output interfaces. The complete `callback_sink.c` example supplies a fake sink whose write operation accepts at most two bytes per call. A `write_all` wrapper must retry the unwritten suffix until the whole buffer has been accepted. A second fake mode returns no progress, and the wrapper must report failure rather than loop forever.

These are not theatrical failures. Embedded UART queues can fill, filesystem writes can fail, network links can stall, and bounded buffers can run out of capacity. Making errors injectable produces reproducible tests for behavior that otherwise appears only in the field.

13.21 Test the smallest complete interface

A function-level test can verify a single boundary, but a project also needs tests for interfaces between modules. The module example in Chapter 8 exercises compilation through a header, implementation, and client. The word-index test exercises the complete command-line program through standard input and output. The PDF build tests documentation generation separately from C compilation.

Keeping these levels distinct helps diagnose failures. If compilation fails because of a C23 syntax diagnostic, rewriting a parser algorithm is unlikely to help. If a program compiles but returns the wrong table, investigate semantics and test data. If the PDF builder fails while C tests pass, investigate the document toolchain.

13.22 Measure coverage without worshipping a number

Line and branch coverage can reveal unexecuted paths, especially error-handling branches. They do not say whether a test checks the right result. A test might run the overflow check but ignore the output state afterward. A branch can be covered without verifying that it preserves a container's ownership contract.

For high-risk functions, review a small table of input classes: minimum size, maximum size, invalid value, missing data, exhausted resource, and valid boundary. Include a human explanation of why each case matters.

13.23 Reproducible CI

A useful CI run records the exact commit, compiler versions, flags, and exit status. Different compilers discover different issues, but changing to a newer toolchain can also change diagnostics. When the compiler misclassifies valid C23 syntax as an extension, handle that specific diagnostic rather than globally disabling `-Werror`.

For publication, add a second gate: the generated PDF must contain every chapter and appendix, have a readable table of contents, and be built from the same source revision being pub-

lished. A source repository containing an older PDF should not be treated as proof that the latest manuscript has been typeset.

Exercises

13.7 Write failure-injection cases for an allocator and a short-writing output sink. **13.8** List what a passing host test cannot establish about MCU interrupt timing. **13.9** Add a CI check that separately reports compiler-test and PDF-build failures. **13.10** Design independent golden vectors for a three-byte header codec.

14. Bits, Bytes, and Portable Data

C gives programmers direct control over integer representations and bytes. That power is especially useful for embedded devices, persistent files, and network protocols. But an integer's mathematical value is distinct from its in-memory representation, and a portable program must not confuse the two.

14.1 Storage units

An object occupies one or more bytes. In C, a byte is the size of a `char` and contains `CHAR_BIT` bits; while eight-bit bytes are overwhelmingly common, the language does not require `CHAR_BIT == 8`. A format specified in *octets* requires eight-bit units. A portable implementation must check that its target can represent and address the format as intended.

```
#include <limits.h>
#include <stdio.h>

int main(void)
{
    printf("bits per byte: %d\n", CHAR_BIT);
}
```

Exact-width integer types such as `uint32_t` exist when the implementation offers a type of exactly the specified width without padding bits. Fixed-width names clarify the protocol's mathematical range, but they do not choose byte order.

14.2 Masks, shifts, and flags

Use unsigned values for bit-level work. A left shift with a count greater than or equal to the promoted operand's width has undefined behavior, as does a negative shift count. The shift operators have their own integer-promotion rules; explicitly selecting a wide enough unsigned operand avoids accidental signed arithmetic.

```
#include <stdint.h>

static uint32_t extract_byte(uint32_t value, unsigned position)
{
    return (value >> (position * 8u)) & UINT32_C(0xff);
}
```

This helper requires the precondition `position < 4`. Without it, the shift count can be invalid. A production interface can check the position and return success through a separate status indicator.

A register read-modify-write operation may not have the semantics of an ordinary RAM variable. For instance, writing a one may clear a device status bit. Apply shifts and masks according to the hardware reference manual, not just the C expression's arithmetic meaning.

14.3 Endianness is part of the format

A little-endian 32-bit unsigned field stores its least significant octet first. A big-endian field stores the most significant octet first. For portable decoding, use shifts and masks on bytes rather than reinterpreting an input buffer as a pointer to `uint32_t`.

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

bool decode_u32_be(const unsigned char *src, size_t n, uint32_t *out)
{
    if (src == nullptr || out == nullptr || n < 4)
        return false;
    uint32_t v = ((uint32_t)src[0] << 24)
               | ((uint32_t)src[1] << 16)
               | ((uint32_t)src[2] << 8)
               | (uint32_t)src[3];
    *out = v;
    return true;
}
```

This assumes eight-bit input bytes as specified by the format. If the environment permits wider bytes, a protocol layer should explicitly define how octets are obtained. The output is assigned only after the bounds check.

14.4 Representation and alignment

An address in a byte stream may not meet the alignment required by an integer type. Converting that address and dereferencing a misaligned integer pointer may be undefined. Even on processors that allow unaligned loads, compiler assumptions or peripheral restrictions may differ. Byte-wise decoding avoids this issue.

Structure padding and bit-field layout may differ across implementations. Avoid transmitting raw structures as protocol records. Instead, define field lengths, byte order, permitted ranges, and checksums. Encode fields explicitly and test them against documented vectors.

14.5 The meaning of volatile

A memory-mapped device register may require volatile-qualified access so that the compiler performs observable reads and writes. But `volatile` does not specify the hardware bus transaction width, cache behavior, atomicity, memory ordering across cores, or synchronization with interrupts. Those require target-specific facilities and device documentation.

When two threads share ordinary application state, prefer atomics or appropriate synchronization rather than a volatile flag. The language's concurrency rules and a hardware peripheral's rules are related but not interchangeable.

14.6 A complete byte codec

See `examples/byte_codec.c`, which encodes and decodes a 32-bit value in big-endian octet order and verifies the result. It checks `CHAR_BIT == 8` at translation time and uses unsigned shifts. Its public-style functions take explicit output capacity and return `false` when storage is insufficient.

The tests exercise zero, a mixed-byte value, and the maximum 32-bit value. An actual protocol should also test malformed lengths, endianness against independent known vectors, and behavior when the caller supplies invalid storage.

14.7 Exercises

14.1 Write `decode_u16_le` with an explicit length and output pointer, preserving the output on failure.

14.2 Explain why `*(const uint32_t *)buffer` is not a portable replacement for a byte decoder.

14.3 Implement a 32-bit rotate-left operation without shifting by the type's width. Test shift counts zero, one, 31, and 32 after defining whether counts are reduced modulo 32.

14.4 Write a packet encoder with a documented maximum length, a version field, and a checksum. Prove that it never writes past its capacity.

14.5 Compare a host byte codec with the actual peripheral register interface of a chosen microcontroller. Identify which assumptions belong to C and which belong to the vendor's device specification.

14.8 Fixed-width data is not fixed layout

Declaring `uint32_t word;` specifies a 32-bit unsigned integer type if the implementation provides it. It does not say which byte of its object representation occupies the lowest address. Nor does it authorize unaligned access to an arbitrary buffer address after casting it to `uint32_t *`.

A format that defines 32-bit fields in network byte order requires explicit big-endian encoding and decoding, regardless of whether the host processor happens to use little-endian loads. Keeping representation conversion in small functions makes protocol code easy to test on multiple architectures.

14.9 Shifts need a domain proof

The expression `1 << position` has a signed left operand. For sufficiently large `position`, the result may not fit, or the shift count may equal or exceed the promoted width. Using an unsigned fixed-width operand addresses the signedness issue but not invalid counts.

```
#include <stdbool.h>
#include <stdint.h>

bool make_u32_mask(unsigned position, uint32_t *out)
{
    if (out == nullptr || position >= 32)
        return false;
    *out = UINT32_C(1) << position;
    return true;
}
```

This example rejects positions outside the 32-bit domain before performing the shift. It does not assume a particular memory representation of `uint32_t`.

14.10 Check the complete frame size

Suppose an output frame contains a four-byte header, a payload, and a one-byte checksum. If the caller gives a buffer of capacity `capacity`, it is tempting to test `capacity >= payload_length + 5`. But that sum may overflow `size_t`.

A safer shape is:

```
if (capacity < 5 || payload_length > capacity - 5)
    return false;
```

Only after this check is it safe to index all fields in a correctly allocated buffer of the declared capacity. This is an arithmetic *and* storage contract: the caller must actually provide a valid region that large.

14.11 MMIO is not just a pointer cast

In embedded programming, a peripheral register may be accessed through a volatile-qualified lvalue or vendor-provided accessor. The C language does not itself specify the address map, the side effects of reading, or whether writing a one clears a flag. The device manual and compiler/ABI documentation remain authoritative for such accesses.

A compound assignment like `REG |= MASK` may compile into a read followed by a write. It may be unsuitable for a register whose read has side effects, whose reserved bits must retain particular values, or whose updates must be atomic with respect to an interrupt. Some peripherals offer dedicated set/reset registers; use the documented mechanism.

14.12 Field tests beat raw dumps

For a byte codec, test zero, all bits set, alternating-bit patterns, and a distinct-byte value such as `0x12345678`. The distinct-byte case is especially useful for finding reversed byte order. Then test output buffers that are too small, null pointers when prohibited, and any nonstandard target assumptions.

Further problems: Write an encoder for a 24-bit unsigned field stored in three octets. Show that each shift count is valid. Explain why an ordinary struct copy and a wire serialization are different operations. Analyze a hypothetical write-one-to-clear status register.

14.13 A wire protocol is a specification, not a structure

Suppose a device transmits a frame with this byte-level layout:

Offset	Width in octets	Meaning
0	1	Magic value `0xA5`
1	1	Version `0x01`
2	2	Unsigned payload length, little-endian
4	0–64	Payload
4 + length	1	XOR of all preceding octets

This is an intentionally simple teaching format. The XOR field detects some accidental errors; it does not authenticate a sender and is not adequate against deliberate modification.

The format does not mention `struct`, padding, alignment, or host byte order. Those are properties of the implementation, not the external message. A portable encoder explicitly writes each field as bytes. A portable decoder reads each field only after establishing that the corresponding bytes exist.

14.14 Encode without partial publication

An encoder can reject an invalid request before writing any output: reject a null destination, insufficient capacity, a payload length beyond 64, and a nonzero payload length with no valid payload storage. Then compute exactly how many bytes are needed, using arithmetic already bounded by the format's maximum.

If failure must leave the destination unchanged, construct the complete frame in a local fixed-size temporary array and copy it only after validation. That costs one extra bounded copy but gives

callers a simple postcondition. For larger frames, an alternative API may write incrementally and report partial output; either policy is acceptable if explicit.

The companion `examples/frame_codec.c` demonstrates a strong-failure interface for both encoding and decoding. It uses `memcpy` only after checking capacities. Its local buffer size is determined by the maximum defined frame size rather than by untrusted input.

14.15 Decode in a safe order

For decoding, perform these checks in sequence:

1. Confirm that the input pointer and output pointer satisfy the API and that at least five octets exist.
2. Check magic and version fields.
3. Decode the two-byte length from individual octets.
4. Ensure length is at most 64 and the input contains exactly the declared frame length.
5. Verify the checksum.
6. Copy payload bytes into a local candidate and commit the result.

If the function subtracts a fixed overhead from the input length, it must first establish that the input length is at least that overhead. This prevents unsigned underflow. Because the maximum payload is small, the final length calculation can be proven to fit in `size_t` on supported targets.

14.16 Endianness and alignment are independent

A machine can allow unaligned access and still have a byte order different from the protocol. Conversely, a target may use the protocol's byte order but require alignment for native multi-byte loads. Casting the input pointer to a multi-byte integer pointer therefore makes more than one unsafe assumption.

The explicit decoder `(uint16_t)bytes[2] | ((uint16_t)bytes[3] << 8)` communicates the wire format and avoids reading a misaligned native integer. Since `uint16_t` might undergo integer promotions, the operation remains carefully bounded by the known eight-bit octet values.

14.17 Testing message boundaries

Test a zero-length payload, a payload at maximum length, one byte less than a complete frame, an extra trailing byte, a corrupted checksum, and an unsupported version. Confirm that failures leave the output object unchanged. Also test against a manually constructed frame rather than relying exclusively on an encoder/decoder round trip.

Exercises

14.6 Change the checksum from XOR to a documented CRC and provide independent golden test vectors. **14.7** Add an error enumeration for malformed, incomplete and oversized frames. **14.8** Design a streaming decoder using bounded state and explain how it handles resynchronization after corrupted input.

15. Expressions, Objects, and the Abstract Machine

A C program is not defined by the machine instructions observed in one build. It is defined by the language's abstract machine: a model of evaluation, objects, values, and observable behavior. A compiler is free to transform an implementation as long as it preserves the behavior required of a conforming program.

This principle explains both C's performance and its hazards. A short expression can have a precise meaning, an implementation-dependent meaning, or no defined meaning at all.

15.1 Values and objects

An *object* is a region of data storage whose contents can represent values. An *expression* can produce a value, designate an object, or have side effects. Consider:

```
int number = 7;
int *address = &number;
*address = 9;
```

The pointer contains a value that designates `number`. The indirection expression `*address` is an lvalue; the assignment modifies the designated object. The validity of the operation depends on `number` being alive, accessible, and represented by a suitable pointer.

Not every expression is a modifiable lvalue. A constant expression, a nonmodifiable qualified lvalue, and an array expression cannot simply be treated as destinations for assignment.

15.2 Storage duration and scope answer different questions

Scope describes where an identifier may be used in source code. *Storage duration* describes the lifetime of storage associated with an object. A local `static` object has block scope but static storage duration:

```
unsigned next_identifier(void)
{
    static unsigned value = 0;
    return ++value;
}
```

The object persists across calls. The function has a new problem: unsigned wraparound eventually repeats an identifier. It is also not inherently safe for unsynchronized concurrent calls. A compact interface does not guarantee suitable semantics.

An automatic object, by contrast, usually exists only for the relevant invocation. Returning its address does not extend its lifetime.

15.3 Sequenced and unsequenced evaluations

Consider the expression

```
int result = f() + g();
```

The language does not require the call to `f` to finish before the call to `g`, or vice versa. It does require both results before their addition can produce the final result. If the functions only compute independent values, that freedom usually does not matter.

If both calls modify the same non-atomic scalar object, their unsequenced side effects can make the expression undefined. Write separate statements when an ordering is required:

```
int left = f();
int right = g();
int result = left + right;
```

The full expressions are sequenced in source order. Still check that the addition itself is representable for signed types.

The operators `&&`, `||`, and comma (when used as an operator) introduce sequencing properties that differ from ordinary arithmetic operators. A function-call argument list is not an ordering guarantee.

15.4 Integer overflow and conversions

For an unsigned integer type, arithmetic is performed modulo one more than its maximum representable value. Signed arithmetic overflow has undefined behavior. These are not implementation details that can safely be ignored because a processor uses two's complement.

```
#include <limits.h>
#include <stdbool.h>

bool increment_int(int value, int *out)
{
    if (out == nullptr || value == INT_MAX)
        return false;
    *out = value + 1;
    return true;
}
```

The guard precedes the addition. A post-addition check would already be too late.

Conversion introduces another boundary. Narrowing an integer to a smaller unsigned type follows modular rules. Converting an out-of-range value to a signed integer type has implementation-defined behavior or can raise an implementation-defined signal. Prefer range checks before narrowing, particularly when parsing files or messages.

15.5 Effective type and aliasing

Object access rules restrict which lvalue types may be used to read or write stored objects. A programmer cannot generally take the address of a `float`, cast it to `int *`, dereference it, and expect portable reinterpretation. Doing so can violate alignment and effective-type rules as well as make code sensitive to representation.

For object-representation inspection, an appropriately sized array of `unsigned char` and `memcpy` are usually clearer. Representation inspection does not imply that every bit pattern is a valid value of every destination type.

```
#include <string.h>
#include <stdint.h>

uint32_t bits_of_float(float x)
{
    uint32_t bits = 0;
    /* Requires equal sizes, verified for the selected target. */
    if (sizeof bits == sizeof x)
        memcpy(&bits, &x, sizeof bits);
    return bits;
}
```

This example exposes representation bytes without violating aliasing rules, but the resulting integer's meaning depends on the target's floating-point representation and byte order. It is **not** a portable way to serialize a floating value.

15.6 Order, lifetime, and optimizers

A valid optimizer may assume that undefined behavior does not occur. Consequently, source checks appearing after invalid operations may be removed. The correct style is to validate the condition *before* executing the operation it protects.

For array access, verify the index against a known count before evaluation of `array[index]`. For allocation, verify multiplication before computing required bytes. For pointer use, ensure the

object has not been freed or otherwise had its lifetime ended.

15.7 A small checked interval

Suppose a program writes `count` bytes beginning at `offset` into a buffer of `capacity` bytes. The tempting check `offset + count <= capacity` can itself overflow when the quantities use an unsigned type. Rewrite the requirement into conditions that can be checked safely:

```
#include <stdbool.h>
#include <stddef.h>

bool fits(size_t offset, size_t count, size_t capacity)
{
    return offset <= capacity && count <= capacity - offset;
}
```

The `&&` operator short-circuits. Therefore `capacity - offset` is evaluated only when `offset <= capacity`. This both avoids underflow and expresses the intended invariant.

A complete executable version with boundary tests appears in `examples/checked_interval.c`.

15.8 Exercises

15.1 Explain why `i = i++ + 1` does not provide a portable way to increment a scalar.

15.2 Give an example of block scope combined with static storage duration. Describe the consequences for recursive calls.

15.3 Implement a checked multiplication of two `size_t` values, preserving its output argument on failure.

15.4 Design a test matrix for `fits` including zero capacity, exact fits, large offsets, and `SIZE_MAX`.

15.5 Explain why reading an integer directly from an arbitrary byte-buffer address can violate alignment or effective-type rules.

15.6 Distinguish a language's unspecified evaluation order from an implementation-defined integer conversion.

15.9 The optimizer reasons from the language rules

Compilers are not required to preserve an apparent intention that depends on undefined behavior. Consider:

```
int *p = /* supplied by caller */;
int value = *p;
if (p == nullptr)
    return -1;
```

The program already dereferences `p` before testing it. A compiler may optimize under the assumption that the dereference had defined behavior; moving the test after the access has made it ineffective. The correct ordering is to check the pointer first and then dereference.

Likewise, `int sum = a + b; if (sum < a) ...` does not provide a general overflow test for signed integers, since overflowing addition already has undefined behavior. Establish representability before evaluating the addition.

15.10 Object representation versus value

Every object has an object representation consisting of bytes. The interpretation of those bytes depends on the type. Some object representations can contain padding or trap representations for

some types or implementations. Reading arbitrary bytes as if they were a fully initialized object of any chosen type is not generally safe.

The character types have special significance for inspecting object representations. Copying bytes using `memcpy` is commonly preferable to violating aliasing rules through an incompatible pointer cast. But reproducing the bytes of a pointer or floating-point value on another machine does not guarantee the same value there.

15.11 Sequence and indirection

An expression such as `array[i++] = array[i]` deserves suspicion: side effects and value computations may be unsequenced in a way that violates the rules. A clearer version splits responsibilities:

```
size_t index = i;
int value = array[index];
array[index] = value;
++i;
```

This example is intentionally trivial, but each step now has an explicit order. The larger lesson is not to ban compact expressions, but to separate operations when sequencing is a necessary part of correctness.

15.12 Bounds and pointer provenance

A pointer is more than a numeric address for the purposes of the C abstract machine. Pointer arithmetic and dereference are constrained by the object and its lifetime. Converting an arbitrary integer to a pointer and dereferencing it is not portable ISO C, even on targets where hardware maps devices at fixed numeric addresses.

Freestanding systems often provide implementation-specific extensions for memory-mapped I/O and linker-defined memory locations. Such code should be isolated and documented rather than presented as a portable language feature.

15.13 A reasoning template for code reviews

For each expression ask: Which types participate? What conversions occur? Which operations can overflow or have invalid operands? What is the evaluation ordering? Which object is accessed? Is its lifetime active, and is the access permitted through this lvalue type?

This short set of questions is particularly effective for parsing, arithmetic kernels, and pointer-heavy utilities.

Further problems: Rewrite an overflow-after-addition test into a valid precondition check. Identify the violated rule in a dangling-pointer return. Explain why `memcpy` may safely move representation bytes even though a cast-and-dereference technique is not generally interchangeable.

15.14 Draw the memory model before optimizing

A useful sketch separates objects and pointers. For a vector, draw one structure containing a pointer, a length, and a capacity. Draw a separate heap block containing the elements. A shallow assignment of the vector copies its three fields into another structure, creating another pointer to the same block. It does not allocate a second block.

Now suppose one vector calls `realloc`. The old block's lifetime ends on success. Any pointer borrowed from the old block is invalid; the second shallow-copied owner also has an invalid pointer. Destroying both structures can produce invalid releases. The simplest fix is not a clever cast or compiler option but an ownership rule forbidding such copies.

15.15 Distinguish value equality from byte equality

For an ordinary `struct`, two values may have equal members but different padding bytes. Comparing their raw representations with `memcmp` is not a general semantic equality test. For floating values, even member-wise == needs an application policy concerning NaNs and signed zeros.

Serialization is a further step: it needs a stable external format independent of structure padding, compiler ABI, and machine byte order. This is why Chapter 14 constructs protocol fields byte-by-byte instead of writing the in-memory structure directly.

15.16 Expression transformations need preconditions

A mathematically valid rearrangement is not always valid for finite-width C arithmetic. The expression `offset + count <= capacity` looks equivalent to `count <= capacity - offset`, but only the latter can be evaluated safely after first checking `offset <= capacity` when all quantities have type `size_t`.

Similarly, `a + b` can overflow for signed integers even when a later comparison attempts to detect it. A valid transformation must preserve the language-defined behavior for all permitted inputs, not merely produce the same result on small examples.

15.17 Observable behavior and the optimizer

Compilers may reorder and eliminate operations when permitted by the language's abstract-machine rules. The fact that a debugger shows a particular sequence of machine instructions does not make that sequence the semantics of the source program.

For concurrent code, atomics and synchronization express ordering requirements across threads. `volatile` affects certain observable accesses but is not a general mechanism for sharing ordinary variables safely. For device registers, additional target-specific rules determine bus transactions and barriers.

Exercises

15.7 Draw the heap and stack objects involved in `vector_append`, then redraw them after successful growth. **15.8** Identify when pointer aliases become invalid. **15.9** Compare two structures semantically without reading padding bytes. **15.10** Transform a potentially overflowing length sum into a checked subtraction formulation.

16. Atomics, Threads, and Communication

Two threads executing the same valid sequential code do not necessarily produce a valid concurrent program. In C, unsynchronized conflicting accesses to a non-atomic object can form a data race and yield undefined behavior. The standard atomic library gives us a way to coordinate access, but it does not eliminate the need to design ownership and progress.

16.1 A race on a counter

Suppose two threads each execute `++counter` one thousand times. The operation is a read, computation, and write; it is not necessarily one indivisible action. Without suitable synchronization, concurrent increments of an ordinary `int` are a data race. A result of two thousand is not guaranteed, and the program has undefined behavior rather than merely a risk of losing a few increments.

An atomic object makes individual accesses atomic:

```
#include <stdatomic.h>
```

```
atomic_uint counter = 0;

/* Called by either worker: */
void count_event(void)
{
    atomic_fetch_add_explicit(&counter, 1u, memory_order_relaxed);
}
```

Relaxed ordering suffices when all that matters is a single independent count, not the visibility of some other data. It does not order ordinary memory operations in a way that can publish a complex object.

16.2 Publication with release and acquire

Imagine a producer fills a payload and then signals that it is ready. The consumer must not read the payload before the producer's writes have been made visible through synchronization.

```
#include <stdatomic.h>
#include <stdbool.h>

static int payload;
static atomic_bool ready = false;

void publish(int value)
{
    payload = value;
    atomic_store_explicit(&ready, true, memory_order_release);
}

bool try_consume(int *out)
{
    if (out == nullptr)
        return false;
    if (!atomic_load_explicit(&ready, memory_order_acquire))
        return false;
    *out = payload;
    return true;
}
```

This is a **one-shot publication example**. It is valid only under a protocol in which one producer publishes once, after which `payload` is never modified, and consumers do not access it before an acquire load observes `true`. Reusing the flag for repeated publications without an additional state transition and synchronization would be incorrect.

A release store synchronizes with an acquire load that reads from it, establishing a happens-before relationship for earlier writes. The atomic flag does not itself turn every access to `payload` into an atomic access.

16.3 A bounded single-producer/single-consumer queue

For repeated communication we need a real protocol. A single-producer/single-consumer (SPSC) ring buffer can keep separate atomic positions. Only the producer modifies the write position, and only the consumer modifies the read position.

Reserve one slot to distinguish full from empty. With capacity eight, the queue can hold at most seven items. The producer writes a non-atomic data element, then publishes its advanced write position with release semantics. The consumer acquires the write position before reading the

element. The consumer releases its advanced read position after finishing; the producer acquires it before reusing the slot.

The example in `examples/spsc_queue.c` implements precisely this one-producer/one-consumer algorithm. It is **not** a multiple-producer or multiple-consumer queue. It also does not directly establish that its operations are safe to invoke from an interrupt handler on every implementation: that is a separate target-specific question involving lock-free guarantees, runtime support, and interrupt semantics.

C23 initialization note. Older atomic examples may use `ATOMIC_VAR_INIT`, but that macro was removed in C23. The standalone SPSC example uses ordinary `0` initialization for its atomic index members before either participant starts. Once concurrent operation begins, the participants access those positions only through atomic operations with the documented ordering.

16.4 Why index wraparound is not numeric overflow

An SPSC ring normally advances an index with `(index + 1) % CAPACITY`. If index values are always less than capacity, and capacity is suitably small, the addition stays in range. The modulo computes a position within the array. It should not be confused with allowing arbitrary integer overflow.

An invariant for the chosen queue is `0 <= read_index < CAPACITY` and `0 <= write_index < CAPACITY`. Equal positions signify empty; advancing the writer to the reader's current position would signify full and must be rejected.

16.5 Lifetime and shutdown

A queue's memory must remain alive until both participants stop accessing it. Atomic operations do not rescue a buffer freed while a worker still runs. A threaded application therefore needs a termination protocol and a join or equivalent completion mechanism.

This often has greater practical importance than micro-optimizing a memory order. A formally correct queue implementation is still unsafe if its containing object is destroyed too early.

16.6 Mutexes remain valuable

Lock-free algorithms are not necessarily faster, simpler, or more predictable. A mutex protects a group of related operations and is often the right design for moderate contention. In an RTOS, priority inheritance and interrupt-safe API restrictions matter; do not assume that a host C mutex can be used inside an interrupt handler.

Atomicity, mutual exclusion, wait-freedom, and real-time boundedness are different properties. Assess them independently.

16.7 C23 and implementation support

The C atomic facilities are specified by the language and standard library, but target availability and lock-free properties depend on the implementation. Check `atomic_is_lock_free` when relevant and verify the supported interrupt and threading model on the actual hardware. Freestanding environments may provide only a subset of hosted facilities.

On common hosted implementations, compiling threaded examples may require additional compiler and linker options. Keep those options separate from programs that do not require threading.

16.8 Exercises

16.1 Explain why `volatile unsigned counter` does not fix a data race.

16.2 Draw the release/acquire relationships for one successful SPSC enqueue and dequeue.

16.3 Extend the SPSC example with a clear shutdown protocol. Explain object lifetime requirements.

16.4 Determine whether a capacity-eight ring that leaves one slot empty can hold eight elements. Provide tests for full and empty transitions.

16.5 Explain why changing two producer threads to call the same SPSC enqueue operation breaks the algorithm.

16.6 Compare an SPSC queue with a mutex-protected queue for latency, correctness, maintainability, and portability.

16.9 Identify the exact shared objects

Concurrency reasoning starts by listing which objects are accessed by more than one participant. In the SPSC queue, the producer publishes `tail` and observes `head`; the consumer publishes `head` and observes `tail`. Both positions are atomic. Data slots are ordinary objects, but the publication protocol ensures the producer and consumer do not access the same occupied slot without the appropriate ordering.

Merely declaring the indices atomic is not sufficient if the buffer is accessed outside that protocol. Every operation must respect the rule that only the producer enqueues, only the consumer dequeues, and the queue object remains alive until both stop.

16.10 Understanding the release/acquire pair

A producer writes a slot, then uses a release store to publish the new tail. A consumer that reads the published tail with an acquire load can safely observe the preceding slot write. After consuming the element, the consumer releases its new head position. The producer uses an acquire load of head before deciding that a slot can be reused.

The two directions solve distinct problems: publication of newly written data and reclamation of space after consumption. Using only one direction would not establish all of the required cross-thread ordering in the intended design.

16.11 Sequential tests do not demonstrate concurrency correctness

A single-threaded test that pushes seven items, observes full, pops three, and wraps around is useful for verifying index arithmetic and FIFO ordering. It does not establish absence of a data race during simultaneous operations.

A concurrency test can run one producer and one consumer with a known sequence and verify that every item is transferred exactly once and in order. A stress test with variable delays may expose mistakes, but even millions of successful transfers cannot prove the memory-order argument. Reasoning about the C memory model remains necessary.

16.12 Interrupts are outside a generic hosted-thread proof

An interrupt handler is not automatically equivalent to a C standard thread. The compiler, runtime, architecture, and RTOS specify which atomic operations are safe in that context, whether they are lock-free, and how memory barriers interact with devices.

An atomic operation implemented through a library helper or lock may be unsuitable in an interrupt. Even lock-free machine instructions can require cache coherency or architectural barriers beyond what a generic application-level description conveys. Verify the target before advertising an SPSC queue as interrupt-safe.

16.13 Lifetime and cancellation

Stopping a producer is not the same as ensuring it has stopped. A queue owner must establish that neither participant will access the queue before releasing its memory. A shutdown protocol might signal cancellation, wake blocked consumers, join worker threads, and only then destroy the queue.

The choice between draining and discarding pending items should be explicit. Either can be correct depending on the application, but implicit destruction while work remains creates lifetime hazards.

Further problems: Create a two-thread FIFO stress test recording item sequence numbers. Explain the effect of replacing a release store with a relaxed store. Sketch a shutdown protocol for a queue shared between a worker thread and an RTOS task.

16.14 Memory ordering is a protocol, not a performance switch

The SPSC queue works because it establishes two publication relationships. The producer writes an ordinary slot and then publishes an updated tail through a release operation. The consumer acquires that tail before reading the slot. In the opposite direction, the consumer finishes reading the slot and releases its updated head; the producer acquires head before reusing that storage.

It is tempting to label every atomic access `memory_order_relaxed` for speed. But relaxed atomics guarantee the atomicity and modification order of the atomic object, not the required visibility of ordinary slot writes to another thread. The release/acquire relationships are part of this algorithm's proof.

Conversely, using `memory_order_seq_cst` everywhere does not magically repair a design that lets two producers concurrently modify a slot intended for one producer. Memory ordering and permitted roles are separate requirements.

16.15 Read the single-producer guarantee literally

An SPSC queue has one producer and one consumer *at a time under the documented protocol*. Calling `push` from a second producer can make both read the same tail, write the same slot, and publish conflicting updates. Atomic tail storage does not turn the remaining sequence into an atomic transaction.

A multiple-producer queue usually needs additional synchronization or a different algorithm. If a single interrupt is the producer and one task is the consumer, you must still verify that the compiler, target architecture and RTOS support the chosen atomic operations in that interrupt context.

A callback that sometimes runs on the producer thread and sometimes runs in an interrupt is not automatically a single producer under a safe schedule. Identify the actual execution contexts.

16.16 Progress guarantees are different

A nonblocking operation returns without sleeping for a mutex, but the terms *lock-free* and *wait-free* have precise algorithmic meanings. Lock-free generally concerns system-wide progress; wait-free concerns per-operation completion in a bounded number of steps. Neither necessarily means low latency under every cache, interrupt or scheduling condition.

The small SPSC queue has bounded local index work, but whether its atomic primitives are implemented without locks depends on the target. On a small MCU, a supposedly atomic operation may be lowered to a helper routine that is inappropriate in an interrupt handler.

Measure real target execution and verify the atomic implementation before making a real-time guarantee.

16.17 Shutdown and ownership

Memory ordering does not establish object lifetime by itself. Imagine a queue owner frees the queue while a consumer is about to load its head. Even though the member is atomic, the storage is gone and the access is invalid.

A shutdown protocol can announce cancellation, stop the producer, let the consumer drain or abandon remaining items by policy, wait for both to complete, and only then reclaim storage. A good API states who owns the queue object and which operation establishes that no in-flight access remains.

For embedded systems, task deletion and ISR unregistering can each require target-specific synchronization. Test the shutdown path, not only steady-state traffic.

Exercises

16.7 Draw the two release/acquire edges on an eight-slot queue. **16.8** Explain why a second producer is unsafe despite atomic index variables. **16.9** Compare mutex-based and atomic-based designs for a real-time telemetry queue. **16.10** Describe a shutdown sequence that makes destruction safe.

17. Function Pointers, Callbacks, and Interfaces

An ordinary function call names an operation directly. A callback allows a caller to supply the operation itself. This is useful for sorting, traversing data, handling events, and separating reusable code from application policy.

17.1 A function pointer

Consider a function that compares two integers:

```
static int compare_int(int a, int b)
{
    return (a > b) - (a < b);
}
```

The subtraction produces -1, 0, or 1 without calculating `a - b`, which could overflow. A pointer to such a function can be declared as:

```
int (*comparison)(int, int) = compare_int;
int result = comparison(4, 7);
```

The parentheses around `*comparison` are essential. Without them, the declaration would describe a function returning a pointer rather than a pointer to a function.

17.2 A callback with context

Practical interfaces often accept both a callback and an opaque context pointer. The context lets the caller maintain state without resorting to global variables:

```
#include <stddef.h>
#include <stdbool.h>

typedef bool (*visit_fn)(int value, void *context);

bool visit_all(const int *items, size_t count,
               visit_fn visitor, void *context)
{
```

```
    if (visitor == nullptr || (count != 0 && items == nullptr))
        return false;
    for (size_t i = 0; i < count; ++i)
        if (!visitor(items[i], context))
            return false;
    return true;
}
```

The visitor returns `true` to continue and `false` to stop. In this design `false` from the traversal means either an invalid argument or an early stop; a richer status type could distinguish them.

The `void *` context is opaque to the traversal function. The caller is responsible for supplying a pointer to the appropriate object and keeping it alive for the duration of the call. Converting an object pointer to `void *` and back preserves its intended use when the underlying type and lifetime are correct.

17.3 A stateful visitor

```
struct accumulator { long long total; };

static bool accumulate(int value, void *context)
{
    struct accumulator *state = context;
    state->total += value; /* assume the sum is representable */
    return true;
}
```

A callback can change the caller's state without changing the traversal algorithm. In production, this example should either check overflow or document a bound on the count and input values.

Callbacks are usually synchronous in simple C library interfaces, but not every callback API is synchronous. An asynchronous system must state whether it retains the context pointer, which thread invokes the callback, and how cancellation and destruction interact.

17.4 Sorting and the comparison contract

The standard `qsort` function takes a comparator that accepts pointers to two objects and returns a negative, zero, or positive result. A comparator must be consistent; it must not produce contradictory orderings for the same values.

```
static int order_ints(const void *left, const void *right)
{
    int a = *(const int *)left;
    int b = *(const int *)right;
    return (a > b) - (a < b);
}
```

The casts are appropriate when the comparator is called by `qsort` on an array of `int`. A comparator should not simply return `a - b` because signed subtraction may overflow. The function cannot assume that its arguments refer to arbitrary or mismatched types.

17.5 Function pointers are not object pointers

The language distinguishes pointers to functions from pointers to objects. Do not assume a function pointer can be stored portably in `void *`. Some operating systems provide dynamic-loading interfaces with platform-specific conventions, but those conventions are not general ISO C guarantees.

Likewise, do not manufacture a function pointer from an integer address and call it without platform-specific guarantees. Embedded startup code and vector tables sometimes rely on implementation-defined representations; such code belongs behind a documented target interface.

17.6 Interfaces as boundaries

A driver interface might offer methods to read, write, and close a device:

```
#include <stddef.h>
#include <stdbool.h>

struct byte_sink {
    bool (*write)(void *context, const unsigned char *data, size_t size);
    void *context;
};
```

An application can then accept a `struct byte_sink` instead of knowing whether output goes to UART, a file, or a memory buffer. This style permits host tests using a fake sink. Yet it carries obligations: the context must remain valid, operations need failure semantics, and concurrent access must be synchronized if shared.

17.7 Costs and limits

Callbacks add indirection and may limit compiler inlining. That cost is often negligible compared with I/O and other operations, but it can matter in a small real-time inner loop. Measure rather than assume. A callback-based interface also needs careful error handling so that failures are not silently ignored.

17.8 Exercises

17.1 Implement a visitor that counts elements greater than a threshold supplied in its context.

17.2 Use `qsort` to order signed integers. Test `INT_MIN` and `INT_MAX` to show why a subtracting comparator is unsafe.

17.3 Design a byte sink whose write function may accept fewer bytes than requested, and define a complete-write wrapper.

17.4 Explain why casting a function pointer to `void *` is not a portable storage strategy.

17.5 Show how a synchronous callback's context lifetime differs from an asynchronous callback's lifetime.

17.9 Make callback contracts explicit

A callback interface has two contracts rather than one. The host function promises when and how it invokes callbacks. The callback promises which values and side effects it supports. A traversal API should specify whether it visits elements in order, whether it may stop early, and whether callbacks are allowed to modify the traversed collection.

Reentrancy is another decision. If a callback invokes the traversal function again, will the operation remain correct? A traversal that uses only local state may be naturally reentrant; one that uses a static cursor may not. If callbacks can be invoked concurrently, the context object also needs an appropriate synchronization policy.

17.10 Context lifetime and asynchronous operations

A synchronous visitor that returns before its caller exits can safely borrow a stack-local context if it does not retain the pointer. An asynchronous driver that stores the callback and context for a later event cannot use a context that has already gone out of scope.

```
struct counter_state { unsigned count; };

void register_callback(void (*callback)(void *), void *context);
```

This declaration alone says nothing about whether the callback is invoked immediately, on another thread, or after the registration function returns. A real API must specify that behavior. It must also provide a safe way to unregister and determine when in-flight callbacks have stopped before the context is destroyed.

17.11 Design a complete-write adapter

An abstract byte sink can report the number of bytes actually accepted rather than simply success or failure. A wrapper then loops until the entire record is written or the sink reports no progress or error. Such an adapter can unify a UART, a buffered memory sink, and a file stream while preserving their differing behavior at the boundary.

The wrapper must use subtraction to calculate remaining bytes, handle a zero-byte result without looping forever, and enforce the caller's storage contract. If the sink is nonblocking, `would block` may deserve a distinct status rather than being treated as fatal.

17.12 qsort and ordering

A comparator used with `qsort` must induce a consistent ordering. For integers, use relational comparisons rather than subtraction to avoid overflow. For structures, compare fields in a documented priority order. For floating values, decide how NaNs should be treated; ordinary relational comparisons do not automatically provide a total order over every floating value.

Sorting also changes element positions. Any pointer to an element's current location may no longer designate the same logical record after sorting. The ownership of the array itself does not change, but references to records by index may need to be updated.

17.13 Interfaces and performance

Function-pointer indirection can inhibit inlining and complicate static call-graph analysis, but sometimes it improves modularity enough to outweigh its cost. In firmware, an interface through function pointers may be suitable for the device layer while a hot DSP inner loop uses direct calls.

Measure on target hardware when performance matters. A desktop microbenchmark may not predict instruction cache, branch prediction, or linker optimizations on a Cortex-M core.

Further problems: Write a reentrant traversal that avoids global state. Define unregister semantics for an asynchronous callback API. Implement an ordered comparator for records containing a timestamp and a unique identifier.

17.14 A practical byte-sink interface

A callback can separate a useful algorithm from the device performing its I/O. Define a writer that receives a context pointer, source bytes, and a count, and returns how many bytes it accepted:

```
#include <stddef.h>

typedef size_t (*sink_write_fn)(void *context,
                                const unsigned char *data,
```

```
                                size_t count);

struct byte_sink {
    sink_write_fn write;
    void *context;
};
```

The interface contract must forbid returning more bytes than requested. A zero result means no progress under this simplified synchronous protocol, so a complete-write wrapper reports failure rather than spinning indefinitely.

17.15 Handling partial writes

A wrapper starts at offset zero. It calls the sink with `data + offset` and `count - offset`, then increases the offset by the number of bytes accepted. Since the call occurs only when `offset < count`, these operations are within a valid caller-supplied byte buffer. If the sink returns zero or a value larger than the remaining count, the wrapper reports failure.

The loop's progress proof is straightforward: on every successful iteration, offset strictly increases and never exceeds count. The loop therefore terminates after a finite number of accepted writes, even if the sink accepts just one byte at a time.

This design is particularly relevant to a UART driver or nonblocking transport. A real asynchronous system may need a 'would block' status and a state object retained across callbacks. That is a different protocol from the simple synchronous function here.

17.16 Test using a fake device

The companion `examples/callback_sink.c` uses a fake writer that can accept only two bytes per call. The complete-write wrapper must repeatedly call it to deliver a longer message. The test then switches the fake into a mode that accepts zero bytes, verifying that the wrapper fails without looping forever.

A fake device is useful because it isolates the algorithm from real hardware and permits exact, repeatable failure behavior. Hardware validation remains necessary for UART timing, DMA, interrupt interaction, and other target-specific concerns.

17.17 Lifetime and reentrancy

The sink's context pointer is borrowed during each synchronous call. If the sink stores that pointer and uses it after returning, a new lifetime contract is required. Likewise, a sink used simultaneously by multiple threads may require a mutex or atomics. The callback pointer itself does not provide thread safety.

A reusable interface should specify whether a callback may call back into the same interface recursively. Such reentrancy can be safe for a purely local algorithm but dangerous for a driver with shared mutable state.

Exercises

17.6 Modify the sink to report 'would block' separately from a permanent error. **17.7** Write a wrapper that resumes a partially sent message on a later call. **17.8** Design a byte sink with a maximum record length and checked capacity. **17.9** Explain what the caller must guarantee about the lifetime of its context object.

18. Text, Encodings, and Robust Parsers

Text is often treated as the simplest form of input, but a text interface combines several independent conventions: how bytes are encoded, what counts as a character, where a line ends, and which text represents a valid value. C gives programmers byte-oriented primitives; a program must choose and document its interpretation of those bytes.

18.1 The byte is not the character

A C string is a sequence of `char` values terminated by a zero byte. The library function `strlen` counts units before the terminator. For UTF-8 text, that count is not necessarily the number of Unicode scalar values or user-perceived characters.

UTF-8 encodes a code point in one to four octets. Continuation octets do not stand alone. A decoder must reject overlong sequences, surrogate code points, values beyond U+10FFFF, and incomplete sequences. A robust decoder also must not read past the caller's buffer when the leading octet suggests more input than remains.

18.2 Parse lengths explicitly

An untrusted message need not have a terminating zero byte. A parser can accept a pointer and a length, so no operation depends on searching outside the supplied region.

```
#include <stddef.h>
#include <stdbool.h>

bool ascii_digit(unsigned char c)
{
    return c >= '0' && c <= '9';
}
```

This function classifies one byte without invoking locale-sensitive character classification. For applications that require whitespace recognition under a locale, `isspace` is useful, but its argument must be `EOF` or a value representable as `unsigned char`.

18.3 Decimal parsing as a checked transition

A decimal parser maintains an accumulated number. Each incoming digit requests a transition from `value` to `value * 10 + digit`. For an unsigned maximum `MAX`, that transition is valid only when

[value \leq \left\lfloor\frac{MAX-digit}{10}\right\rfloor.]

The inequality must be tested *before* multiplication. A checked parser can therefore reject overflow without ever evaluating the overflowing expression.

A complete implementation appears in `examples/parse_unsigned.c`. Its contract is intentionally narrow: only nonempty ASCII decimal digits are allowed, and failure leaves the output unchanged. This is different from `strtoul`, which has defined syntax for optional whitespace, signs, and bases and uses `errno` for range errors.

18.4 A bounded UTF-8 decoder

The companion program `examples/utf8_decode.c` decodes a single Unicode scalar value from a counted input buffer. It reports how many octets were consumed on success and does not change either output on failure.

For example, the sequence `E2 82 AC` represents U+20AC (the euro sign). But `C0 80` must be rejected: it is an overlong representation of U+0000. The byte sequences `ED A0 80` and `F4 90 80` must also be rejected because they denote disallowed scalar values.

The example's purpose is decoding, not Unicode normalization, segmentation, case folding, or character rendering. Those operations require additional specifications and, in real-world applications, frequently a carefully selected text library.

18.5 Failure and incomplete input are distinct

A streaming decoder may encounter a prefix of a potentially valid sequence, such as the first two octets of a three-octet UTF-8 character. A one-shot parser can report failure and ask its caller to provide more bytes. A streaming parser often benefits from three results: valid, invalid, and incomplete.

That distinction prevents a network receiver from prematurely rejecting input merely because it arrived in pieces. It also prevents an actual malformed sequence from being treated as indefinitely incomplete.

18.6 The language's execution character set

C programs have implementation-defined character representations for source and execution environments, within the language's requirements. The ordinary literal `'A'` does not universally imply the integer 65. An ASCII-oriented parser must state its ASCII requirement explicitly.

The examples in this chapter use octet values for UTF-8 and the digit range for decimal input. Such assumptions are appropriate when decoding formats defined in ASCII or UTF-8, but they are not claims about every C execution character set.

18.7 A secure parsing checklist

Before parsing an untrusted input, identify its maximum length, byte encoding, permitted grammar, numeric ranges, truncation policy, and error reporting. Validate a complete candidate before writing durable output state. Avoid relying on null terminators unless the interface guarantees them.

Tests should include zero-length input, all valid boundary values, incomplete prefixes, extra trailing bytes, and malformed encodings. Pair handwritten test vectors with fuzzing, and preserve discovered failures as regression tests.

18.8 Exercises

18.1 Implement a parser for unsigned hexadecimal digits with an explicit buffer length and checked overflow.

18.2 Extend the UTF-8 decoder to distinguish incomplete input from malformed input.

18.3 Explain why byte length, Unicode scalar count, and displayed character count can differ.

18.4 Add fuzz tests checking that malformed input never changes the decoder's outputs.

18.5 Design a streaming line protocol that accepts CRLF and LF line endings while bounding the line length.

18.6 Compare a locale-sensitive classification function with a format-specific ASCII byte test.

18.9 UTF-8 validity is more than continuation bytes

A four-byte UTF-8 sequence is not valid merely because its final three bytes begin with `10` in binary. The decoded value must also lie between U+10000 and U+10FFFF. A three-byte sequence must not encode a value below U+0800 or any surrogate in U+D800–U+DFFF. A two-byte sequence must not encode an ASCII value.

These rules reject overlong encodings and invalid Unicode scalar values. A robust decoder should validate both the structure of each sequence and the numerical range of its result before publishing it.

18.10 Character count versus display width

UTF-8 code points are not synonymous with visual characters. A base letter followed by a combining accent can use two scalar values but appear as one grapheme cluster. Some visible characters use multiple code points joined by special sequences. Consequently, a user-interface operation such as cursor movement or truncation by displayed character count needs Unicode segmentation rules beyond a basic decoder.

A low-level C parser can be correct while knowing nothing about grapheme clusters. Its contract should say that it counts bytes or scalar values, not vaguely claim to count characters.

18.11 Incremental decoding as a state machine

A streaming decoder can retain a partial code point, a count of expected continuation bytes, and enough information to reject an invalid range. Each input byte updates that state. The result can be one of three conditions: a scalar value completed, more bytes needed, or malformed input.

On an I/O boundary, incomplete is not necessarily invalid. If the stream ends while the decoder still expects bytes, the incomplete sequence becomes a final input error according to the protocol's policy. This mirrors the word-counting state machine from Chapter 1, but with stricter validity requirements.

18.12 Parsing machine-oriented text

Protocol text is not necessarily locale-dependent natural language. A command such as `SET 1234` may be explicitly ASCII with no whitespace variation; a JSON document is UTF-8 with its own grammar and escape sequences. Use a parser matching the format rather than applying broad locale-sensitive rules indiscriminately.

A counted span makes boundaries explicit. For an ASCII command field, scanning exactly its declared number of bytes avoids relying on an unrelated terminator. Reject embedded NUL bytes if the grammar forbids them, even though an ordinary C string function might stop at the first one and ignore trailing bytes.

18.13 Validate before normalizing

Converting malformed byte sequences to lowercase or replacing bytes before validating the input can introduce ambiguity. For format-specific ASCII tokens, validate each byte first, then normalize the allowed subset. For general Unicode text, decoding and normalization need a documented Unicode library and version.

Further problems: Extend the decoder to report incomplete separately from invalid. Produce golden test vectors for one-, two-, three-, and four-byte values. Design a text protocol that rejects embedded NUL and trailing garbage rather than silently ignoring it.

18.14 Separate three kinds of text boundaries

A byte stream has transport boundaries, an encoding has code-unit and code-point boundaries, and a user interface has grapheme or displayed-character boundaries. These are not interchangeable.

A UTF-8 character may be split across two `fread` calls or UART DMA transfers. The decoder must retain the incomplete prefix or the adapter must buffer sufficient bytes. Even after a valid scalar value has been decoded, two adjacent values may combine into a single user-perceived character. A simple MCU command parser often needs only the first two concepts, while a text editor needs all three.

A counted input buffer should always be processed according to its supplied length. A zero byte inside the buffer is valid as a byte value; whether it is allowed by the *protocol* is a separate grammar decision. Using `strlen` on arbitrary counted data can stop early at an embedded zero or read past the allocation if there is no terminator.

18.15 Build the UTF-8 validity proof

For one-byte sequences, the leading byte lies in 00–7F. Valid two-byte sequences begin with C2–DF; C0 and C1 are excluded because they would produce overlong encodings. Valid three-byte sequences begin with E0–EF but need additional restrictions on the second byte or, equivalently, a final decoded-scalar range check. Four-byte sequences begin with F0–F4 and likewise require a result no greater than U+10FFFF.

Every continuation byte must have its top two bits `10`. After combining all payload bits, reject decoded values below the shortest possible value for that sequence length. Reject surrogate code points U+D800–U+DFFF. These tests, in that order or an equivalent validated order, establish that the input represents a Unicode scalar value.

The standalone `utf8_decode.c` handles one code point and reports its consumed length on success. A complete streaming implementation needs an additional status to distinguish a truncated but potentially valid prefix from a definitely malformed sequence.

18.16 Avoid accidental locale dependencies

C's `<ctype.h>` functions operate on character values according to their documented rules and may be locale-sensitive. For a protocol defined in ASCII, the explicit range check `c >= '0' && c <= '9'` may make the grammar clearer, subject to the execution character representation. For raw UTF-8 bytes, interpreting every byte through `tolower` is incorrect Unicode processing.

A production system should choose its textual contract at its boundary: ASCII protocol tokens, UTF-8 source text, or locale-sensitive human input. Once chosen, the parser should not silently switch conventions halfway through processing.

18.17 Bounded streaming text

A command reader with a maximum length of 64 bytes needs both an accumulation buffer and a policy for overflow. If a command exceeds the limit, immediately terminating the connection may be appropriate for a strict protocol. Alternatively, it can discard subsequent bytes until a line delimiter and then resume with the next record.

Do not accept the first 64 bytes as a complete valid command: truncation can change semantics. A long dangerous command might share a harmless-looking prefix with a valid shorter command. Rejecting the entire overlong record preserves the intended grammar.

A streaming decoder also needs an EOF policy: an incomplete multibyte sequence at the end of a stream is not the same thing as clean EOF between complete records.

Exercises

18.7 List valid and invalid UTF-8 leading-byte ranges. **18.8** Build a state machine that distinguishes 'need more bytes' from 'invalid encoding'. **18.9** Explain why NUL may be a valid byte while still being forbidden inside a chosen text protocol. **18.10** Specify the overflow and resynchronization behavior of a bounded line reader.

19. Compile-Time Programming and the Preprocessor

The C preprocessor is a separate, token-oriented facility that runs before ordinary translation. It can select platform-specific code, define constants, construct repetitive expressions, and include headers. Its power makes restraint important: a macro is not a typed function, and conditional compilation should not obscure the program's logic.

19.1 Include files and translation units

A header normally contains declarations and interface-related definitions. A source file includes that header and supplies the corresponding external definitions. Inclusion is conceptually textual: the preprocessor inserts the header's tokens into the translation unit.

```
#ifndef BUFFER_H
#define BUFFER_H
#include <stddef.h>
struct buffer_view { const unsigned char *data; size_t size; };
#endif
```

The guard prevents repeated processing of the same interface within one translation unit. It does not make two incompatible versions of an interface agree or prevent multiple external definitions across translation units.

19.2 Object-like macros

An object-like macro can name a manifest constant:

```
#define BUFFER_CAPACITY 256
```

For a numeric quantity with an appropriate integer type, an enumeration constant or C23 `constexpr` object can be clearer, subject to the required constant-expression context and toolchain support. A macro has no type of its own; it substitutes preprocessing tokens that are interpreted later.

19.3 Function-like macros and double evaluation

```
#define MAXIMUM(a, b) ((a) > (b) ? (a) : (b))
```

This macro can evaluate a chosen argument twice. An invocation involving `i++` is dangerous even if its parentheses are correct. Replacing a macro with a `static inline` function usually improves type checking and evaluation behavior, but an inline function still has to handle overflow and other ordinary C hazards.

19.4 Stringizing and token pasting

The # operator can stringify a macro argument, and ## can paste preprocessing tokens. These mechanisms are helpful for diagnostics and repetitive declarations but can create opaque interfaces.

```
#define STRINGIZE(x) #x
#define CONCAT(a, b) a ## b
```

For example, `STRINGIZE(hello)` produces the string literal `"hello"`. Token-pasting does not create arbitrary source text; its result must be a valid preprocessing token. Expand-once helper macros may be necessary when callers pass macro arguments whose expanded values should be concatenated.

19.5 Conditional compilation

```
#if defined(PLATFORM_DEVICE)
    /* device adaptation */
#elif defined(PLATFORM_HOST)
    /* host adaptation */
#else
    #error "Choose a supported platform"
#endif
```

C23 also provides `#elifdef` and `#elifndef`. Use feature probes where possible: checking a vendor name is not proof that a particular header or library function is available. Keep platform branches behind narrow adapter interfaces so most application code can be compiled identically everywhere.

19.6 C23 embedding and diagnostics

The C23 `#embed` directive is designed to incorporate resource data into a translation unit. It is particularly interesting for fixed firmware resources such as calibration tables or static model parameters. Actual support differs among compilers, so a production project needs a feature probe and a fallback generation step.

C23 also standardizes `#warning`. Diagnostics emitted this way can be useful when a configuration is supported but discouraged. Unlike `#error`, a warning does not necessarily stop translation.

19.7 Static assertions

Use `static_assert` to make fundamental assumptions visible at translation time:

```
#include <limits.h>
static_assert(CHAR_BIT == 8, "octet storage required");
```

The constraint is appropriate for an octet-specific wire format; it would be unnecessarily restrictive for a general-purpose algorithm that does not depend on byte width.

19.8 Generic selection

The `_Generic` selection mechanism works on types rather than token spelling. It can choose an appropriate typed operation:

```
#include <math.h>
#define abs_floating(x) _Generic((x), \
    float: fabsf, double: fabs, long double: fabsl)(x)
```

The selected function takes the value once, avoiding a common multiple-evaluation problem of expression macros. This example requires a supported floating type. A more complete public API should decide how to diagnose unsupported types and document conversion policy.

19.9 An example of compile-time checks

The companion program `examples/compile_time.c` combines `static_assert`, stringizing, token pasting, a C23 empty initializer, and a small function. Its checks are deliberately modest: their purpose is to show that preprocessing and language-level type checking cooperate but are not identical mechanisms.

19.10 Exercises

19.1 Explain why a function-like macro can evaluate an argument more than once.

19.2 Implement two-level stringizing that expands a macro before turning its value into a string.

19.3 Write a configuration header that rejects simultaneous host and device target selections.

19.4 Write a compile probe for `#embed` and document a fallback for a toolchain that lacks it.

19.5 Define the differences among `#if`, `static_assert`, and a runtime `if` statement.

19.6 Use a `_Generic` selection to choose one of two distinct, typed integer-printing functions.

19.11 Preprocessing is not type checking

The preprocessor expands tokens before the compiler evaluates ordinary C types and expressions. This means a macro can construct an expression that looks plausible but has surprising side effects. A typed function receives evaluated arguments once for each parameter, but the order of evaluation among arguments is still not specified merely by the call.

Prefer `enum` constants, appropriately typed `constexpr` objects where supported, and `static inline` functions for tasks that do not require token substitution. Reserve macros for conditional compilation, stringizing, token-pasting, or genuinely generic constructs that cannot be expressed as ordinary C functions.

19.12 Expand macros deliberately

Two-level macros are needed when a macro argument should be expanded before being stringized:

```
#define STR_DIRECT(x) #x
#define STR_EXPANDED(x) STR_DIRECT(x)
#define MAJOR_VERSION 3

const char *version = STR_EXPANDED(MAJOR_VERSION);
```

`STR_DIRECT(MAJOR_VERSION)` yields the identifier spelling, while `STR_EXPANDED(MAJOR_VERSION)` yields `"3"`. Macro expansion rules are precise but easy to misread. Prefer explicit, small helper macros with a test illustrating their purpose.

19.13 Conditional compilation and dead branches

A `#if` directive removes or retains preprocessing tokens before the compiler type-checks ordinary statements. A runtime `if` statement, even if its condition is a constant expression, still belongs to the translated C program and is subject to language constraints.

Use conditional compilation for platform-specific includes, unsupported instructions, or truly unavailable APIs. When both implementations are valid C on the current target, a normal function call or runtime branch may be easier to maintain.

19.14 Using embed responsibly

C23's `#embed` is valuable for incorporating binary resources into source initialization. A resource might contain firmware calibration constants or a model binary. Treat the bytes as untrusted data from a build-input perspective: validate their length, version, and format before interpreting them, and record their provenance.

Embedding data does not necessarily guarantee where the linker places it, whether it resides in ROM or RAM, or whether a given MCU can fetch it efficiently. Such placement is governed by target toolchain and linker configuration. Provide a generated C array fallback for compilers without the directive.

19.15 Static assertions expose assumptions early

A compile-time assertion can check that an integer width, structure-related condition, or application constant meets a documented requirement. It cannot check a runtime packet's length or prove that an allocation will succeed.

C23 accepts `static_assert` directly as a keyword. To detect a feature's availability, compile a tiny representative translation unit with the intended compiler flags rather than assuming support from a marketing claim about C23 compliance.

19.16 Build diagnostics as a teaching aid

The book's `compile_time.c` demonstrates simple macro expansion and C23 initialization. Add variations that intentionally fail `static_assert` in a scratch file and observe the compiler's message. Then compare that translation-time diagnostic to a runtime check that reports an error without preventing compilation.

Further problems: Explain the difference between `STR_DIRECT(VERSION)` and `STR_EXPANDED(VERSION)`. Implement a portable fallback for an embedded byte array. Design a compile-time condition selecting one of two platform adapters and fail explicitly when neither is supported.

19.17 A preprocessor macro is not a generic function

A function-like macro may evaluate an argument more than once, and even a macro wrapped in parentheses does not become type-safe. The familiar `MAX(a, b)` substitution illustrates the danger when a caller passes `index++`. The expression chosen by the conditional operator may be evaluated again.

A `static inline` function for a specific type avoids duplicate evaluation of its arguments. The caller's argument expressions are still evaluated in an order that C does not necessarily prescribe, so do not write calls that depend on sequencing between them.

A `_Generic` selection can choose among several typed functions at translation time, allowing a type-directed wrapper without converting every value to one common type. Be careful to specify supported types and a clear policy for unsupported arguments.

19.18 Use static assertions for real assumptions

A portable byte decoder may not need to assume an eight-bit byte if it operates on abstract values in the standard character domain, but an octet-specific wire codec does need an explicit policy about eight-bit units. A compile-time assertion can make that dependency visible to future maintainers.

Do not scatter assertions about irrelevant architecture properties throughout a general library. A function operating on `size_t` lengths and opaque pointers may be perfectly valid on targets with

different pointer widths, whereas a hardware register interface can have genuine address and alignment requirements.

19.19 Configuration is part of the reproducible build

A macro such as `DEVICE_TARGET` changes which tokens are translated. Therefore it can change program behavior just as surely as a source edit. Record the selected configuration, compiler version, flags, and target alongside test results.

For embedded code, a host adaptation may use a fake device sink, while the MCU build uses a UART driver. The public interface should remain the same; conditional compilation should select the adapter, not duplicate the entire application algorithm.

19.20 Keep the generated book reproducible

The book's PDF is also a build artifact. A manuscript change should trigger generation from the current source and a check that required headings appear in the output. A page count should be measured from the PDF itself, not inferred from the Markdown word count.

The build is only one acceptance gate. A PDF may technically compile but have broken line wrapping, duplicated section numbering, or code examples that run off the page. The final editorial pass requires inspecting rendered pages and checking the technical content independently.

Exercises

19.7 Write a two-level stringizing macro and demonstrate its expansion with a numeric macro. **19.8** Compare a macro and a typed inline function when passed an increment expression. **19.9** Design a compile-time configuration that selects a host or target byte sink. **19.10** Explain why a successful document build is not proof that every C listing compiles.

Appendix A. A Working Reference to C23

This appendix is a navigational guide, not a replacement for ISO/IEC 9899:2024. Implementations differ in optional features and in how far their C libraries have caught up.

A.1 Translation model

A source file is preprocessed and translated as part of a *translation unit*. Header contents are incorporated by inclusion; macro substitution and conditional inclusion precede ordinary compilation. External symbols are resolved when separately compiled translation units are linked. String literals and static objects are initialized according to the rules for their storage duration.

A.2 Fundamental types

- `void` describes absence of a value.
- `bool` is C23's boolean type. `true` and `false` are keywords.
- Character types include `char`, `signed char`, and `unsigned char`. Plain `char` is a distinct type, with implementation-defined signedness.
- Standard signed integer types are `signed char`, `short`, `int`, `long`, and `long long`; each has an unsigned counterpart.
- Floating types include `float`, `double`, and `long double`, with additional optional IEC-related facilities.
- A pointer type refers to an object or function; it does not automatically encode the size or lifetime of the referenced storage.

- Array types contain sequences of elements. Structures contain named members; unions share storage; enumerations provide named integer constants.
- C23 introduces bit-precise integer types written `_BitInt(N)` and `unsigned _BitInt(N)`, subject to implementation limits.

The sizes of fundamental types are not universally fixed. Test the actual target's limits using `<limits.h>`, `<stdint.h>`, and `<float.h>`.

A.3 Expressions at a glance

From highest binding strength to lowest, a practical *simplified* precedence guide:

Level	Operators or forms	Association
Postfix	`[] () . ->` postfix `++ --`	left to right
Unary	prefix `++ --`, `& * + - ! ~`, `sizeof`, `alignof`	right to left
Cast	`(type)`	right to left
Multiplicative	`* / %`	left to right
Additive	`+ -`	left to right
Shift	`<< >>`	left to right
Relational	`< <= > >=`	left to right
Equality	`== !=`	left to right
Bitwise AND	`&`	left to right
Bitwise XOR	`^`	left to right
Bitwise OR	‘	‘
Logical AND	&&	left to right
Logical OR	‘	
Conditional	`?:`	right to left (special grammar rules)
Assignment	‘= += -= *= /= %= «= »= &= ^=	=‘
Comma	,	left to right

Precedence does not imply evaluation order. Thus `f() + g()` does not specify which call happens first. The distinction matters whenever expressions change shared state.

A.4 Declarations

Read a declaration outward from the name, respecting parentheses:

```
int *p;                  /* pointer to int */
int a[8];                /* array of eight int */
int (*fp)(double);       /* pointer to function returning int */
int *make_value(void);   /* function returning pointer to int */
```

A prototype specifies parameter types. In C23, an empty function-parameter list declares a function with no parameters; `(void)` remains explicit and clear. Avoid relying on earlier C conventions for nonprototype declarations.

A.5 Storage, linkage, and qualifiers

Automatic storage normally begins on block entry and ends when execution leaves the block. Static storage lasts for the program execution. Thread storage is associated with a thread where supported. Allocated storage lasts from successful allocation to deallocation.

At file scope, `static` can give an identifier internal linkage; `extern` can declare an externally linked identifier. `const` restricts modification through a qualified lvalue; `volatile` affects observable accesses but does not provide atomic synchronization; `restrict` is a promise about pointer-based accesses and must only be used when its aliasing conditions hold.

A.6 Control statements

`if` chooses a branch; `switch` dispatches among case labels. `for`, `while`, and `do` iterate. `break` exits a loop or switch; `continue` advances to the next loop iteration; `return` exits a function. `goto` transfers control to a label in the same function, but must obey restrictions on entering variably modified scopes and similar rules.

A.7 Behavior categories

- **Defined:** the standard requires specified results.
- **Implementation-defined:** the implementation chooses and documents a permitted result.
- **Unspecified:** the implementation may choose among permitted results.
- **Undefined:** no requirements are imposed; programs must not depend on an observed behavior.

A program may compile without diagnostics and still execute undefined behavior.

A.8 Selected C23 additions

C23 standardizes binary integer literals, digit separators, empty initialization, `nullptr`, `typeof`/`typeof_unqual`, `auto` type inference, `constexpr` objects, bit-precise integers, attributes, `#embed`, `#elifdef`, and `#elifndef`. The library expands with, among other things, checked integer operations and bit utilities. Do not confuse adoption of C23 syntax in a compiler with complete deployment availability of every library feature.

A.9 A disciplined compatibility check

Compile tiny feature probes using the real target compiler and link against the real target C library. Record compiler version, C library version, standard mode, target architecture, and the precise feature probed. Treat a feature as available only after the probe compiles, links, and where possible executes.

A.10 A compact declaration guide

Declarations become much easier to read when they are decomposed into base type, identifier, and type constructors. The following are different contracts:

```
int *p;                 /* pointer to an int */
int values[4];          /* four int elements */
int (*row)[4];          /* pointer to an array of four int */
int *factory(void);     /* function returning pointer to int */
int (*callback)(int);   /* pointer to function taking int */
```

The parentheses decide whether * applies to a named object or to the result of a function declarator. Prefer typedef names for complicated function-pointer interfaces once the meaning is understood; typedef does not create a new distinct type, but it can improve readability.

A.11 Conversions checklist

- **Integer promotions:** narrow integer types are often promoted to `int` or `unsigned int` before arithmetic.
- **Usual arithmetic conversions:** operands in a binary arithmetic operation may be converted to a common type.
- **Assignment conversion:** the right-hand value is converted to the destination type if permitted.
- **Pointer conversions:** object pointers may convert to and from `void *` in the specified circumstances, but pointer arithmetic and access rules still apply.

- **Array-to-pointer conversion:** most array expressions convert to pointers to their first elements, with specified exceptions.

For each conversion, ask whether the mathematical value survives and whether the destination type supports the intended operation. A syntactically valid cast is not proof of a meaningful conversion.

A.12 Alignment and lifetime at a glance

Every complete object type has an alignment requirement. Automatic storage, static storage, thread storage, and allocated storage differ in lifetime rules. A dynamically allocated region can generally hold suitably aligned ordinary objects, but special over-aligned objects may require dedicated allocation facilities or implementation-specific support.

A pointer to a released allocation is no longer a valid way to access the former object. Resetting one alias to `nullptr` does not repair other aliases. A borrowed pointer is safe only while the owner continues to supply a live object.

A.13 C23 compatibility notes

C23's `bool`, `true`, and `false` are keywords. `nullptr` is a null pointer constant with its own type, `nullptr_t`. `static_assert`, `alignof`, and `alignas` also have modern language spellings. A new program need not imitate the macro-based spellings used historically for compatibility.

C23 has no old-style, unprototyped function declaration. Prefer `f(void)` when zero parameters are intended, even though `f()` also means zero parameters in C23. Doing so keeps intent clear when readers encounter older dialects.

The language's bit-precise integer facility, floating-point options, and standard-library additions are not uniformly available in every cross compiler. Test the exact feature on the target, record the version and mode, and provide a fallback when necessary.

A.14 Selected undefined operations

The following are examples to recognize, not experiments whose observed results define the language:

- Out-of-bounds array accesses and invalid pointer dereference.
- Signed integer overflow and integer division by zero.
- Using an automatic object's address after its lifetime ends.
- Violating effective-type or alignment access restrictions.
- Accessing a shared non-atomic object in a data race.
- Using `realloc` with a size argument of zero in C23.
- Performing an invalid shift or violating sequencing rules.

Some implementation-defined behaviors, by contrast, are perfectly legitimate once the implementation documents them and the program accepts that platform dependency.

A.15 References for the careful reader

The normative authority is ISO/IEC 9899:2024. For accessible navigation and implementation details, consult the C language pages at cppreference, GCC's C standards support page, and Clang's C status page. Those references help locate a topic but do not replace the exact standard text when a subtle conformance question matters.

Useful online links:

- https://en.cppreference.com/w/c/23
- https://en.cppreference.com/w/c/language

- https://gcc.gnu.org/projects/c-status.html
- https://clang.llvm.org/c_status.html

Appendix B. The Standard Library by Task

This is a field guide to the most commonly needed hosted C facilities, not a complete list of the standard's clauses.

Need	Header	Key facilities
Input, output, files	`<stdio.h>`	`FILE`, `fopen`, `fclose`, `fread`, `fwrite`, `fgets`, `printf`
Byte and string operations	`<string.h>`	`memcpy`, `memmove`, `memcmp`, `strlen`, `strcmp`
Allocation, conversion	`<stdlib.h>`	`malloc`, `calloc`, `realloc`, `free`, `strtol`, `qsort`
Integer limits	`<limits.h>`	`INT_MAX`, `CHAR_BIT`, etc.
Exact-width integer types	`<stdint.h>`	`uint32_t`, `SIZE_MAX` and related types/macros
Integer formatting	`<inttypes.h>`	`PRIu32`, `SCNx64` etc.
Type and offset	`<stddef.h>`	`size_t`, `ptrdiff_t`, `offsetof`
Character classification	`<ctype.h>`	`isdigit`, `isspace`, `tolower`
Error indications	`<errno.h>`	`errno`, `ERANGE`
Mathematics	`<math.h>`	`sqrt`, `sin`, `fabs`, `isfinite`
Assertions	`<assert.h>`	`assert`
Variadic functions	`<stdarg.h>`	`va_list`, `va_start`, `va_arg`, `va_end`
Atomics	`<stdatomic.h>`	atomic types and operations
Threads	`<threads.h>`	`thrd_t`, mutexes, condition variables where supported
Checked integer arithmetic	`<stdckdint.h>`	`ckd_add`, `ckd_sub`, `ckd_mul` (C23)
Bit utilities	`<stdbit.h>`	leading/trailing bit counts, population count, bit width, and bit

B.1 Length is not capacity

`strlen` finds a terminating zero byte. It does not accept a buffer size, so calling it on a nonterminated array is unsafe. Functions like `snprintf` take output capacity, and their return values should be checked to detect truncation.

```
char buffer[16];
int needed = snprintf(buffer, sizeof buffer, "%u", 12345U);
if (needed < 0 || (size_t)needed >= sizeof buffer) {
    /* formatting error or truncation */
}
```

B.2 Integer conversion

`strtol` is usually preferable to `atoi` when handling untrusted text. Set `errno = 0` before conversion, inspect the end pointer, and check `ERANGE`. Even after a successful `long` result, verify the destination type's range.

B.3 Byte copy versus overlapping copy

`memcpy` requires source and destination regions not to overlap. `memmove` permits overlap, but both require valid regions of at least the requested size. Neither performs object-bound checks for you.

B.4 Character conversion

The `<ctype.h>` functions accept `EOF` or a value representable as `unsigned char`. Passing a negative signed `char` directly is generally invalid. For byte-oriented classification, cast:

```
unsigned char byte = (unsigned char)some_char;
if (isdigit(byte)) { /* ... */ }
```

The character functions can be locale-sensitive. They are not general Unicode text processing primitives.

B.5 Optionality and freestanding environments

A freestanding embedded system may implement only a subset of the hosted library. Standard features may also depend on toolchain, linker, and C library combinations. Document whether examples require a hosted environment, and test all target-specific functions directly.

B.6 A checked numeric conversion

A typical `strtol` call is more careful than merely checking whether the return value is zero:

```
#include <errno.h>
#include <limits.h>
#include <stdbool.h>
#include <stdlib.h>

bool parse_positive_int(const char *text, int *out)
{
    if (text == nullptr || out == nullptr)
        return false;
    errno = 0;
    char *end;
    long value = strtol(text, &end, 10);
    if (end == text || *end != '\0' || errno == ERANGE ||
        value < 0 || value > INT_MAX)
        return false;
    *out = (int)value;
    return true;
}
```

This accepts leading whitespace and an optional plus sign because `strtol` does. Despite its name, it also accepts zero; rename it `parse_nonnegative_int` if the name is meant to describe the precise grammar. It rejects trailing text. The output is unchanged on failure.

B.7 Formatted-output contracts

With `printf`, each conversion specification must match the argument's promoted type. For `size_t`, use `%zu`; for a `uint32_t` whose formatting differs across platforms, use the macros supplied by `<inttypes.h>`.

A format string obtained from outside the program can be dangerous: passing it as the first argument may introduce unexpected conversion directives. To print user-controlled text, use a fixed format such as `printf("%s", input)` when a valid terminated string is guaranteed; use a counted-byte write operation for untrusted spans without terminators.

For `snprintf`, a nonnegative return value is the number of characters that would have been written, excluding the terminating zero. If that number is at least the supplied capacity, output was truncated. A negative result signals an encoding or formatting error under the interface's rules.

B.8 Searching and sorting

`qsort` arranges objects in an array using a comparator. The comparator must interpret the correct object type and provide a consistent ordering. For integers, the result of subtracting the two

values is not always representable; use relational comparisons.

`bsearch` assumes the array is sorted consistently with the supplied comparator. It does not validate this precondition. If records with equal keys have distinct associated data, the API contract must say whether any matching record is acceptable.

B.9 Allocation and zeroed storage

`malloc` allocates uninitialized storage; `calloc` writes zero bytes to its allocated region; `realloc` may resize and move an allocation. Check all requested-size calculations before allocation. In C23, `realloc(p, 0)` is undefined and must not be used as a portable deallocation idiom.

All-bits-zero is not guaranteed to represent a null pointer or floating-point zero on every implementation. Ordinary initializer syntax such as `struct value v = {};` sets members according to the language's initialization rules, which is a different concept from a raw byte memset.

B.10 C23 bit utilities, precisely

The new `<stdbit.h>` header supplies operations including `stdc_leading_zeros`, `stdc_trailing_zeros`, `stdc_count_ones`, `stdc_has_single_bit`, `stdc_bit_width`, `stdc_bit_floor`, and `stdc_bit_ceil`, plus related variants and byte-order macros.

It does **not** provide the C++ `std::rotl` and `std::rotr` interface. If you need rotations in C, write a properly bounded unsigned helper or use a documented implementation extension.

The associated header `<stdckdint.h>` supplies checked integer arithmetic macros. In a practical project, test whether both headers exist and link successfully with the intended C library before treating them as dependencies.

B.11 Platform limitations

C's hosted library includes facilities that may be absent or restricted in freestanding firmware. A microcontroller environment may use retargeted `printf`, a specialized allocator, or no filesystem at all. In such a system, keep the byte-level transformation algorithms separate from device transport and I/O.

Threads and atomics also depend on implementation support. Never assume that a function which works on a desktop is safe to call inside an interrupt handler.

B.12 Error policy examples

When a library function signals failure with a sentinel, immediately consult the documented mechanism for distinguishing failure from a normal result. For `fread`, inspect `ferror` and `feof` following a short read. For `strtol`, inspect `errno` and the end pointer. For `malloc`, check the returned pointer while remembering that zero-size requests have special semantics.

Practice: Create a small table containing every library function used in your program, its return-value meaning, and your application's chosen behavior on failure.

Appendix C. Selected Worked Exercises

These solutions illustrate contracts and tests rather than prescribe one clever implementation.

C.1 Absolute difference without unsigned underflow

```
unsigned difference(unsigned a, unsigned b)
{
```

```
    return a >= b ? a - b : b - a;
}
```

Both subtractions occur only in nonnegative order. Test `(0, 0)`, `(1, 0)`, `(0, UINT_MAX)`, and both operand orders.

C.2 Checked parsing of an unsigned decimal

```
#include <stdbool.h>
#include <limits.h>

bool parse_unsigned(const char *text, unsigned *out)
{
    if (text == nullptr || out == nullptr || *text == '\0')
        return false;

    unsigned value = 0;
    for (const unsigned char *p = (const unsigned char *)text;
         *p != '\0'; ++p) {
        if (*p < '0' || *p > '9') return false;
        unsigned digit = (unsigned)(*p - '0');
        if (value > (UINT_MAX - digit) / 10U) return false;
        value = value * 10U + digit;
    }
    *out = value;
    return true;
}
```

The key inequality is evaluated *before* a potentially overflowing multiplication. The output is unchanged on failure. This version accepts no leading sign or whitespace, by design.

C.3 Iterative factorial with an overflow check

```
#include <stdbool.h>
#include <limits.h>

bool factorial(unsigned n, unsigned long *out)
{
    if (out == nullptr) return false;
    unsigned long result = 1;
    unsigned i = 2;
    while (i <= n) {
        if (result > ULONG_MAX / i) return false;
        result *= i;
        if (i == n) break;
        ++i;
    }
    *out = result;
    return true;
}
```

The explicit final-iteration break also avoids unsigned loop-counter wraparound if `n == UINT_MAX`. Checking multiplication alone is not the whole correctness proof.

C.4 Little-endian byte decoding

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

bool read_u32_le(const unsigned char *bytes, size_t count, uint32_t *out)
{
    if (bytes == nullptr || out == nullptr || count < 4) return false;
    uint32_t result = (uint32_t)bytes[0]
                    | ((uint32_t)bytes[1] << 8)
                    | ((uint32_t)bytes[2] << 16)
                    | ((uint32_t)bytes[3] << 24);
    *out = result;
    return true;
}
```

The cast happens before shifting, so promotions do not make the top byte shift a signed overflow. Endianness is described by the wire format, not assumed from host memory.

C.5 Cleanup on every path

```
#include <stdio.h>
#include <stdbool.h>

bool write_record(const char *path)
{
    FILE *fp = fopen(path, "w");
    if (fp == nullptr) return false;
    bool ok = fputs("record\n", fp) != EOF;
    if (fclose(fp) == EOF) ok = false;
    return ok;
}
```

Notice that `fclose` runs even after `fputs` fails. The function reports stream-level success, not guaranteed durable storage on physical media.

C.6 A safe interval check

An interval starting at `offset` and containing `count` bytes fits within `capacity` when the start is valid and the count fits the remaining space:

```
#include <stddef.h>
#include <stdbool.h>

bool fits(size_t offset, size_t count, size_t capacity)
{
    return offset <= capacity && count <= capacity - offset;
}
```

The short-circuit operator prevents unsigned subtraction from being evaluated if `offset` already exceeds capacity. Test zero capacity, exact fit, one-byte overrun, and `SIZE_MAX` boundaries.

C.7 Checked allocation dimensions

```
#include <stdint.h>
#include <stddef.h>
```

```
#include <stdlib.h>
#include <stdbool.h>

bool allocate_ints(size_t count, int **out)
{
    if (out == nullptr || count == 0 ||
        count > SIZE_MAX / sizeof(int))
        return false;
    int *p = malloc(count * sizeof *p);
    if (p == nullptr)
        return false;
    *out = p;
    return true;
}
```

The function rejects zero-sized requests and leaves the output unchanged on failure. The caller owns the returned allocation and must call `free` once. The result is not initialized; assign to elements before reading them.

C.8 A consistent qsort comparator

```
static int compare_ints(const void *left, const void *right)
{
    int a = *(const int *)left;
    int b = *(const int *)right;
    return (a > b) - (a < b);
}
```

Returning `a - b` could overflow for extreme signed values, invalidating the intended ordering. Use the comparator with `qsort` only when both pointers refer to valid `int` elements supplied by that call.

C.9 A non-modifying failure test

Suppose a parser promises not to modify its output on failure. Before invoking it on an invalid string, set `unsigned result = 0x5a5aU;`. Verify the parser returns false *and* that the result remains `0x5a5aU`. Repeat with empty input, trailing junk, and an out-of-range number. A failed result alone does not establish the postcondition.

C.10 A boundary test plan for a ring buffer

Initial state	Operation	Required behavior
Empty	Pop	Report false; do not change output
Empty	Push	Report true; count increases
One element	Pop	Return first item; become empty
Full	Push	Report false; preserve existing items
Partially consumed	Wraparound pushes	FIFO order is preserved
Returned to empty	Pop	Report false again

Sequential tests establish logical behavior, not thread safety. The atomic SPSC queue in Chapter 16 needs additional concurrent reasoning and testing.

C.11 How to check an exercise solution

For each solution, write down preconditions, postconditions on both success and failure, possible arithmetic overflow, and ownership effects. Then test exact boundaries. If your implementation uses a library function, read the library contract rather than assuming it matches the function's name.

Solutions are examples of reasoning, not necessarily the only acceptable design. An alternative is valid when it meets the same explicit contract.

C.12 A counted slice and its ownership contract

A counted slice has a data pointer and a length. It does not automatically own the referenced array. The caller must keep storage alive for any operation reading the slice.

```
#include <stddef.h>
#include <stdbool.h>

struct int_slice {
    const int *data;
    size_t length;
};

bool slice_get(struct int_slice s, size_t index, int *out)
{
    if (out == nullptr || s.data == nullptr || index >= s.length)
        return false;
    *out = s.data[index];
    return true;
}
```

On failure, the function leaves `*out` unchanged. It requires that a nonempty slice identify at least `length` accessible elements; a misleading length is a caller contract violation, not something a simple C function can discover from the pointer.

A subslice also needs to avoid unsigned overflow. Its valid-range test should be `offset <= original.length && count <= original.length - offset`, not `offset + count <= original.length`. With an empty original whose data pointer is null, return a null pointer for an empty result rather than attempting arithmetic on a null pointer.

The complete program in `examples/slices.c` exercises these rules, including `SIZE_MAX` and one-past positions.

C.13 Failure-injected vector growth

The vector in `examples/vector_checked.c` owns its element allocation. An append operation uses a function pointer to a resizing function so a test can force reallocation to fail at a chosen point.

```
typedef void *(*resize_fn)(void *, size_t);

static void *deny_resize(void *old, size_t bytes)
{
    (void)old;
    (void)bytes;
    return nullptr;
}
```

After filling the vector's first allocation, save the pointer, length, capacity and last element. Invoke append with `deny_resize`. A correct strong-failure implementation returns false without changing

any saved property.

The second test repeats the append with a real `realloc` implementation; it then verifies the new value, the count, and safe destruction. Crucially, it does not use the old saved pointer after successful reallocation, because that pointer may be invalid even when its bit pattern looks unchanged.

This injection technique can be extended to `malloc` and to fake I/O sources. It is more reliable than hoping the operating system happens to run out of memory during a small test.

C.14 Explain a failing test

A failed assertion is a starting point for diagnosis. Before modifying source, identify whether the program violated its own contract, the test made an invalid assumption, or the compiler did not implement the chosen language/library facility.

For example, a compiler warning that classifies a standardized C23 binary literal as a vendor extension does not automatically mean that the source is nonconforming. The response may be to use a more recent compiler or to suppress that one incorrect diagnostic while retaining other warnings. By contrast, using a macro removed in C23 is a source compatibility problem and should be corrected in the example itself.

A good verification record contains the failing command, exact diagnostic, cause, smallest corrective change, and the result of running tests on the corrected version. Avoid claiming an entire book has been verified when only a single executable has been compiled.

C.15 A bounded frame encoder and decoder

A frame format should be described in bytes before it is represented by C types. The Chapter 14 example uses a magic octet, version octet, two little-endian length octets, zero to 64 payload octets, and an XOR checksum.

The encoder's preconditions include a writable destination of the declared capacity and a readable payload of the declared length. It rejects payloads longer than 64 and destinations smaller than `5 + length`. Because the maximum length is already fixed at 64, the total frame size can be proven representable on every target supported by the example. It constructs the complete frame in a local candidate buffer and then copies it to the caller's destination; this preserves the destination on validation failure.

The decoder checks fixed header size before reading fields, validates format identifiers, verifies that the declared payload matches the complete input length, and checks the checksum. Only after every check passes does it publish its local candidate result.

Testing: Start with a manually computed empty frame, then test a full 64-byte payload. Corrupt one payload byte without updating the checksum. Truncate a frame by one byte. Supply an extra byte. Confirm the output remains unchanged after rejection. The complete program is `examples/frame_codec.c`.

An XOR checksum offers limited accidental error detection, not sender authentication. A stronger checksum such as a CRC addresses some error patterns but is also not cryptographic authentication.

C.16 A partial-write callback

The byte sink in Chapter 17 returns the number of bytes accepted in a call. A complete-write wrapper must never advance by more than the remaining count, and must stop when the sink reports no progress.

Let `offset` be bytes successfully accepted and `count` the original request. Maintain the invariant `offset <= count`. On each loop iteration, ask the sink for `count - offset` bytes starting at `data`

`+ offset`. Reject a returned value of zero or a value greater than requested. Otherwise add the positive count to `offset`.

Every successful iteration strictly increases `offset`, proving termination for a bounded input. This proof says nothing about how long a callback blocks; an RTOS deadline requires additional assumptions about the device operation.

The program `examples/callback_sink.c` uses a fake writer accepting at most two bytes per call. A five-byte message therefore requires three calls. A second mode always returns zero so the wrapper can demonstrate failure without looping indefinitely.

An asynchronous writer needs a richer design: a 'would block' result may be normal, and the wrapper may have to retain state across calls. Never use the synchronous implementation as an automatic substitute for an interrupt-safe or nonblocking API.

C.17 Separate compilation as a testable contract

The module example consists of `examples/modules/checksum.h`, `checksum.c`, and `main.c`. The header declares a function taking a byte pointer, count, and output pointer. The implementation includes that header so an incompatible definition is diagnosed at compile time. The client also includes the same header and verifies that an invalid argument leaves the result unchanged.

The checksum is intentionally XOR, which is simple enough that the program's structure is more instructive than the calculation. The example should not be mistaken for a cryptographic hash.

A useful extra test is to compile `main.c` without linking `checksum.c`; the expected undefined-reference error illustrates the linker's job. Then compile `checksum.c` by itself with `-c`, compile `main.c` similarly, and link the object files.

A public header should include its own dependencies. It must not require clients to include `<stddef.h>` or `<stdbool.h>` before including it.

C.18 What an example demonstrates—and what it does not

A passing byte-sink test demonstrates that the wrapper handles its tested partial-write and no-progress cases. It does not prove that a real UART driver handles DMA coherency or interrupt priority correctly. A passing frame-decoder test demonstrates behavior on its supplied packets, not exhaustive validation of every possible byte sequence. A passing module example demonstrates correct linkage under the chosen build command, not platform independence of every possible implementation.

For each exercise solution, separate observed test results from the assumptions and reasoning needed to use it safely in another environment. This distinction is essential in production embedded systems.

C.19 Review checklist for the three complete examples

When reviewing `frame_codec.c`, verify that no length or checksum byte is read before its bounds check. Compare the decoded frame length with the precise number of supplied bytes, not merely with buffer capacity. Confirm that the encoder's temporary frame fits the fixed maximum and that a failed encode never writes into the caller's destination. The XOR checksum is not a security guarantee.

For `callback_sink.c`, prove that `offset` starts at zero, never exceeds the requested count, and increases on every successful loop iteration. A zero-byte callback result must terminate with failure rather than create a busy loop. The callback's promise not to return more than requested is checked; its promise to read only the provided source span remains part of the callback contract.

For the checksum module, compile the header in isolation and verify that it declares every type it needs. Link a test client with the implementation file, then intentionally omit the implementation

and observe the expected linker failure. These exercises distinguish a language error, interface error, runtime test failure, and missing link dependency.

The important lesson is that tests, code review, and explicitly written invariants reinforce each other. A short, compile-tested demonstration can still require careful reasoning before its logic is reused in embedded production code.

Appendix D. Glossary of Modern C

A concise language benefits from precise terminology. The words below are used consistently throughout the manuscript.

Abstract machine. The language-defined model against which program behavior is judged. A compiler may transform code while preserving required observable behavior.

Alignment. A requirement constraining where an object of a particular type may be stored. Misaligned accesses can have undefined behavior.

Array. A contiguous sequence of elements of one element type. In many expression contexts an array converts to a pointer to its first element; this does not make arrays and pointers identical types.

Automatic storage duration. Storage normally associated with a block invocation and released when its lifetime ends.

Binary interface (ABI). Platform-specific conventions governing calling conventions, object layout and linkage. A C source declaration does not fix one universal ABI.

Borrowed pointer. A pointer whose use depends on an object owned or managed elsewhere. The pointer does not itself extend the object's lifetime.

Buffer capacity. The number of elements or bytes that an allocated or provided buffer may hold. Distinguish it from the number currently containing meaningful data.

Callback. A function supplied to another component for invocation under a documented protocol. Callbacks may need a context pointer and lifetime rules.

Checked arithmetic. An operation that detects whether a mathematical result fits the destination type before relying on its value.

Compile-time assertion. A translation-time constraint written with `static_assert` or a compatible form.

Data race. Conflicting access to a shared memory location by concurrent threads without the synchronization required by C's memory model, where at least one access modifies it and the accesses are non-atomic. Such a race has undefined behavior.

Effective type. Rules determining which lvalue types may be used to access stored objects. Relevant to aliasing optimizations and representation reinterpretation.

Endianness. The byte ordering used for a multi-byte representation. File formats and wire protocols must define their own byte ordering rather than assume the host's.

Freestanding implementation. An implementation environment, often embedded, with different entry-point and library obligations from a hosted environment.

Half-open range. An interval that includes its start and excludes its end, conventionally `[begin, end)`. This representation makes length equal to end minus begin when both positions belong to a valid common domain.

Implementation-defined behavior. Behavior for which the implementation must choose and document a permitted alternative.

Invariant. A property preserved across the operations of an algorithm or data structure. For a vector, `length <= capacity` is fundamental.

Lifetime. The period during which an object exists. A pointer can retain an old bit pattern after its target object's lifetime has ended, but that does not make dereferencing it valid.

Lvalue. An expression designating an object or function under the rules of C. Not every lvalue is modifiable.

Object representation. The bytes of an object's stored representation. Copying them does not automatically define a portable external serialization format.

Opaque type. A type whose representation is hidden from the client, commonly through an incomplete structure declaration and operations accepting pointers.

Ownership. An application-level convention identifying which component is responsible for releasing a resource. C does not enforce a unique-owner discipline automatically.

Precondition. A condition the caller must satisfy before calling an operation, such as providing a valid pointer and sufficient length.

Postcondition. A property guaranteed when an operation finishes under its documented conditions, including what happens on failure.

Release/acquire synchronization. Atomic memory ordering used to publish data and establish visibility of earlier writes to a reader that observes the publication.

Restrict. A pointer qualification expressing an access promise enabling optimization when its conditions are respected; not a runtime alias check.

Sentinel. A distinguished value such as `EOF` indicating a condition outside the normal result domain.

Signed overflow. Evaluating a signed arithmetic result not representable in the result type. This remains undefined behavior in C23.

Static storage duration. Storage that persists for program execution, regardless of whether the identifier has file scope or block scope.

Stream. C's hosted abstraction for a sequence of input or output characters or bytes, often associated with a terminal or file.

Translation unit. The result of preprocessing one source file, including its headers, that is compiled as a unit.

Undefined behavior. A program action for which the standard imposes no requirements. An observed result in one build provides no portable contract.

Unspecified behavior. Behavior for which the standard allows more than one alternative without requiring the implementation to document its choice.

Volatile. A qualifier affecting accesses with observable side effects. It is not a general replacement for atomics or mutexes.

This glossary is deliberately selective. The ISO standard supplies precise normative definitions, sometimes with narrower conditions than a one-sentence teaching description can express.

Appendix E. Conformance and Migration Checklist

This appendix separates three questions commonly confused in modern C work: **What does C23 specify? What does the compiler implement? What does the target environment provide?**

E.1 Identify the build

Record the C compiler vendor, version, language-mode option, target triple, C library version, linker and startup/runtime configuration. A hosted GCC build on a development machine does not establish the capabilities of an Arm cross-compiler paired with a reduced runtime library.

A language-version probe can inspect `__STDC_VERSION__`. Its C23 value is `202311L`, but this alone does not prove that every new library facility is available.

E.2 Core-language probes

Use tiny independent translation units to test the exact construct you plan to depend on. Features worth testing separately include:

- C23 boolean and null-pointer keywords: `bool`, `true`, `false`, `nullptr`.
- Empty initialization with `{}`.
- Binary integer literals with `0b` and quote digit separators.
- `auto` deduction, `typeof` and `typeof_unqual`.
- `constexpr` objects.
- `_BitInt(N)` within the required target width.
- Standard attribute syntax.
- The `#embed` directive for required binary resources.
- `#elifdef` and `#elifndef` conditions.

A successful syntax test is not an execution test. For numerical and representation-sensitive features, execute a small representative test on the target when possible.

E.3 Standard-library probes

C23's `<stdckdint.h>` and `<stdbit.h>` deserve independent include-and-link checks. A compiler may accept `-std=c23` while the selected C library does not provide them. The same caution applies to optional threading and atomic facilities in a freestanding environment.

For the checked arithmetic library, write a probe that calls `ckd_add` with representable and overflowing operands. For bit utilities, test a representative `stdc_count_ones` use. Avoid assuming availability because a header file with the same name happens to exist; compile and link the intended operation.

E.4 Representation assumptions

Record `CHAR_BIT`, integer widths, alignment requirements, and any assumptions about floating-point formats. Use static assertions only where the application requires the constraint. A text-only algorithm need not reject machines with unusual byte widths simply because a different example in the book requires eight-bit octets.

An external protocol should define field widths, byte order, and validation rules. Do not serialize native structs by writing `sizeof(struct)` bytes unless the external format explicitly specifies the exact local ABI as its contract.

E.5 A practical migration order

1. Build the existing code under its original supported standard and save test evidence.
2. Enable warnings and fix invalid declarations, mismatched prototypes, and ignored errors.
3. Audit signed arithmetic, narrowing conversions, pointer lifetime, and buffer-size calculations.
4. Remove reliance on old-style function declarations and other obsolescent assumptions.
5. Move to C23 mode without changing algorithms.
6. Introduce modern spelling and features selectively where the tested toolchains support them.

7. Run differential tests against the original behavior for valid inputs.
8. Validate on each real embedded target and document any implementation-defined dependencies.

A source-level conversion that compiles is not a complete migration. The program's runtime contract, memory usage and timing must still be verified.

E.6 Deployment acceptance checklist

Before accepting a release, confirm that the build uses the intended source commit; all shipped features passed their targeted probes; test inputs cover boundaries and failure cases; the compiler and linker warnings were reviewed; sanitizer results, where available, are recorded; and hardware-dependent code was exercised on real hardware.

The document build has its own acceptance criteria: all chapters and appendices appear, headings render in the correct hierarchy, the contents links work, code listings are legible, and the PDF's page count is measured directly. A PDF file existing on GitHub does not by itself prove that it corresponds to the latest manuscript commit.

E.7 Authoritative resources

- ISO/IEC 9899:2024, *Programming languages — C* (normative reference)
- https://en.cppreference.com/w/c/23 — overview and links to individual language/library topics
- https://gcc.gnu.org/projects/c-status.html — GNU C standard support
- https://clang.llvm.org/c_status.html — Clang language support

Library and compiler documentation change over time. Date your conformance matrix and retest after updating any part of a deployment toolchain.